

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр»
за спеціальністю 121 «Інженерія програмного забезпечення»
(освітня програма «Інженерія програмного забезпечення»)

на тему:

**«Вебплатформа для торгівлі годинниками та аксесуарами з підтримкою
аукціону та обміну»**

Виконав студент групи ІПЗ-22-4
МІНАЙЛЕНКО Віктор Юрійович

Керівник роботи:
ЄФРЕМОВ Юрій Миколайович

Рецензент:
САВІЦЬКИЙ Роман Святославович

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

«ЗАТВЕРДЖУЮ»

Зав. кафедри інженерії
програмного забезпечення
Тетяна Вакалюк
«28» лютого 2026 р.

ЗАВДАННЯ
на кваліфікаційну роботу

Здобувач вищої освіти: МІНАЙЛЕНКО Віктор Юрійович

Керівник роботи: ЄФРЕМОВ Юрій Миколайович

Тема роботи: **«Вебплатформа для торгівлі годинниками та аксесуарами з підтримкою аукціону та обміну»,**

затверджена Наказом закладу вищої освіти від **«27» лютого 2026 р., № 92/с**

Вихідні дані для роботи: предметом дослідження є процес взаємодії користувачів із вебплатформою для торгівлі годинниковими аксесуарами та механізми організації аукціонних торгів у вебсередовищі;

Об'єктом дослідження є процеси проектування, розгортання та тестування вебплатформ електронної комерції з модулями торгів у реальному часі.

Консультанти з бакалаврської кваліфікаційної роботи із зазначенням розділів, що їх стосуються:

Розділ	Консультант	Завдання видав	Завдання прийняв
1	Єфремов Ю. М.	01.03.2026р.	01.03.2026р
2	Єфремов Ю. М.	01.03.2026р	01.03.2026р
3	Єфремов Ю. М.	01.03.2026р	01.03.2026р

РЕФЕРАТ

Випускна кваліфікаційна робота бакалавра виконана на тему «Вебплатформа для торгівлі годинниками та аксесуарами з підтримкою аукціону та обміну». Робота складається з пояснювальної записки та розробленого додатку. Пояснювальна записка містить вступ, три розділи, висновки та перелік використаних джерел. Загальний обсяг звіту становить 128 сторінок, робота містить 81 рисуноків, 25 таблиць, 30 джерел літератури та 2 додатки.

У звіті розглянуто предметну область електронної комерції у сфері продажу наручних годинників та аксесуарів до них. Проаналізовано особливості функціонування сучасних онлайн-маркетплейсів і спеціалізованих платформ, виявлено їхні переваги та недоліки. Визначено архітектуру та технологічний стек для розробки власного програмного рішення.

На підставі проведеного аналізу сформовано вимоги до програмної платформи RoyalTick, що реалізує повний цикл електронної торгівлі: від перегляду каталогу годинників і аксесуарів до оформлення замовлення з подальшими можливостями розширення функціоналу аукціону та обміну.

Ключові слова: ВЕБПЛАТФОРМА, ЕЛЕКТРОННА КОМЕРЦІЯ, ГОДИННИКИ, NEXT.JS, MONGODB, TYPESCRIPT, ОНЛАЙН-МАГАЗИ

					ІПЗ.КР.Б – 121 – 26 – ПЗ			
Змн.	Арк.	№ докум.	Підпис	Дата	Вебплатформа для торгівлі годинниками та аксесуарами з підтримкою аукціону та обміну	Літ.	Арк.	Аркушів
Розроб.		Мінайленко В.Ю						
Перевір.		Єфремов Ю.М.					2	167
Керівник						Житомирська політехніка, група ІПЗ-22-4		
Н. контр.		Тичина О.П.						
Зав. каф.		Вакалюк Т.А.						

ABSTRACT

The bachelor's thesis is dedicated to the topic "Web Platform for Watch and Accessory Trading with Auction and Exchange Support." The work consists of an explanatory note and the developed application. The explanatory note includes an introduction, three chapters, conclusions, and a list of references. The total volume of the report is 128 pages, and the work contains 81 figures, 25 tables, 30 literary sources, and 2 appendices.

The report examines the subject area of e-commerce in the field of wristwatches and related accessories. The operational features of modern online marketplaces and specialized platforms are analyzed, and their advantages and disadvantages are identified. The system architecture and technology stack for developing a custom software solution are defined.

Based on the analysis performed, the requirements for the software platform RoyalTick have been formulated. This platform implements a full e-commerce cycle, ranging from browsing the watch and accessory catalog to placing an order, with further opportunities to expand the auction and exchange functionality.

Keywords: WEB PLATFORM, E-COMMERCE, WATCHES, NEXT.JS, MONGODB, TYPESCRIPT, ONLINE STOR

					ІПЗ.КР.Б – 121 – 26 – ПЗ			
Змн.	Арк.	№ докум.	Підпис	Дата	Вебплатформа для торгівлі годинниками та аксесуарами з підтримкою аукціону та обміну	Літ.	Арк.	Аркушів
Розроб.		Мінайленко В.Ю						
Перевір.		Єфремов Ю.М.					2	167
Керівник						Житомирська політехніка, група ІПЗ-22-4		
Н. контр.		Тичина О.П.						
Зав. каф.		Вакалюк Т.А.						

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	6
ВСТУП.....	7
РОЗДІЛ 1. АНАЛІЗ ВИКОРИСТАННЯ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ	9
1.1. Постановка завдання.....	9
1.2. Аналіз аналогів програмного продукту	11
1.3. Обґрунтування вибору архітектури системи	19
1.4. Обґрунтування вибору інструментальних засобів та вимоги апаратного забезпечення	26
Висновки до першого розділу.....	30
РОЗДІЛ 2. ПРОЄКТУВАННЯ ВЕБПЛАТФОРМИ ДЛЯ ТОРГІВЛІ ГОДИННИКОВИМИ АКСЕСУАРАМИ З АУКЦІОННИМ МОДУЛЕМ	31
2.1. Визначення варіантів використання та об'єктно-орієнтованої структури системи	31
2.2. Розробка бази даних системи.....	42
2.3. Проєктування та реалізація алгоритмів роботи системи	66
2.4. Реалізація функціоналу вебплатформи royaltick	77
2.5. Контейнеризація та автоматизація розгортання	81
Висновки до другого розділу.....	84
РОЗДІЛ 3. РОЗГОРТАННЯ, ІНТЕРФЕЙС ТА ТЕСТУВАННЯ ВЕБПЛАТФОРМИ ROYALTICK.....	85
3.1. Порядок розгортання та налаштування системи	85
3.2. Структура інтерфейсу вебплатформи	88
3.3. Тестування вебплатформи та аналітика	117
Висновки до третього розділу	123
ВИСНОВКИ	125
ДЖЕРЕЛА ІНФОРМАЦІЇ.....	126
ДОДАТКИ.....	129

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API – прикладний програмний інтерфейс (Application Programming Interface)

БД – база даних

ПЗ – програмне забезпечення

UI/UX – інтерфейс та досвід користувача (User Interface / User Experience)

CRUD – базові операції керування даними (Create, Read, Update, Delete)

SSR – серверний рендеринг (Server-Side Rendering)

SSG – статична генерація сторінок (Static Site Generation)

JWT – JSON Web Token

ODM – об'єктно-документний маппер (Object Document Mapper)

SEO – пошукова оптимізація (Search Engine Optimization)

SCSS – Sassy CSS, препроцесор каскадних таблиць стилів

CI/CD – безперервна інтеграція та доставка (Continuous Integration / Continuous Delivery)

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		6

ВСТУП

Актуальність теми. Ринок годинників та аксесуарів переживає активну трансформацію в онлайн-середовище. Зростання інтересу до колекціонування, збільшення кількості приватних продавців та стійкий попит на майданчики для перепродажу годинників формують запит на спеціалізовані з розвиненим функціоналом. Проте попри очевидний попит, більшість доступних рішень не здатні повноцінно задовольнити потреби цієї аудиторії. Універсальні маркетплейси є надто загальними вони не мають спеціалізованих фільтрів для пошуку за механізмом, матеріалом корпусу, діаметром або брендом, що робить процес пошуку конкретного аксесуара незручним і тривалим. Тематичні форуми та спільноти, попри свою спеціалізованість, є технічно застарілими, не захищають учасників від шахрайства і не мають жодних інструментів для безпечного укладання угод. Інші платформи орієнтовані виключно на преміальний сегмент і є недоступними для більшості українських користувачів через відсутність підтримки гривні та локальних платіжних систем. Окремою проблемою є повна відсутність прозорих аукціонних механізмів на українському ринку. Аукціон є природним і об'єктивним способом визначення справедливої ринкової вартості раритетних або унікальних годинників, однак жодна з наявних платформ не пропонує повноцінного інструменту для проведення відкритих торгів з верифікованими учасниками та захищеним каналом комунікації між сторонами угоди. Саме тому розробка спеціалізованої вебплатформи RoyalTick, яка поєднує зручний каталог із модулем аукціонних торгів у реальному часі, системою верифікації учасників та захистом угод, є актуальним і практично важливим завданням.

Мета випускої кваліфікаційної роботи спроектувати та розробити спеціалізований маркетплейс для торгівлі годинниковими аксесуарами із вбудованим онлайн-аукціоном. Щоб цього досягти, було детально розібрано рішення конкурентів, складено список технічних вимог і створено готовий сайт, де є чотири категорії товарів, чіткий поділ користувачів за ролями та підключена платіжна система.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		7

Об'єктом дослідження є процес проєктування та розробки спеціалізованої вебплатформи для торгівлі годинниками та аксесуарами з підтримкою аукціонних торгів у реальному часі.

Предметом дослідження є методи, архітектурні підходи та програмні інструменти реалізації вебплатформи «RoyalTick», зокрема принципи побудови модульного моноліту на базі Clean Architecture, механізми організації аукціонних торгів та алгоритми забезпечення безпеки онлайн-угод.

Методи дослідження. У роботі використовуються методи системного аналізу для дослідження вимог до продукту та порівняльного аналізу існуючих платформ, принципи об'єктно-орієнтованого програмування для побудови архітектури застосунку та методи візуального моделювання для проєктування бази даних і логіки системи. Для якісної роботи вебплатформи було проведено аналіз існуючих платформ для торгівлі годинниками та аксесуарами з метою визначення їхніх переваг та недоліків. Спроектовано архітектуру системи та структуру бази даних з використанням UML-діаграм, а також розроблено модуль каталогу товарів чотирьох категорій з системою фільтрації та пошуку. Реалізовано модуль аукціонних торгів з автоматичним завершенням, системою ставок та приватними чатами між учасниками угоди; забезпечено безпеку платформи через JWT-авторизацію, верифікацію профілів та хешування паролів засобами bcryptjs, ще було інтегровано платіжний шлюз WayForPay та хмарне сховище Cloudinary. Вебплатформу розгорнуто з використанням Docker та автоматизованого GitLab CI/CD пайплайну.

За темою випускної кваліфікаційної роботи бакалавра було опубліковано тези на “Міжнародної науково-технічної конференції Інформаційно-комп’ютерні технології” - 2026, 02-03 квітня 2026 року [1, 2].

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		8

РОЗДІЛ 1. АНАЛІЗ ВИКОРИСТАННЯ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

1.1. Постановка завдання

Сучасний ринок годинників та аксесуарів до них активно переходить в онлайн, проте нинішні майданчики не здатні повністю задовольнити потреби колекціонерів та ентузіастів. Покупці постійно стикаються з незручним пошуком і відсутністю захисту під час угод, продавці позбавлені надійного інструменту для проведення прозорих відкритих торгів, а сама спільнота розпорошена по застарілих форумах. Через це процеси пошуку товарів, організації аукціонів, захисту фінансових операцій та комунікації між учасниками реалізовані або незручно, або взагалі відсутні, що суттєво знижує цінність таких сервісів.

Тому перед початком розробки було сформульовано чітке завдання створити спеціалізовану вебплатформу RoyalTick, яка об'єднає в одному місці інтернет-магазин годинникових аксесуарів, модуль онлайн-аукціонів у реальному часі та простір для спілкування. Система розрахована на кілька груп користувачів із різними рівнями доступу. Звичайний гість може вільно переглядати каталог і купувати товари без реєстрації. Зареєстрований користувач отримує доступ до розширених можливостей може виставляти власні лоти, спілкуватися у спільноті та підписуватися на інших авторів. Верифікований учасник після підтвердження телефону та картки отримує право робити ставки на аукціоні й вести приватне листування для завершення угод. Модератор контролює чесність торгів, реагує на скарги та допомагає вирішувати суперечки в чатах, а адміністратор керує всім контентом і профілями через окрему ізольовану панель управління.

Вебплатформа ефективно вирішує кілька ключових проблем, які наразі залишаються без якісного рішення на ринку. Оскільки на сайті представлено чотири різні категорії годинники, ремінці, коробки та засоби догляду кожна з них має свій унікальний набір характеристик. Для годинників це тип механізму, матеріал корпусу чи водозахист, для ремінців матеріал, ширина й

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		9

застібка, а для коробок місткість та оздоблення. Жорстка реляційна структура бази даних змусила б створювати безліч порожніх полів або писати складні й важкі запити, тому для зберігання даних обрано документно-орієнтовану базу MongoDB, яка дозволяє кожній картці товару містити лише ті атрибути, які їй дійсно потрібні. Організація чесних та прозорих аукціонів є однією з найголовніших складових. Головна загроза для відкритих торгів це фейкові акаунти, які штучно накручують ціну або виграють лоти без наміру їх оплачувати. Для захисту від подібних маніпуляцій у системі впроваджено обов'язкову верифікацію користувачів через підтвердження номера телефону та банківської картки. Перевірка картки за алгоритмом Луна відбувається прямо в браузері ще до відправки даних на сервер, а самі дані зберігаються виключно у вигляді bcrypt-хешів, що виключає їх компрометацію чи створення дублікатів профілів. Аукціонний алгоритм автоматично визначає переможця після завершення таймера, миттєво відкриває приватний чат між сторонами та автоматично блокує покупця, якщо той не виходить на зв'язок у визначений термін. Більшість вітчизняних майданчиків не мають вбудованої оплати, перекладаючи всі ризики на страх і ризик користувачів. У RoyalTick було реалізовано інтеграцію з українським платіжним шлюзом WayForPay за допомогою підписів HMAC-MD5, що гарантує цілісність даних і унеможливорює їх підміну під час транзакції. Після успішного переказу коштів система фіксує замовлення, надсилає email-підтвердження через Nodemailer та очищає кошик покупця.

Формування довіри та репутації стоїть гострим питанням, адже жоден маркетплейс не може нормально функціонувати без прозорої репутаційної системи. Тому на вебплатформі реалізовано двостороннє оцінювання після завершення угоди і продавець, і покупець виставляють один одному оцінки. Ці бали формують публічний рейтинг у профілі, що допомагає учасникам приймати зважені рішення про майбутні угоди. Доповненням до цього є модуль спільноти, де досвідчені колекціонери можуть ділитися досвідом, обговорювати новинки та допомагати новачкам.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		10

Для реалізації цих завдань необхідно детально розібрати наявні аналоги, виділити їхні сильні та слабкі сторони, сформулювати технічні вимоги до майбутньої вебплатформи та обґрунтувати вибір стеку технологій. Результатом цього аналізу стане чітке розуміння ринкового позиціонування платформи RoyalTick та тієї практичної цінності, яку вона принесе користувачам.

1.2. Аналіз аналогів програмного продукту

Сьогодні світовий ринок розкішних речей активно змінюється та переходить в онлайн. Люди все частіше обирають сайти, де можна не просто купити нову річ, а й вигідно обміняти або продати свої вживані товари. Коли мова йде про дорогі годинники та деталі до них, покупці стають дуже вимогливими. Для них на першому місці стоїть оригінальність товару та чесність продавця. Щоб створити платформу RoyalTick, було детально вивчено три популярні майданчики з різних сегментів. Це великий аукціон Violity, тематичний сайт Watch.ua та відомий світовий маркетплейс Chrono24.

На сьогоднішній день Violity є беззаперечним лідером українського ринку антикваріату та предметів колекціонування, функціонуючи як масштабна цифрова екосистема для купівлі-продажу. Ресурс вийшов за межі звичайного "дошки оголошень і став повноцінним онлайн-аукціоном із величезною аудиторією. Там продають абсолютно все від старовинних монет до сучасних годинників та запчастин для них.

Уся робота цього сайту побудована навколо системи торгів. Користувачі роблять ставки в реальному часі, тому система має витримувати великі навантаження, особливо в останні хвилини перед закриттям лота. Для захисту від шахраїв на Violity працює двостороння система рейтингів для продавців та покупців. Вона будується на перевірених відгуках людей. Цей механізм базується виключно на реальних, верифікованих відгуках від користувачів, які вже успішно завершили свої угоди. Впровадження такого інструменту контролю є критично важливим аспектом для сегмента електронної комерції, пов'язаного з обігом дорогих товарів та антикваріату. Воно дозволяє створити

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		11

прозоре цифрове середовище, де учасники торгів можуть розраховувати на взаємну довіру та безпеку під час фінансових операцій і передачі лотів.

Таблиця 1.1.

Аналіз переваг та недоліків платформи «Violity»

Переваги	Недоліки
Найбільша аудиторія серед українських аукціонів	Перевантажений застарілий інтерфейс, у якому важко розібратися
Зручний механізм торгів зі ставками в реальному часі	Немає мобільного додатка, а версія для телефонів погано налаштована
Двостороння система рейтингів на основі реальних відгуків	Немає вбудованого захисту грошей під час розрахунків
Великий вибір категорій, включаючи годинники та запчастини	Складний і неточний пошук у специфічних розділах
Величезна кількість активних оголошень щодня	Ручна перевірка оголошень, яка часто затягується
Багато років на ринку, що викликає велику довіру людей	Застарілі технології та дуже рідкісні оновлення сайту

Усі ці системні мінуси, плутанина в окремих категоріях і банальна відсутність нормальної мобільної версії чудово показують, як старі технології можуть зіпсувати враження від користування сайтом. Здавалося б, майданчик прокручує величезну кількість угод щодня, але його зовнішній вигляд абсолютно не встигає за звичками сучасних людей, які хочуть швидко і зручно зайти на сайт зі свого смартфона. Через це інтерфейс часто здається незручним і перевантаженим. Щоб краще зрозуміти, як розробники структурували інформаційні блоки, де саме вони розкидали елементи керування та який вигляд має робочий простір цього аукціону в реальності, найкраще просто

поглянути на його графічний зріз. Оригінальний вигляд головної сторінки та інтерфейс вебсайту Violity наочно наведено на (рис 1.1).

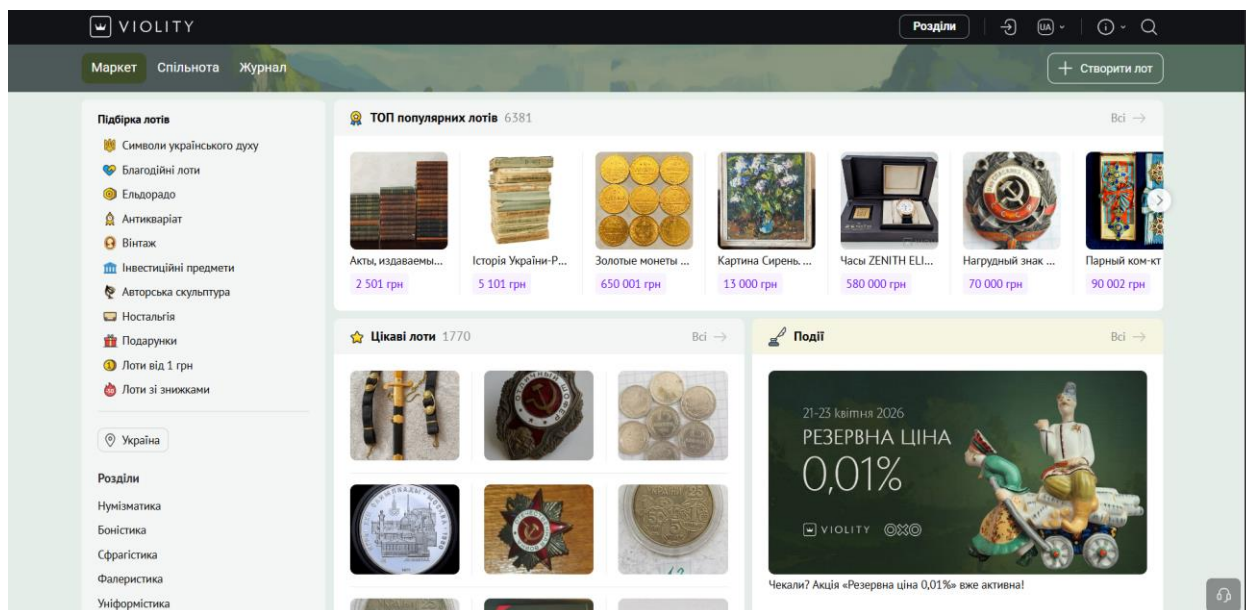


Рис.1.1. Інтерфейс сайту Violity

Сайт Watch.ua це суто спеціалізований майданчик, який з часом став справжньою точкою збору для українських фанатів годинникової справи. Усе починалося як звичайний камерний форум «для своїх», але згодом проєкт переріс у серйозний майданчик для дискусій, де люди залюбки діляться досвідом і знаннями. Головна фішка сайту в тому, що він працює як жива спільнота. Тут немає звичної структури інтернет-магазину: увесь контент, огляди та навіть оголошення про продаж створюють самі ж учасники просто всередині тематичних гілок обговорення.

Проте з технічного боку вебплатформа сильно буксує, адже її движок побудований на базі застарілих форумних систем. Через це вся інформація йде суцільним лінійним потоком. Дані про товари не вийде розкласти за полицками у зручній сучасній базі з чіткими характеристиками вони так і залишаються звичайними текстовими повідомленнями в темах. Зрозуміло, що такий підхід повністю ламає можливість зробити нормальний автоматичний пошук за конкретними параметрами. Крім того, на таку «древню» систему практично нереально прикрутити сучасні сервіси доставки чи безпечну оплату карткою. Зрештою, саме ці технічні обмеження та застарілий формат форуму

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		13

заважають платформі перетворитися на повноцінний сучасний маркетплейс, що чітко видно під час детального аналізу її сильних і слабких сторін.

Таблиця 1.2.

Аналіз переваг та недоліків платформи «Watch.ua»

Переваги	Недоліки
Єдина спеціалізована спільнота для українських любителів годинників	Технічна база побудована на застарілій форумній системі
Жива експертна аудиторія, яка реально розбирається в темі	Дані про товари зберігаються як звичайні текстові повідомлення, без структурованої бази
Можливість отримати фахову оцінку або консультацію від досвідчених колекціонерів	Відсутній пошук за характеристиками товару
Пряма комунікація між продавцями і покупцями в тематичних гілках	Немає вбудованої безпечної оплати картою
Великий архів обговорень з реальним досвідом користувачів	Немає захисту покупця при угоді
Давня присутність на ринку та висока довіра серед колекціонерів	Відсутня повноцінна мобільна версія сайту

Попри таку купу технічних мінусів, цей сайт усе одно зтягує людей своєю особливою атмосферою та ламповістю спільноти. Щоправда, коли вперше туди заглядаєш, старий форумний движок одразу дає про себе знати розібратися в купі тем та безкінечних гілках обговорень буває тим ще квестом. Візуально майданчик ніби застряг десь у минулому, що дуже сильно помітно на тлі сучасних спритних інтернет-магазинів. Щоб на власні очі побачити цю суто текстову структуру, зрозуміти, де там взагалі розкидані основні розділи та як бідним користувачам доводиться шукати потрібні оголошення, найкраще просто глянути на скріншот. Реальний вигляд головної сторінки та інтерфейс вебсайту Watch.ua показано нижче на (рис.1.2).

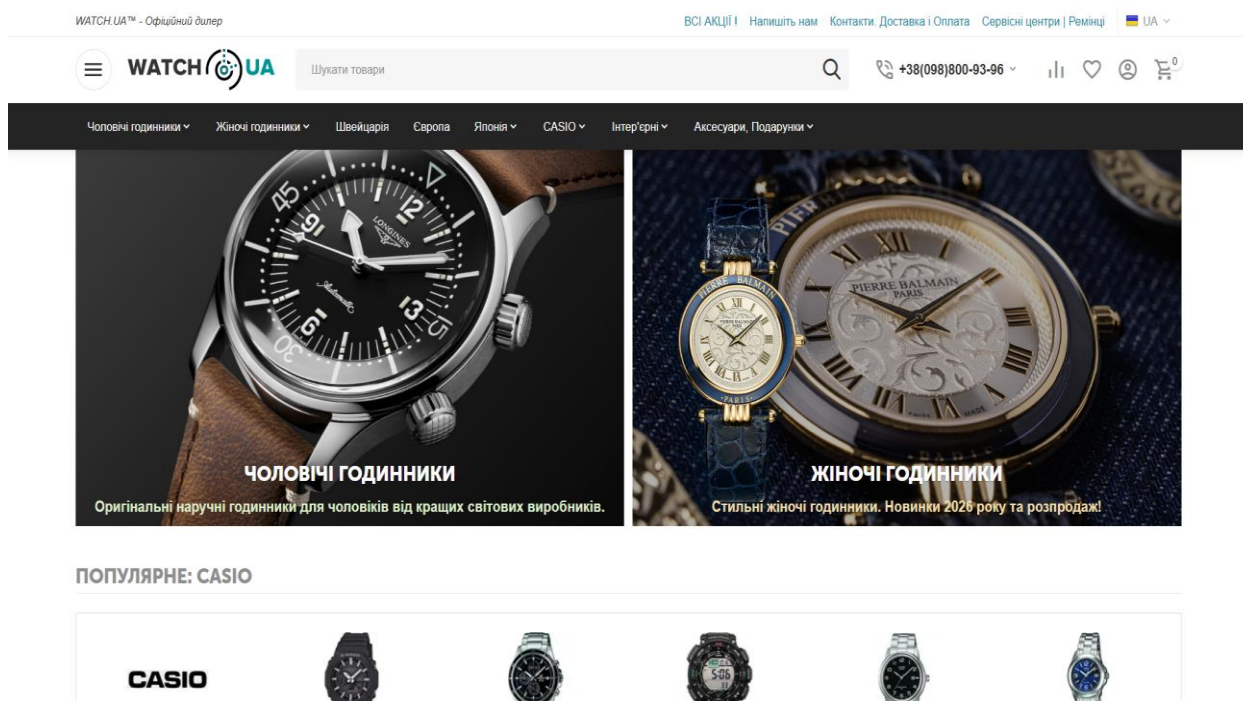


Рис.1.2. Інтерфейс сайту Watch.ua

Сайт Chrono24 - це головний світовий маркетплейс для купівлі та продажу дорогих годинників. Тут можуть успішно торгувати як великі компанії, так і звичайні приватні продавці. Вебплатформа побудована на базі сучасних технологій, які легко витримують величезні навантаження. Завдяки цьому всі дані на сайті оновлюються миттєво, ціни автоматично переводяться у потрібну валюту, а сам інтерфейс адаптований під десятки різних країн.

Технічна частина системи налаштована так, щоб забезпечити максимальну безпеку для кожного клієнта. На сайті працює вбудована система безпечних угод, яка надійно захищає гроші покупця під час оплати. Також сервіс використовує розумні алгоритми для перевірки оригінальності годинників. Вони допомагають підтверджувати профілі продавців та автоматично порівнюють ціни на товари з реальними ринковими показниками. Такий підхід зводить усі ризики до мінімуму та робить платформу надійним посередником для угод по всьому світу.

Система пошуку та фільтрів на Chrono24 взагалі вважається зразковою для інтернет-магазинів у цій сфері. Програма використовує детальну базу даних, тому користувачі можуть легко шукати годинники за десятками різних

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				15
Змн.	Арк.	№ докум.	Підпис	Дата		

параметрів, наприклад, за типом механізму, матеріалом корпусу, роком випуску чи наявністю оригінальних документів.

Таблиця 1.3.

Аналіз переваг та недоліків платформи «Chrono24»

Переваги	Недоліки
Найбільша у світі база оголошень про продаж годинників	Вебплатформа не адаптована для українського ринку, немає підтримки гривні та української мови
Вбудована система безпечних угод, яка захищає гроші buyer-а	Складний і тривалий процес реєстрації для приватного продавця
Верифікація продавців та автоматична перевірка цін відносно ринку	Немає жодних інструментів для спільноти чи спілкування між користувачами
Зразковий параметричний пошук за десятками характеристик	Вебплатформа орієнтована переважно на преміальний сегмент, бюджетні товари практично не представлені
Підтримка десятків валют та локалізація для різних країн	Кошти продавця утримуються в системі тривалий час до підтвердження угоди
Сучасна технічна база, яка витримує великі навантаження	Міжнародна доставка та митне оформлення повністю лягають на плечі покупця
Мобільний додаток з повним функціоналом	Служба підтримки повільно реагує на суперечки між сторонами

Попри певні мінуси з адаптацією під наш локальний ринок, з точки зору ергономіки та дизайну цей ресурс усе одно залишається чудовим прикладом

для копіювання. Щоб наочно побачити, як розробники організували головну сторінку та картки годинників на Chrono24, варто поглянути на (рис 1.3).

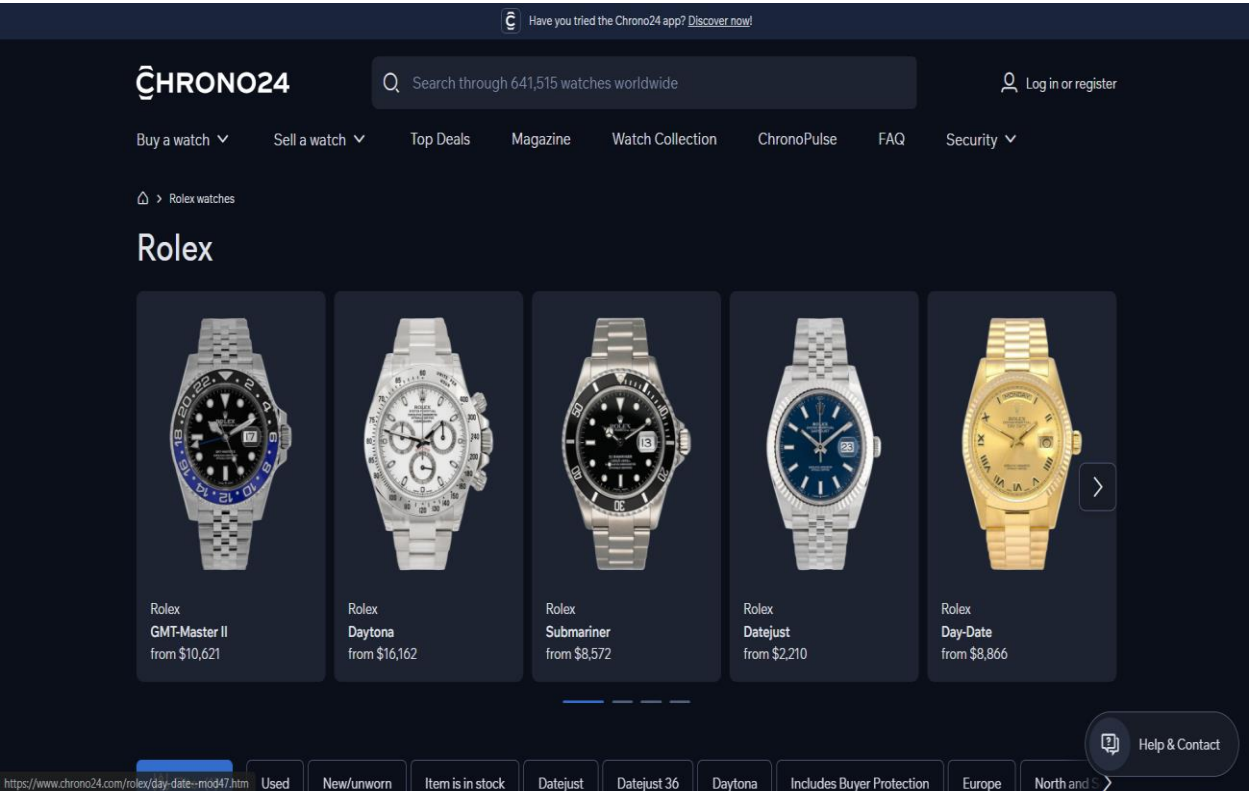


Рис.1.3. Інтерфейс сайту Chrono24

Таблиця 1.4 містить детальний порівняльний аналіз технічних та бізнес-рішень досліджуваних систем. Вибрані критерії відображають ключові вимоги до сучасних e-commerce платформ у сегменті аксесуарів.

Таблиця 1.4.

Порівняльна характеристика функціональних можливостей платформ

Критерій	Violity	Watch.ua	Chrono24	RoyalTick
Модель продажів	Аукціон	Оголошення на форумі	Маркетплейс	Аукціон та фіксована ціна
Аксесуари	Відсутня	Часткова	Відсутня	Повна
Аукціонний функціонал	Є	Відсутній	Є	Є (з депозитною системою)

Критерій	Violity	Watch.ua	Chrono24	RoyalTick
Захист угод	Відсутній	Відсутній	Є (ескроу)	Є (депозит та верифікація)
Верифікація учасників	Відсутня	Відсутня	Є (продавців)	Є (телефон та картка)
Параметричний пошук	Обмежений	Відсутній	Розширений	Є
Спільнота та форум	Є	Є	Відсутня	Є
Мобільна адаптація	Обмежена	Відсутня	Є (додаток)	Є (адаптивний дизайн)
Мультимовність	Відсутня	Відсутня	Є (десятки мов)	Є (UA / EN)
Приватний чат	Є	Відсутній	Відсутній	Є
Система рейтингів	Є	Відсутня	Є	Є

Якщо подивитися на результати порівняння в таблиці 1.4, стає очевидно, що знайти потрібні аксесуари для годинників сьогодні досить складно. Наявні сайти мають серйозні мінуси. Violity занадто загальні, тому знайти там конкретну деталь чи ремінець вкрай важко через неточний пошук. Інші майданчики, як-от Watch.ua, технічно застаріли та взагалі не захищають покупців від шахраїв, перекладаючи всі ризики на людей. Навіть світові лідери рівня Chrono24 зосереджені лише на самих годинниках.

Вебплатформа RoyalTick створена для того, щоб вирішити всі ці проблеми. Ми додали унікальну можливість прямого обміну речами між користувачами та впровадили систему безпечних угод для захисту грошей. З технічної сторони ми обрали сучасну та надійну архітектуру «модульного

моноліту». Такий підхід разом із розумною системою пошуку за конкретними параметрами дозволяє створити зручний та безпечний простір для колекціонерів. Це доводить, що запуск RoyalTick є повністю виправданим. Наш проєкт зможе зайняти вільну нішу на українському ринку та задати новий рівень якості для всіх, хто захоплюється годинниками.

1.3. Обґрунтування вибору архітектури системи

Правильно обрати внутрішню структуру коду - це одне з найважливіших рішень під час створення сайту, адже від цього безпосередньо залежить, наскільки стабільно працюватиме система під навантаженням і скільки зусиль знадобиться на її підтримку в майбутньому. Невдалий вибір архітектури на початковому етапі може призвести до того, що проєкт доведеться повністю переписувати з нуля вже за кілька місяців після запуску. Вебплатформа RoyalTick об'єднує в собі дуже різні речі великий каталог товарів, систему онлайн-аукціонів, приватні чати для користувачів, повноцінну спільноту та панель керування для адміністратора. Кожна з цих частин працює за своїми правилами, тому потрібно було розділити їх у коді так, щоб вони не заважали одна одній, але й не ускладнювали програму без потреби. Невдалий вибір архітектури на початковому етапі може призвести до того, що проєкт доведеться повністю переписувати з нуля вже за кілька місяців після запуску. Щоб знайти найкращий варіант, було детально порівняно два найпопулярніші підходи: класичний цілісний моноліт та систему окремих мікросервісів.

При класичному монолітному підході весь сайт створюється як одна велика та нероздільна програма[5]. У ній зовнішній вигляд сайту, його внутрішні правила та робота з базою даних повністю переплетені між собою. Головний плюс тут у простоті є лише один проєкт, який легко запустити та налаштувати. На самому початку розробки це дозволяє рухатися дуже швидко. Проте з часом, коли сайт розростається, починаються серйозні проблеми. Усі функції стають занадто залежними одна від одної. Наприклад, якщо ми захочемо щось змінити в системі аукціону, це може випадково і непередбачувано зламати каталог товарів або вхід на сайт. Крім того, для будь-

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		19

якого дрібного оновлення доводиться щоразу повністю перезапустити всю величезну систему, через що росте ризик якихось збоїв. З навантаженням теж виникають труднощі. Якщо через активні торги в останні хвилини аукціону почне гальмувати лише один цей розділ, доведеться копіювати та розширювати всю програму цілком на нові сервери, навіть якщо інші функції сайту в цей час ніхто не використовує. Така неперворотність системи значно збільшує фінансові витрати на оренду серверних потужностей і робить технічну підтримку платформи неефективною.

Мікросервісна архітектура працює зовсім інакше тут сайт розбивають на купу маленьких незалежних програм. Кожна з них відповідає за свою окрему функцію і має власну базу даних. Такий розподіл дозволяє розширювати на серверах лише ті розділи сайту, де дійсно виросло навантаження, а розробники можуть писати та оновлювати кожен сервіс окремо від інших. Проте для проєкту на цьому етапі така модель є занадто громіздкою та складною. Налаштування зв'язку між різними серверами, контроль за синхронізацією даних у різних базах, відстеження помилок у десятках дрібних програм - усе це сильно заплутує розробку та звичайне тестування коду. До того ж витрати на утримання такої великої інфраструктури на самому початку будуть значно вищими, ніж реальна користь від її гнучкості. Було розглянуто мікросервіси як чудовий варіант на майбутнє, коли вебплатформа сильно виросте, але для першого запуску сайту цей підхід себе не виправдовує.

Для платформи RoyalTick було обрано сучасну модульну структуру на базі Next.js [8]. Це дало найкраще поєднання сайт легко запускати та підтримувати, але при цьому весь код чітко розділений на незалежні блоки. Це дозволяє за необхідності швидко перейти на мікросервіси в майбутньому без глобальної зміни логіки додатку. Наочно побачити порівняння різних варіантів побудови системи можна на (рис.1.4).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				20
Змн.	Арк.	№ докум.	Підпис	Дата		

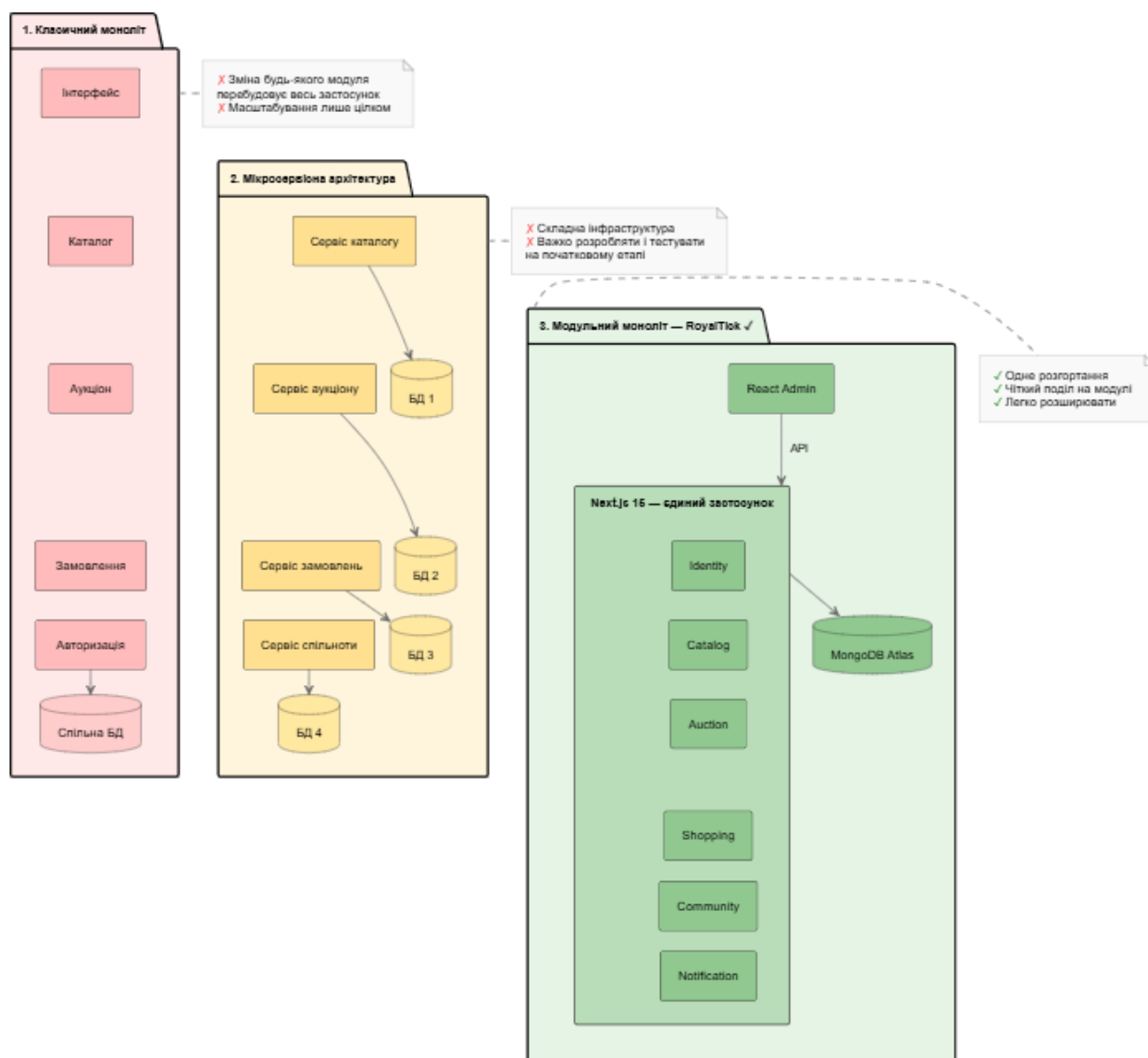


Рис. 1.4. Порівняльний аналіз архітектурних стратегій

Завдяки можливостям Next.js App Router, зовнішня частина сайту та його серверна логіка об'єднані в один спільний Full-Stack проєкт. При цьому відображення сторінок налаштоване гнучко і працює у двох режимах, залежно від того, що саме бачить користувач. Перший режим - це створення сторінок безпосередньо на сервері (SSR). Ми використовуємо його для каталогу, карток товарів та аукціонних лотів. Оскільки сервер сам збирає готову сторінку і миттєво віддає її користувачу, сайт завантажується дуже швидко, а пошукові системи (наприклад, Google) без проблем бачать увесь текст та правильно піднімають сайт у результатах пошуку. Другий режим - це обробка даних прямо у браузері користувача (CSR). Він працює для всіх живих та інтерактивних елементів кошика, особистого кабінету, форм для підняття

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Євдємов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		21

ставок та спливаючих вікон. Це дозволяє сайту миттєво реагувати на будь-які кліки та дії людини без постійного перезавантаження сторінок. Вся серверна частина працює через внутрішні маршрути (API Routes) самого Next.js. Вони відповідають за безпечний обмін даними та чітко перевіряють права доступу користувачів залежно від їхньої ролі на сайті. Панель керування для адміністратора зроблена як окрема швидка програма на базі React Admin і підключається до того самого сервера [14]. Повну схему того, як взаємодіють усі ці частини системи, можна подивитися на (рис.1.5).

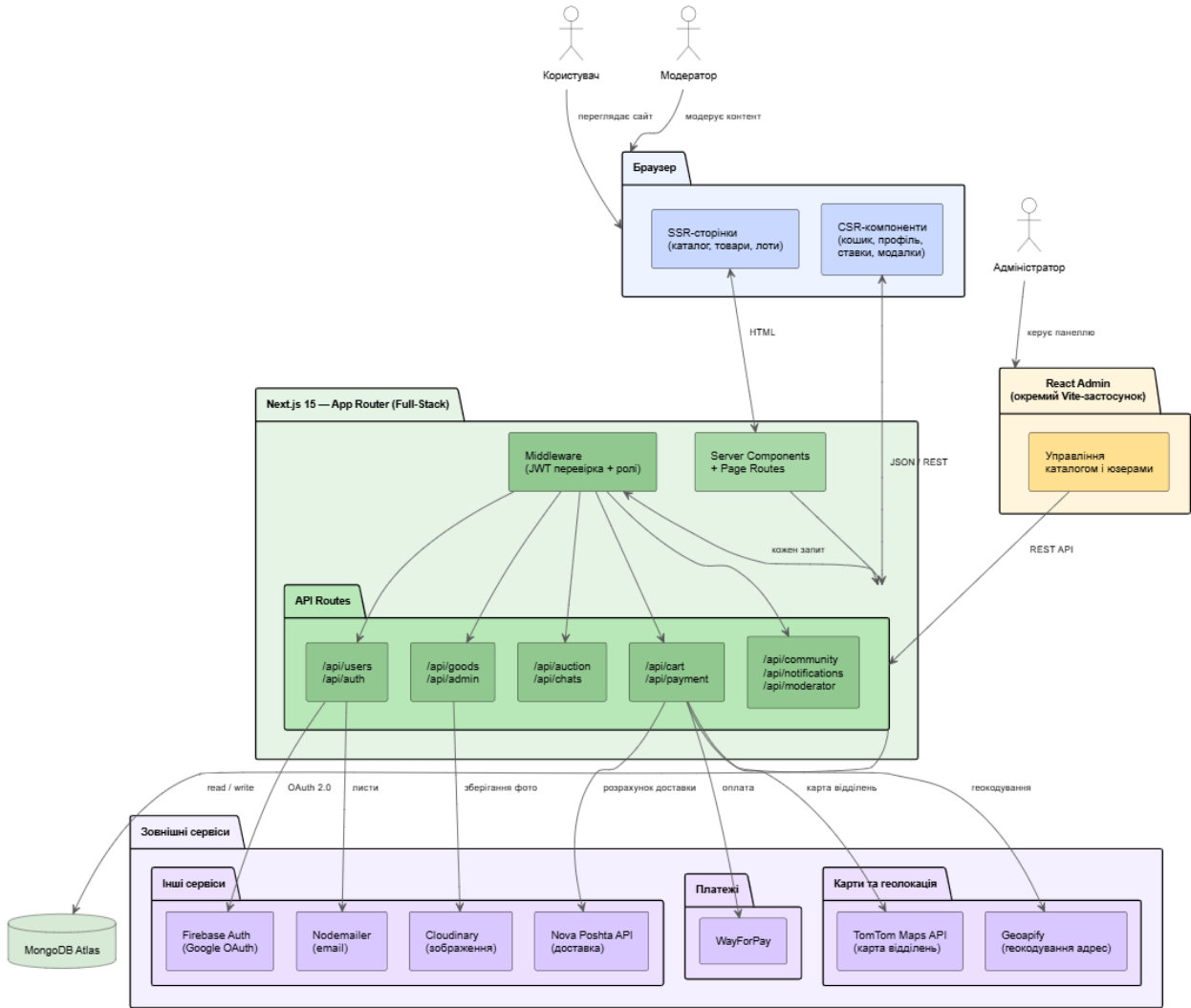


Рис. 1.5. Архітектурна схема вебплатформи RoyalTick

Для організації коду було обрано принципи Чистої Архітектури, щоб не перетворити проєкт на заплутану грудку коду, яку важко оновлювати [27]. Головна ідея тут проста - розбити систему на незалежні рівні за принципом «матрьошки». Зовнішні рівні знають про існування внутрішніх, але серце

програми завжди залишається ізольованим. Завдяки цьому головні правила роботи маркетплейса взагалі не залежать від того, яку базу даних ми використовуємо чи як налаштовані мережеві запити. Якщо завтра виникне потреба замінити MongoDB на іншу систему або повністю перемалювати дизайн, логіка самого аукціону чи замовлень не змінить жодного рядка коду.

У самому центрі системи знаходяться базові об'єкти, навколо яких будується весь бізнес годинники, ремінці, коробки, засоби догляду, а також профілі користувачів, лоти та замовлення. Навколо них вибудований рівень живих сценаріїв. Він керує тим, що реально відбувається на сайті, наприклад, коли людина робить ставку на торгах, купує товар чи проходить верифікацію. І вже на самому краєчку системи розташовані адаптери - це технічний місток, який зшиває все до купи. Саме тут серверні маршрути Next.js приймають запити, візуальні React-компоненти показують інтерфейс користувачу, а спеціальні репозиторії записують дані в MongoDB. Такий підхід дозволяє тримати весь проєкт у повному порядку, навіть коли кількість функцій починає стрімко рости. Головне, що ми отримуємо від такого розподілу - це повна свобода дій під час тестування та розробки окремих фіч, адже зміни в одному місці гарантовано не «впустять» усю систему.

Крім того, завдяки чітким межах між рівнями, будь-який новий розробник зможе швидко розібратися в структурі папок і не буде блукати годинами в коді. Звісно, на практиці правильне налаштування всіх цих технічних містків та інтерфейсів вимагає трохи більше часу на самому початку, але в перспективі це повністю окупається під час масштабування бізнесу чи додавання нових модулів, наприклад, тієї ж системи обміну.

Для того, щоб наочно уявити, як саме ці архітектурні шари вкладені один в один, де проходять межі залежностей та як влаштована внутрішня ізоляція логіки RoyalTick, доцільно розглянути фінальну схему, що наведена нижче на (рис.1.6).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		23

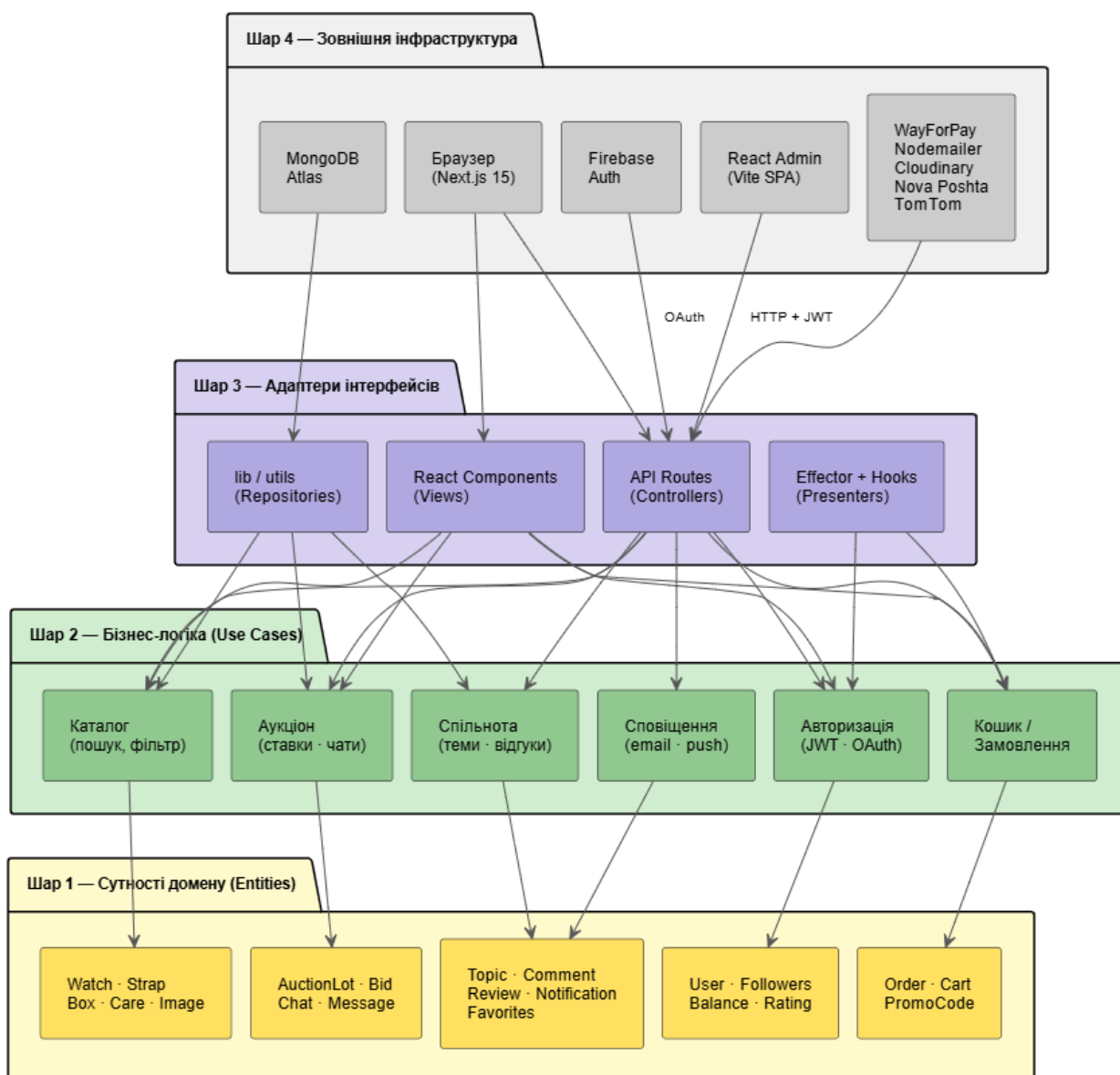


Рис. 1.6. Схема шарів системи за принципами Clean Architecture

Щоб велика кількість функцій не перетворилася на хаос, ми розділили систему на шість окремих частин, де кожна займається виключно своєю справою і не лізе в роботу сусідів. Перший блок відповідає за профілі та безпеку. Він керує реєстрацією, створенням захищених токенів, а також перевіркою телефонів і банківських карток. Поруч із ним працює каталог, який зберігає всю інформацію про товари та відповідає за швидкий пошук із фільтрами. За кошик, оформлення замовлень, розрахунок доставки та зв'язок із платіжною системою WayForPay відповідає третій блок, який повністю закриває тему покупок. Усіма торгами, ставками, таймерами, приватними чатами для переможців та системою відгуків керує окремий аукціонний

модуль. Для розвантаження системи спільноту з її блогами, коментарями та фотографіями ми винесли в окрему зону. Останнім елементом став пасивний блок сповіщень. Він сам нічого не ініціює, а лише чекає на сигнали від інших частин сайту, щоб вчасно надіслати користувачу повідомлення. Головна фішка такого поділу в тому, що кожна частина має свої власні колекції в базі даних і ніяк не може напряму читати чи змінювати чужі файли. Наприклад, блок безпеки є єдиним місцем, де зберігаються особисті дані людей. Усі інші модулі під час роботи отримують від нього лише унікальний ідентифікатор користувача і навіть не знають його пароля чи телефону. Повну схему того, як ці блоки ізольовані та як саме вони взаємодіють між собою, можна побачити на (рис.1.7).

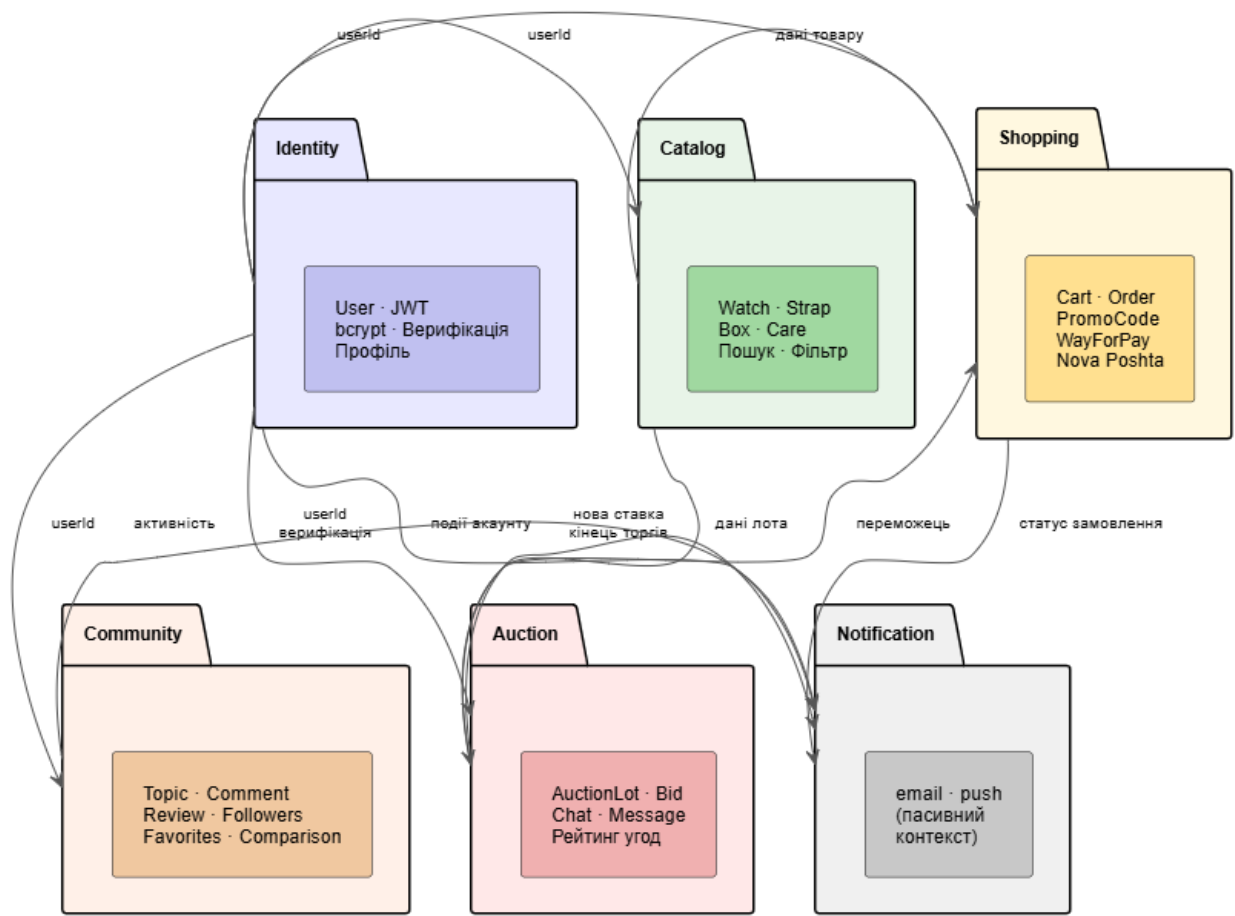


Рис. 1.7. Діаграма обмежених контекстів (Bounded Contexts) платформи RoyalTick

Обрана архітектура забезпечує низьку зв'язаність між модулями і високу зв'язність всередині кожного з них. Каталог, аукціон, спільнота і система

замовлень розвиваються незалежно, не впливаючи один на одного при змінах. Якщо в майбутньому аукціонний модуль потребуватиме окремого масштабування через зростання навантаження, його можна буде винести в окремий мікросервіс без переписування решти платформи. Це робить обрану архітектуру не лише оптимальним рішенням для поточного етапу, а й надійним фундаментом для довгострокового розвитку проєкту.

1.4. Обґрунтування вибору інструментальних засобів та вимоги апаратного забезпечення

Технологічний стек для RoyalTick було підібрано під конкретні практичні задачі потрібна була швидка робота каталогу із хорошою SEO-оптимізацією, стабільні аукціони, безпека користувачів та зручна адмінка. Кожен інструмент обирався не через популярність, а тому, що він найкраще вирішував свою частину роботи.

Серцем усього проєкту став фреймворк Next.js з архітектурою App Router. Він дозволив об'єднати зовнішній інтерфейс і серверний код в один Full-Stack застосунок, завдяки чому нам не довелося створювати окремий бекенд. Для сторінок каталогу, карток товарів та аукціонних лотів було вирішено налаштувати генерацію HTML прямо на сервері. Це забезпечило швидке перше завантаження сайту, а головне - пошукові роботи Google тепер без проблем бачать і правильно індексують увесь контент. Водночас динамічні елементи, як-от кошик, особистий кабінет та форми для ставок, працюють прямо в браузері користувача. Це дає миттєвий відгук інтерфейсу без постійного перезавантаження сторінок.

Уся серверна логіка крутиться навколо вбудованих маршрутів API Routes, які формують надійне RESTful API з чітким розмежуванням прав доступу для різних ролей користувачів. Основною мовою розробки обрано TypeScript[12]. Оскільки на сайті крутиться багато складних та різноманітних даних - від детальних характеристик годинників до статусів аукціону та JWT-токенів - сувора типізація допомагає ловити помилки ще під час написання коду, не пускаючи їх на живий сайт. Це також сильно спрощує роботу, коли в

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		26

код потрібно внести якісь правки. Для збереження всієї інформації було підключено хмарну NoSQL базу даних MongoDB Atlas. Цей вибір продиктований специфікою маркетплейсу, де кожна категорія товарів має абсолютно унікальний набір характеристик. Ремінці, коробки, годинники та засоби догляду описуються абсолютно по-різному, і в класичній реляційній базі це призвело б до створення десятків зайвих таблиць або купи порожніх клітинок. Гнучка структура документів MongoDB дозволяє кожному товару зберігати лише ті поля, які йому дійсно потрібні. Сам хмарний сервіс Atlas додатково закриває питання безпеки, оскільки автоматично робить резервні копії і звільняє нас від налаштування серверної інфраструктури з нуля.

Керування станом даних на клієнтській стороні обрано бібліотеку Effector. На відміну від громіздкого Redux, він використовує простішу та зрозумілішу декларативну модель, яка не змушує писати купу зайвого шаблонного коду. Окремі сховища для авторизації, кошика, поточного товару та модальних вікон роблять рух даних передбачуваним і значно спрощують пошук багів.

Систему безпеки та входу на сайт побудовано на двох рівнях. Автентифікація працює на парі JWT-токенів короткостроковий токен доступу діє 10 хвилин, а токен оновлення - 30 днів, причому система оновлює сесію автоматично і непомітно для користувача. Усі паролі перед записом у базу проходять надійне хешування через bcryptjs з унікальною сіллю, що повністю захищає їх від зламу через готові таблиці перебору. Як альтернативу додано швидкий вхід через Google-акаунт, який реалізували за допомогою Firebase Authentication на базі протоколу OAuth 2.0. Це дозволило делегувати складну перевірку профілю надійним серверам самої компанії Google.

Гроші на сайті ходять через платіжний шлюз WayForPay. На ньому повністю зав'язано поповнення внутрішнього балансу користувача, з якого потім максимально зручно поповнювати баланс для виставлення лоту. Щодо технічного боку, то тут усе строго кожен фінансовий запит підписується за допомогою шифрування HMAC-MD5, а сервер постійно ловить callback-

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		27

сповіщення від платіжної системи, щоб миттєво реагувати на успішну транзакцію та зараховувати кошти на гаманець у профілі Доставка товарів автоматизовано через Nova Poshta API, завдяки чому користувач завжди бачить актуальний список відділень та реальну вартість відправки. Щоб показувати ці відділення на карті, ми підключили TomTom Maps API, а за перетворення звичайних текстових адрес у точні географічні координати відповідає сервіс Geoapify.

Усі зображення товарів зберігаються та обробляються в хмарі Cloudinary. Завантажені фотографії автоматично стискаються без втрати якості та роздаються через мережу CDN, що дозволяє сторінкам сайту завантажуватися значно швидше. Зовнішній вигляд платформи було створено за допомогою SCSS Modules, де кожен компонент отримує свій ізольований простір імен, що повністю виключає конфлікти стилів. Усі фірмові кольори, відступи та шрифти винесені в глобальні змінні для зручного керування дизайном.

Динамічності інтерфейсу додає бібліотека Framer Motion, завдяки якій картки товарів з'являються плавно, кошик реагує на мікроінтеракції, а переходи між сторінками залишають відчуття дорогого та якісного продукту. Для зручного керування всім цим масивом даних ми розробили окрему адміністративну панель як швидкий Vite-застосунок на базі React Admin. Вона підключається до того самого сервера і дає адміністраторам зручний візуальний інтерфейс для модерації каталогу, редагування профілів та оновлення картинок. Окремо налаштовано Nodemailer, який через SMTP-протокол надсилає користувачам красиві HTML-листи для підтвердження реєстрації, зміни пароля чи оновлення статусів їхніх замовлень. Для стабільного запуску та розгортання проєкту було упаковано застосунок у Docker-контейнери.

Для головної програми Next.js написано багатоетапну збірку. На першому етапі підтягуються залежності, на другому збирається сам проєкт з усім оточенням, а на фінальному етапі створюється максимально легкий

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		28

production-образ. Цей образ запускається від імені безпечного користувача без адмінських прав, що значно підвищує захист сервера та зменшує розмір файлів. Панель адміністратора пакується в окремий контейнер. Обидва контейнери автоматично проходять перевірку в нашому CI/CD пайплайні на базі GitLab.

Система сама тестує код, проводить його статичний аналіз і після успішного проходження всіх етапів автоматично деплоїть проєкт на платформу Vercel[16]. З боку користувача для роботи з платформою потрібен лише будь-який сучасний браузер із підтримкою JavaScript. При цьому сам інтерфейс повністю адаптований під екрани мобільних телефонів, починаючи від ширини у 320 пікселів. На основі аналізу всієї цієї архітектури та ресурсомісткості обраного стеку ми розрахували та визначили фінальні системні вимоги, які необхідні для стабільного розгортання та роботи нашої платформи.

Таблиця 1.5.

Вимоги для функціонування програми

Параметр	Мінімальні вимоги	Рекомендовані вимоги
CPU	2 ядра, 2.0 GHz	4 ядра, 3.0 GHz+
RAM	4 ГБ	8 ГБ
Накопичувач	20 ГБ SSD	50 ГБ SSD
Мережа	10 Мбіт/с	100 Мбіт/с
ОС	Ubuntu 20.04 / Windows 10	Ubuntu 22.04 LTS
Node.js	v15+	v20 LTS

Обраний комплекс інструментальних засобів створює збалансоване середовище, що відповідає сучасним вимогам до веброзробки та забезпечує надійне підґрунтя для довготривалої підтримки проєкту.

Висновки до першого розділу

Аналіз трьох майданчиків Violity, Watch.ua та Chrono24 чітко показав, що український онлайнринок годинникових аксесуарів зараз розірваний на шматки. Жодна вебплатформа не закриває запити колекціонерів та любителів повністю.

Базуючись на виявлених мінусах конкурентів, було сформовано завдання для створюваної платформи. До списку вимог увійшли детальний каталог для чотирьох категорій товарів із гнучкими фільтрами, живий аукціон із обов'язковою перевіркою користувачів, приватні чати для безпечних угод із двостороннім підтвердженням та оцінкою продавця, а також розділ для спілкування та зручна панель адміністратора.

Оптимальним рішенням для проєкту став модульний моноліт на базі Next.js. Завдяки принципам Чистої Архітектури вдалося розділити систему на шість ізольованих зон безпеки, каталог, покупки, аукціон, спільноту та сповіщення. Такий підхід застрахував код від заплутування та дозволив модулям розвиватися незалежно. Якщо в майбутньому якийсь із них потребуватиме окремого масштабування через наплив людей, його можна буде спокійно відрізати та перетворити на мікросервіс, не чіпаючи всю іншу платформу.

Наприкінці весь застосунок було запаковано в Docker-контейнери та налаштовано автоматичний CI/CD пайплайн на GitLab. Тепер система сама тестує код і деплоїть його на сервер, що виключає будь-які сюрпризи при оновленні. Усе це разом дозволило створити залізобетонний технічний фундамент для безпосередньої програмної реалізації, яка детально описується вже у наступному розділі.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		30

РОЗДІЛ 2. ПРОЄКТУВАННЯ ВЕБПЛАТФОРМИ ДЛЯ ТОРГІВЛІ ГОДИННИКОВИМИ АКСЕСУАРАМИ З АУКЦІОННИМ МОДУЛЕМ

2.1. Визначення варіантів використання та об'єктно-орієнтованої структури системи

У платформі RoyalTick поєднано три абсолютно різних формати класичний інтернетмагазин, аукціонні торги та тематичний блог(спільнота). Оскільки кожен із цих напрямків працює за власними правилами, ще на етапі проєктування було детально продумано всі можливі дії користувачів та розмежовано їх за рівнями доступу[10]. Загалом виділено чотири ролі.

Усе починається зі звичайного гостя, тобто незареєстрованого користувача. Йому відкрито повний доступ для перегляду всього каталогу товарів, описів аксесуарів, активних лотів та публікацій у спільноті, а також можливість додавати товари до кошика порівнювати їх і здійснювати покупки. Таке рішення було прийнято свідомо, адже закриті стіни реєстрації лише відлякують нову аудиторію, тоді як повна відкритість допомагає з перших секунд оцінити користь від сайту.

Після створення облікового запису відкривається базовий комерційний рівень. З'являється можливість створювати власні теми у спільноті, підписуватися на інших колекціонерів та виставляти особисті речі на продаж через аукціон.

Проте для безпосередньої участі в торгах і підняття ставок однієї реєстрації замало тут у дію вступає рівень верифікованого користувача. Цей статус не видається автоматично задля безпеки фінансових операцій. Для його активації необхідно підтвердити номер телефону та прив'язати банківську картку. Валідація картки відбувається за алгоритмом Луна прямо в браузері, ще до моменту відправки даних на сервер, а самі інформаційні хеші захищаються через bcryptjs. Це повністю блокує спроби створювати фейкові профілі-дублікати. Лише після такої перевірки користувачеві дозволяється робити ставки, відкривається доступ до приватного чату з продавцем після

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		31

завершення торгів та з'являється право виставляти оцінки іншим учасникам угоди.

Таблиця 2.1.

Розподіл функціоналу за рівнями доступу

Функція	Гість	Користувач	Верифікований користувач	Модератор	Адмін
Перегляд каталогів і лотів	+	+	+	+	+
Кошик, замовлення, обране	+	+	+	+	+
Виставлення лота	-	+	+	-	-
Ставки на аукціон	-	-	+	-	-
Приватний чат після угоди	-	-	+	+	-
Модерація контенту	-	-	-	+	+
Управління каталогом	-	-	-	-	+
Управління користувачами	-	-	-	-	+

У модератора немає функцій покупця чи продавця. Його робота чистити спам у блогах, реагувати на скарги та підключатися до приватних чатів, щоб вирішити проблему, якщо між учасниками аукціону виник суперечка. Адміністратор системи має більше можливостей, але працює суто через ізольовану панель React Admin. Він керує обліковими записами та наповнює каталог новими категоріями товарів.

Діаграму варіантів використання системи RoyalTick наведено на (рис.2.1).



Рис. 2.1. Діаграма варіантів використання системи

Характеристика об'єкта комп'ютеризації. В основі платформи закладено три процеси. Перший процес звичайний продаж аксесуарів через каталог із

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Євремів Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		33

фіксованою вартістю та доставкою. Другий аукціонні торги, де фінальна ціна лота визначається ставками покупців у реальному часі. Третій блог з темами, який допомагає оцінити чи визначити вартість товару. Ці процеси зав'язані на єдине ядро з шести контекстів Identity, Catalog, Shopping, Auction, Community та Notification.

На рівні бази даних система керує такими основними сутностями:

Таблиця 2.2.

Основні сутності предметної області

Сутність	Призначення
User	Профіль, верифікаційні хеші, баланс, рейтинг
Watch / Strap / Box / Care	Товари чотирьох категорій з різними схемами атрибутів
AuctionLot	Лот з параметрами торгів, фото, таймером та статусом
Bid	Ставка з прив'язкою до лота і користувача
Chat / Message	Приватне листування після завершення торгів
Order / Cart	Замовлення з деталями доставки і складом
Code	Код для зміни email
Topic / Comment	Публікації спільноти з вкладеними коментарями
Review / Rating	Відгуки між учасниками угод
Favorites / Comparison	Збережені і порівнювані товари
Notification	Системні сповіщення про події
Followers	Підписки між користувачами

Перегляд каталогу та товарів. Дозволяє будь-якому відвідувачу переглядати каталог без необхідності реєстрації. Система надає розгалужену фільтрацію за брендом, ціновим діапазоном, типом механізму та матеріалом корпусу, а також сортування за ціною, новизною та популярністю. Сторінки каталогу та окремих товарів рендеруються на сервері (SSR), що забезпечує їх коректну індексацію пошуковими системами та швидке перше завантаження.

Управління кошиком та оформлення замовлення. Як зареєстрований так і незареєстрований користувач може додавати товари до кошика із зазначенням розміру та кількості, змінювати склад кошика, застосовувати промокоди для отримання знижки та вибирати спосіб доставки на інтерактивній карті через Нову пошту, кур'єром чи самовивозом. При успішній оплаті через WayForPay система фіксує поточну ціну товарів, формує запис замовлення в базі даних та надсилає підтвердження на електронну пошту користувача через сервіс Nodemailer.

Список бажань та порівняння товарів. Дозволяє користувачу зберігати товари, що зацікавили, для подальшого повернення до них. Модуль порівняння надає можливість зіставити технічні характеристики до чотирьох товарів однієї категорії одночасно, що суттєво полегшує прийняття рішення про купівлю.

Участь в аукціоні. Охоплює два сценарії. Виставлення товару на аукціон користувач обирає товар, встановлює початкову ціну, крок ставки, резервну ціну та тривалість торгів. Участь у торгах користувач переглядає активні лоти та робить ставки, а також має можливість придбати лот через кнопку “купити зараз”, якщо продавець додав таку можливість. Після завершення таймера система автоматично визначає переможця та надсилає відповідні сповіщення всім учасникам торгів а також формує чат між переможцем і продавцем.

Спільнота. Користувачі створюють пости для оцінки товару або обговорення важливих тем. Зацікавлені можуть коментувати записи інших та підписуватись на колег по хобі. Модератор відстежує спам, а також правильність заповнення і дотримань правил при спілкуванні.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		35

Адміністрування каталогу. Надає адміністраторам повний контроль над вмістом каталогу через окрему панель управління. Адміністратор може переглядати список усіх товарів з можливістю фільтрації, переглядати деталі окремої позиції та видаляти товари як поодинокі, так і масово. Усі адміністративні запити проходять подвійну перевірку валідності JWT-токену та наявності ролі адміністратора.

Наступним логічним кроком після визначення ролей та сценаріїв використання є побудова об'єктно-орієнтованої моделі платформи RoyalTick за допомогою діаграми класів, яка демонструє з яких базових сутностей складається система та як саме вони взаємодіють між собою.

Колекції бази MongoDB було розбито за шість логічних доменів перший керує користувачами (профілі, підписки, верифікаційні хеші та рейтинги репутації), а другий формує каталог з чотирьма видами товарів. Комерційну логіку, включаючи кошики, замовлення та промокоди, було винесено в третій блок, тоді як четвертий повністю реалізує аукціонні процеси від лотів і ставок до приватних чатів. П'ятий модуль закриває модуль спільноти, зберігаючи теми блогу, коментарі, списки обраного, порівняння та відгуки, а за роботу системних сповіщень відповідає останній, шостий блок.

Логіку зв'язків між сутностями бази даних було розділено на три типи, орієнтуючись об'єкти під час видалення. Жорстку залежність реалізує композиція дочірній запис повністю знищується слідом за батьківським, тому видалення лота автоматично стирає пов'язані ставки й чат, а видалення теми у блозі тягне за собою всі коментарі. Більш гнучкий підхід має агрегація, де об'єкти лишаються самостійними якщо адміністратор прибере якийсь аксесуар із каталогу, ця позиція все одно збережеться в обраному у користувачів. Останній тип, асоціація, фіксує суто авторство або участь у процесах, визначаючи сутність User як власника кошика (Cart), творця теми (Topic), ініціатора замовлення (Order) чи учасника торгів через ставку (Bid).

Для стандартизації станів об'єктів у системі використовуються фіксовані набори значень.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		36

Таблиця 2.3.

Переліки значень системи

Перелік	Значення
Роль користувача	guest, user, verified, moderator, admin
Категорія товару	watches, straps, boxes, care
Статус лота	active, ended, cancelled
Статус замовлення	pending, paid, shipped, delivered, cancelled
Тип авторизації	local, google
Статус верифікації	unverified, verified
Статус сповіщення	unread, read

Така структура системи забезпечує її гнучкість. Аукціонний модуль розвивається повністю незалежно від каталогу, а додавання нової категорії товарів не торкається логіки торгів чи спільноти. Завдяки цьому вебплатформа готова до будь-яких змін нові функції додаються швидко, а головне без ризику зламати те, що вже стабільно працює.

Таблиця 2.4.

Функціональні та нефункціональні вимоги

Категорія	Вимога	Опис та технологічний стек
Функціональні вимоги	Автентифікація	Реєстрація/вхід через email + пароль або Google (Firebase OAuth 2.0).
	Верифікація	Підтвердження мобільного номера та банківської картки. Валідація за алгоритмом Луна на фронтенді, захист даних через хеші bcryptjs.

Категорія	Вимога	Опис та технологічний стек
Функціональні вимоги	Каталог	Чотири категорії аксесуарів із власними наборами атрибутів. Параметричні фільтри, пошук, списки обраного та порівняння. Рендеринг через SSR для SEO.
	Замовлення	Робота з кошиком, активація промокодів. Інтерактивний вибір відділень Нової Пошти на карті. Проведення онлайн-оплат через шлюз WayForPay.
	Аукціон	Публікація лотів із фото та таймером торгів. Оновлення ставок у реальному часі, опція швидкого викупу «купити зараз». Автоматичний приватний чат після фіналу, оцінювання учасників угоди.
	Спільнота	Створення тем, вкладені коментарі, підтримка тегів та медіафайлів. Підписки на профілі колекціонерів, інструменти модерації контенту.

Категорія	Вимога	Опис та технологічний стек
Функціональні вимоги	Сповіщення	Розсилка email-листів через Nodemailer та робота внутрішніх push-сповіщень про нові ставки чи повідомлення.
	Адміністрування	Повний CRUD контенту каталогу, управління акаунтами та зміна ролей користувачів через ізольовану панель React Admin.
Нефункціональні вимоги	Архітектура	Фронтенд та основна логіка на Next.js (App Router), адмін-панель на базі Vite + React Admin. База даних – MongoDB.
	Продуктивність	Серверний рендеринг (SSR) для сторінок каталогу й активних лотів, клієнтський (CSR) - для динамічних компонентів. Затримка debounce на інпутах 400 мс.
	Безпека	Перевірка JWT-токенів та ролей через Next.js Middleware. Хешування паролів bcryptjs. HMAC-MD5 підписи для платіжних інфоблоків. Налаштування CORS та обов'язковий HTTPS.

Категорія	Вимога	Опис та технологічний стек
Нефункціональні вимоги	Інфраструктура	Контейнеризація Docker для сервісів shop та admin. Налаштований CI/CD пайплайн у GitLab. Деплоймент платформи на хостинг Vercel.
	Інтерфейс	Повна мультимовність інтерфейсу (UA/EN). Адаптивна верстка під мобільні та десктопні екрани від 320px.

Сформований перелік функціональних та нефункціональних вимог чітко визначає технічні межі системи, проте для безпосереднього переходу до етапу написання коду цього все ще недостатньо. Потрібно було детально візуалізувати внутрішню структуру даних та логіку їхньої взаємодії на рівні програмних модулів. Оскільки вебплатформа оперує великою кількістю сутностей від профілів користувачів до лотів аукціону виникла потреба об'єднати їх у єдину, зрозумілу для розробника мережу. Це дозволяє уникнути хаосу в коді та чітко розставити пріоритети. На основі шести доменів предметної області та з урахуванням специфіки нереляційних моделей у MongoDB, було спроектовано повну об'єктно-орієнтовану схему майбутнього маркетплейсу. Такий підхід допомагає наочно простежити, як саме жорсткі зв'язки композиції чи гнучкіші типи асоціацій впливатимуть на поведінку об'єктів під час їхнього видалення чи оновлення. Графічне представлення взаємозв'язків між усіма ключовими компонентами, внутрішніми полями кожної сутності та типами їхніх відношень наочно продемонстровано на діаграмі класів доменних моделей, що наведена нижче на (рис.2.2).

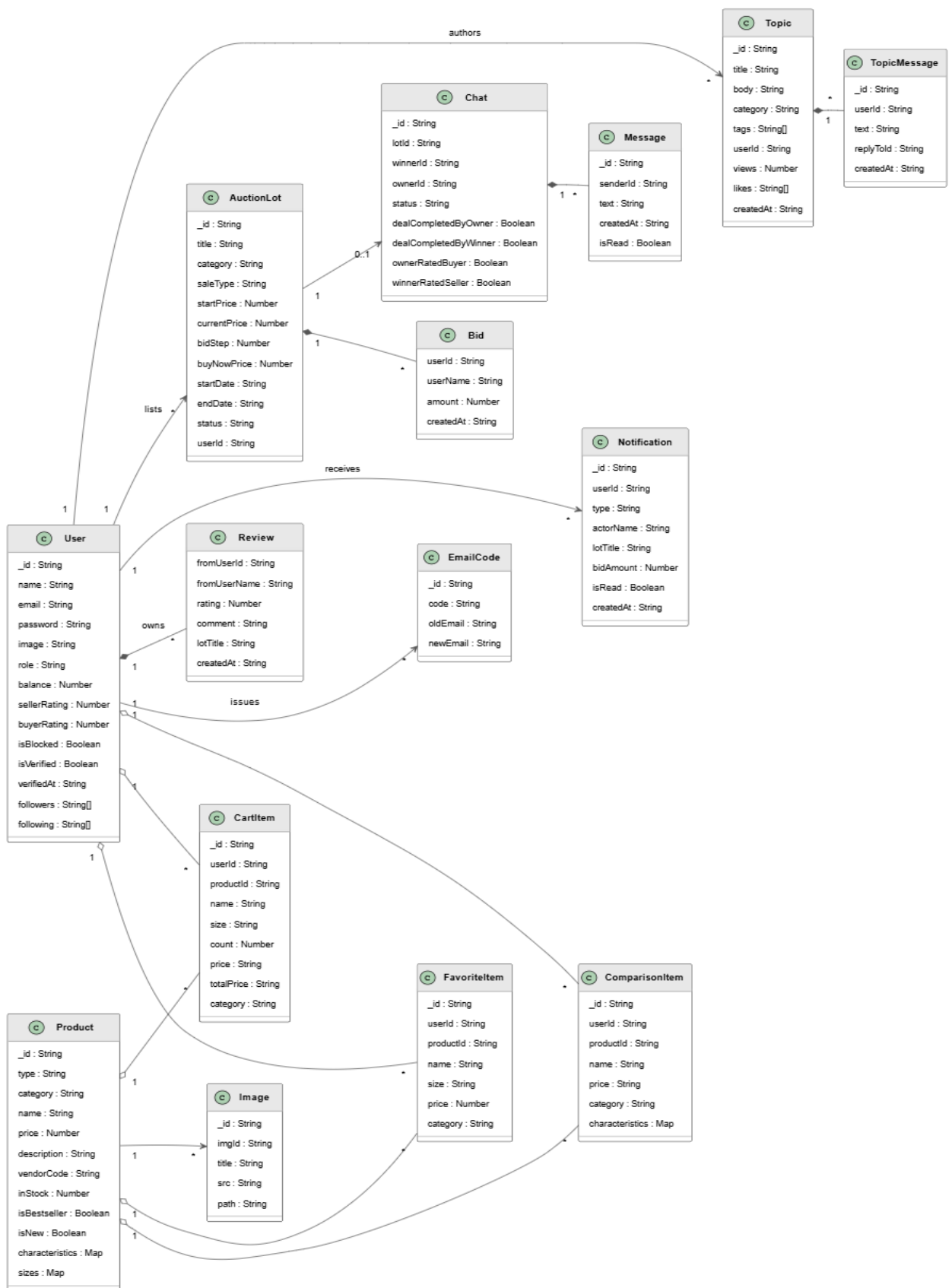


Рис. 2.2. Діаграма класів доменних моделей

Для детального відображення структури даних та архітектури розроблюваної системи було побудовано об'єктно-орієнтовану модель.

Скорочений варіант діаграми класів доменних моделей платформи RoyalTick наведено у див.рис.додаток.А. Вона містить вичерпний перелік усіх ключових сутностей, їхні логічні поля, типи даних, перелічення, а також деталізовані зв'язки між ними.

2.2. Розробка бази даних системи

Для збереження інформації вибрали документно-орієнтовану базу MongoDB, яку розгорнули в хмарі MongoDB Atlas. NoSQL-підхід став логічним через специфіку самого каталогу [20]. Оскільки чотири категорії товарів мають зовсім різні характеристики, жорстка реляційна структура змусила б або створювати купу порожніх полів у таблицях, або постійно писати важкі JOIN-запити. Гнучкий формат документів дозволив кожному аксесуару тримати в базі лише свій унікальний набір полів.

Усю структуру бази розбито на чотирнадцять колекцій, розподілених між логічними доменами застосунку. Користувачі зберігаються в users. Каталог товарів розділили на чотири окремі колекції watches, straps, boxes та case. Для комерційного блоку створено колекції cart, favorites та comparison. Торги та комунікація аукціону тримаються в lots та chats, а активність спільноти в topics. Також виділено кілька службових колекцій notifications, codes та images.

Головне архітектурне рішення при проєктуванні моделей чітко розмежування між посиланнями та вбудованими документами (embedded). Вбудовування обрали для тих даних, які логічно не існують без батьківського об'єкта. Так, масив ставок bids лежить прямо всередині документа лота, повідомлення messages лежить в чаті, а коментарі всередині теми блогу. Схожим чином вчинили з репутацією відгуки про користувача як про продавця (sellerReviews) чи покупця (buyerReviews) вбудовані безпосередньо в його профіль у users. Це дозволяє витягувати всю пов'язану інформацію за один швидкий запит до бази.

Посилання (через userId або productId) застосовано там, де сутності живуть своїм життям. Наприклад, записи в кошику, обраному чи порівнянні

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				42
Змн.	Арк.	№ докум.	Підпис	Дата		

зберігають лише id товару. Якщо адміністратор оновить інформацію в каталозі, зв'язки не зламаються. Цілісність цих даних контролюється вже на рівні коду самого Next.js застосунку. При цьому колекція users є ядром системи на неї через userId посилається більшість інших об'єктів, що дозволяє чітко визначати автора будь-якої дії на платформі. Для специфічних завдань реалізовано окрему логіку. Колекція codes потрібна суто для одноразових токенів при зміні пошти туди записується сам код, старий та новий email, а після успішного апдейту документ просто видаляється з бази. Через колекцію images адмінка React Admin тимчасово працює із завантаженими медіафайлами до моменту їхньої фінальної обробки.

Що стосується інфраструктури, то хмарний сервіс MongoDB Atlas сам закриває питання з автоматичними бекапами та масштабуванням, тому правити код при збільшенні навантаження не доведеться. Такий підхід до організації сховища даних дозволяє зберегти час, адже нам не доводиться писати складні міграції під час кожного дрібного оновлення моделей. Оскільки NoSQL-архітектура є дуже гнучкою, система може спокійно еволюціонувати разом із ростом самого маркетплейсу [28]. Вдале комбінування вбудованих документів для дрібних зв'язків таких, як ставки та повідомлення та класичних ідентифікаторів для великих сутностей допомагає підтримувати ідеальний баланс між швидкістю читання даних із бази та чистотою коду на сервері. Це мінімізує затримки при рендерингу сторінок на сервері (SSR) і гарантує, що користувачі отримуватимуть актуальні ціни та оновлення ставок без жодних фризів чи очікувань.

Для того, щоб детально розібратися, як саме всі ці чотирнадцять колекцій взаємодіють на практиці, де саме використовуються прямі посилання, а де дані вбудовуються всередину батьківських об'єктів, необхідно візуалізувати архітектуру сховища. Повну схему розподілу колекцій, назви внутрішніх полів та логіку зв'язків між ними детально зображено на (рис.2.3).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		43

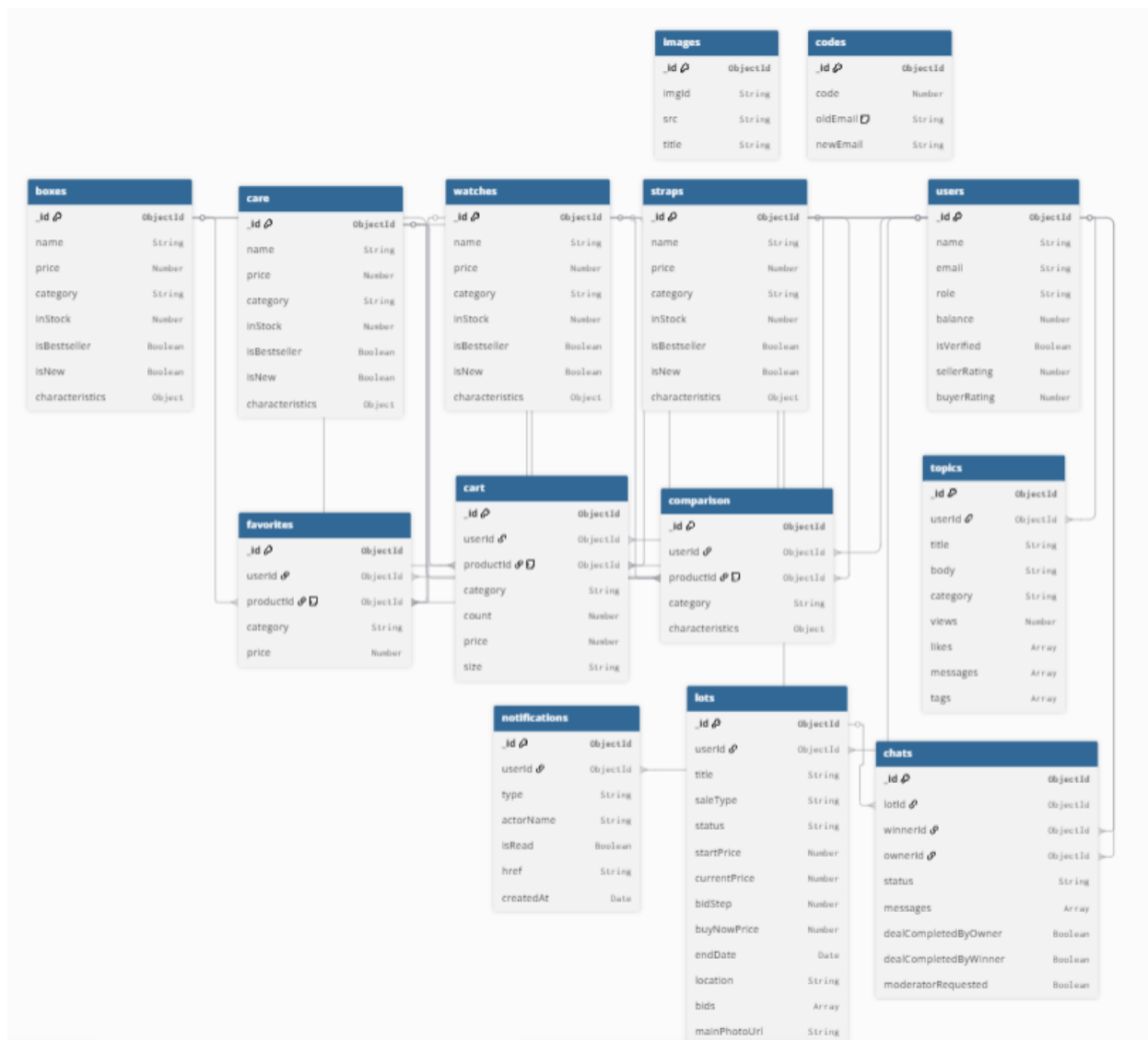


Рис. 2.3. Структура таблиц бази даних

Колекція watches є основною колекцією каталогу та зберігає документи годинників. Наявність масиву sizes безпосередньо в документі товару є перевагою документоорієнтованого підходу кожен елемент масиву містить поля size та count, що відображає залишок конкретного розміру на складі. Поля isNew та isBestseller дозволяють формувати відповідні секції на головній сторінці без додаткових запитів.

Колекція straps зберігає документи ремінців для годинників. На відміну від watches, документи цієї колекції не містять поля sizes та waterResistance, але мають специфічні поля color та width. Це наочно демонструє гнучкість MongoDB різні категорії товарів мають різну структуру документів в межах однієї бази даних.

Колекція `boxes` містить документи коробок для годинників з мінімальним набором полів `назва`, `бренд`, `ціна`, `зображення` та `матеріал`. Колекція `care` зберігає документи засобів догляду за годинниками з додатковим полем `careType`, що вказує на тип засобу.

Колекція `users` є центральною колекцією системи ідентифікації. Поля `provider` та `providerId` забезпечують підтримку OAuth-авторизації без необхідності окремої колекції для зовнішніх профілів якщо користувач авторизується через Google, поле `provider` матиме значення «google», а `providerId` відповідний ідентифікатор у системі Google. Поле `passwordHash` відсутнє або дорівнює `null` для таких акаунтів, оскільки їх автентифікація здійснюється виключно через Firebase Authentication [15].

Колекція `cart` зберігає позиції кошика кожного користувача у вигляді окремих документів `CartItem`. Зберігання поля `price` на момент додавання товару до кошика є свідомим архітектурним рішенням, яке захищає від ситуації, коли адміністратор змінює ціну товару між додаванням до кошика та оформленням замовлення користувач завжди бачить ту ціну, за якою додавав товар.

Колекція `favorites` зберігає позиції списку бажань. Колекція `comparison` зберігає позиції модуля порівняння з додатковим полем `type` для визначення типу порівняння в межах категорії. Обидві колекції зберігають лише посилання на товар через `productId` та `category`, не дублюючи жодного атрибута товару це забезпечує консистентність даних якщо адміністратор змінює назву або ціну товару, ці зміни одразу відображаються скрізь.

Колекція `auction` є центральною колекцією аукціонного модуля. Ключовим архітектурним рішенням є зберігання масиву `bids` безпосередньо в документі лоту. Кожен елемент масиву містить поля `Id`, `amount` та `placedAt`. Це забезпечує атомарність операції додавання нової ставки вся інформація про торги зберігається в одному документі, що унеможливлює ситуацію часткової записаності даних при збої. Після завершення торгів поля `winnerId`, `finalPrice` та `completedAt` заповнюються, а статус лоту змінюється на `ended`.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		45

Колекція users є головно., структура якої наведена в таблиці 2.5. Вона виступає центром всієї бази даних, оскільки більшість інших колекцій посилаються на неї через поле `userId`. Колекція зберігає не лише базові облікові дані, а й рейтинг користувача як продавця і покупця, вбудовані масиви відгуків, списки підписників та підписок, а також поля для блокування акаунту модератором.

Таблиця 2.5.

Опис колекції users

Поле	Тип	Обов'язкове	Опис
<code>_id</code>	ObjectId	так	Унікальний ідентифікатор користувача
<code>name</code>	String	так	Ім'я користувача
<code>password</code>	String	так	Хешований пароль (bcryptjs)
<code>email</code>	String	так	Електронна адреса користувача
<code>image</code>	String	ні	Шлях до зображення аватара профілю
<code>role</code>	String	так	Системна роль: user, verified, moderator, admin
<code>authSource</code>	String	так	Джерело авторизації: credentials (локальна), google
<code>authSource</code>	String	так	Авторизація: (локальна), google
<code>balance</code>	Number	так	Внутрішній баланс рахунку користувача

продовж. табл. 2.5

Поле	Тип	Обов'язкове	Опис
sellerRating	Number	так	Середній рейтинг користувача як продавця
sellerRatingsCount	Number	так	Кількість отриманих оцінок як продавця
buyerRating	Number	так	Середній рейтинг користувача як покупця
buyerRatingsCount	Number	так	Кількість отриманих оцінок як покупця
sellerReviews	Array	ні	Масив об'єктів-відгуків про роботу як продавця
following	Array (ObjectId)	ні	Масив ідентифікаторів (userId) користувачів, на яких підписаний сам
isBlocked	Boolean	так	Прапорець блокування користувача
blockReason	String	ні	Причина блокування користувача
blockedAt	Date	ні	Дата та час накладання блокування

продовж. табл. 2.5

Поле	Тип	Обов'язкове	Опис
blockedBy	ObjectId	ні	Посилання на модератора або адміна, який видав бан
buyerReviews	Array	ні	Масив вбудованих об'єктів-відгуків про роботу як покупця
followers	Array (ObjectId)	ні	Користувачі, які підписалися

Структуру основної колекції каталогу наведено в таблиці 2.6. Колекція watches зберігає інформацію про годинники. Окрім стандартних полів назви та ціни, є об'єкт characteristics з технічними параметрами брендом, механізмом, матеріалом корпусу та ремінця, типом скла і функціями. Поля isNew та isBestseller дозволяють формувати відповідні секції на головній сторінці без додаткових запитів до бази.

Таблиця 2.6.

Опис колекції watches

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор запису
category	String	так	Категорія товару
type	String	так	Тип годинника
name	String	так	Назва моделі товару
price	Number	так	Ціна годинника
description	String	ні	Текстовий опис моделі та її особливостей

продовж. табл. 2.6

Поле	Тип	Обов'язкове	Опис
characteristics	Object	так	Об'єкт із технічними характеристиками (бренд, механізм, матеріал корпусу, тип скла, матеріал ремінця, колір, стать, функції)
images	Array (String)	ні	Масив відносних шляхів до файлів зображень
vendorCode	String	так	Унікальний артикул товару в системі
inStock	Number	так	Кількість одиниць товару, яка є в наявності на складі
isBestseller	Boolean	так	Прапорець для виведення позиції у блок лідерів продажу
isNew	Boolean	так	Прапорець для маркування товару як новинки
popularity	Number	так	Лічильник переглядів
sizes	Object	ні	Словник доступних розмірів із булевими значеннями наявності
createdAt	Date	так	Дата та час створення документа в базі

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Євдемов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		49

продовж. табл. 2.6

Поле	Тип	Обов'язкове	Опис
updatedAt	Date	так	Дата та час останнього редагування запису

Колекція straps, описана в таблиці 2.7, зберігає інформацію про ремінці та браслети для годинників. На відміну від watches, ця колекція має набір характеристик ширину, довжину та матеріал. Це демонструє, що різні категорії товарів мають різну структуру документів в межах однієї бази даних без порожніх полів.

Таблиця 2.7.

Опис колекції straps

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор
category	String	так	Загальна категорія товару (значення: "straps")
type	String	так	Тип браслета або ремінця
name	String	так	Назва моделі товару
price	Number	так	Роздрібна ціна виробу
description	String	ні	Текстовий опис моделі та матеріалів
characteristics	Object	так	Об'єкт із характеристиками
images	Array (String)	ні	Масив відносних шляхів до зображень ремінця

продовж. табл. 2.7

Поле	Тип	Обов'язкове	Опис
vendorCode	String	так	Унікальний артикул товару в системі
inStock	Number	так	Кількість одиниць товару на складі
isBestseller	Boolean	так	Прапорець для відображення у блоці хітів продажу
isNew	Boolean	так	Прапорець для маркування товару як новинки
popularity	Number	так	Лічильник переглядів
sizes	Object	ні	Словник доступних розмірів (наприклад, {"180 / 18"})
createdAt	Date	так	Дата та час створення документа в базі
updatedAt	Date	так	Дата та час останнього редагування запису

Таблиця 2.8 описує колекцію boxes, яка зберігає документи коробок та шкатулок для зберігання годинників. Колекція має мінімальний набір полів матеріал, колір та місткість.

Таблиця 2.8.

Опис колекції boxes

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор

продовж. табл. 2.8

Поле	Тип	Обов'язкове	Опис
category	String	так	Загальна категорія товару
type	String	так	Конструктивний тип коробки
name	String	так	Назва моделі товару
price	Number	так	Ціна коробки
description	String	ні	Текстовий опис моделі, матеріалів та дизайну
characteristics	Object	так	Об'єкт із характеристиками
images	Array (String)	ні	Масив зображень шкатулки
vendorCode	String	так	Унікальний артикул товару в системі
inStock	Number	так	Кількість одиниць товару на складі
isBestseller	Boolean	так	Маркування товару як хіт продажу
isNew	Boolean	так	Маркування товару як новинки
popularity	Number	так	Лічильник переглядів
createdAt	Date	так	Дата та час створення документа в базі
updatedAt	Date	так	Дата та час останнього редагування запису

У таблиці 2.9 наведено структуру колекції care, яка зберігає засоби догляду за годинниками. Особливістю є поле careType, що дозволяє розрізняти підкатегорії засобів без створення окремих колекцій. Як і інші товарні колекції, care підтримує поля isNew та isBestseller для виведення на головній сторінці.

Таблиця 2.9.

Опис колекції care

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор запису засобу догляду
category	String	так	Загальна категорія товару (значення: "care")
type	String	так	Класифікація або тип засобу (наприклад professional тощо)
name	String	так	Назва товарної позиції
price	Number	так	Роздрібна ціна виробу
description	String	ні	Текстовий опис засобу та інструкція з використання
characteristics	Object	так	Об'єкт із фізичними властивостями
images	Array (String)	ні	Зображення товару
vendorCode	String	так	Унікальний артикул товару в системі

продовж. табл. 2.9

Поле	Тип	Обов'язкове	Опис
inStock	Number	так	Кількість одиниць товару на складі
isBestseller	Boolean	так	Прапорець для відображення у блоці хітів продажу
isNew	Boolean	так	Прапорець для маркування товару як новинки
popularity	Number	так	Лічильник переглядів
sizes	Object	ні	Розміри комплекту із булевими значеннями наявності

Центральною колекцією аукціонного модуля є `lots`, структура якої наведена в таблиці 2.10. В ній зберігається масив ставок `bids` безпосередньо в документі лота це забезпечує операції додавання нової ставки і дозволяє отримати повну інформацію про торги за один запит. Колекція також підтримує два типи продажу через поле `saleType`: класичний аукціон і продаж за фіксованою ціною, що визначає логіку відображення на сторінці лота.

Таблиця 2.10.

Опис колекції `lots`

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор лота
title	String	так	Назва лота
category	String	так	Категорія товару
subcategory	String	ні	Підкатегорія товару
description	String	так	Детальний опис лота

продовж. табл. 2.10

Поле	Тип	Обов'язкове	Опис
condition	String	так	Стан товару
saleType	String	так	Тип продажу
startPrice	Number	так	Початкова ціна лота
currentPrice	Number	так	Поточна ціна лота
bidStep	Number	так	Мінімальний крок ставки
reservePrice	Number	ні	Резервна ціна (мінімальна сума, за яку продавець готовий віддати лот)
buyNowPrice	Number	ні	Ціна миттєвої покупки (бліц-ціна)
startDate	Date	так	Дата та час початку торгів
endDate	Date	так	Дата та час завершення торгів
autoExtend	Boolean	так	Прапорець автоматичного продовження торгів при ставці в останні хвилини
location	String	так	Місцезнаходження товару
deliveryMethods	Array (String)	так	Доступні способи доставки
deliveryPayer	String	так	Хто платить за доставку

продовж. табл. 2.10

Поле	Тип	Обов'язкове	Опис
returnsAllowed	Boolean	так	Можливість повернення товару
guarantees	String	ні	Гарантії, які надає продавець
buyerComment	String	ні	Додатковий коментар для покупця
moderatorNote	String	ні	Службова нотатка модератора
videoUrl	String	ні	Посилання на відеоогляд товару
mainPhotoUrl	String	так	Шлях до головного фото (збережено в Cloudinary)
additionalPhotoUrls	Array (String)	ні	Масив посилань на додаткові фотографії лота
userId	ObjectId	так	Посилання на ідентифікатор продавця (ref: users)
userName	String	так	Публічне ім'я продавця
userEmail	String	так	Контактна електронна адреса продавця
status	String	так	Поточний статус лота: active, completed, cancelled

продовж. табл. 2.10

Поле	Тип	Обов'язкове	Опис
bids	Array	ні	Масив вбудованих об'єктів-ставок користувачів
comments	Array	ні	Масив вбудованих об'єктів-коментарів усередині лота
createdAt	Date	так	Дата та час створення лота в базі даних

Таблиця 2.11 описує колекцію chats, яка забезпечує комунікацію між продавцем і покупцем після завершення аукціону. Чат створюється автоматично системою після визначення переможця торгів. Повідомлення зберігаються як вбудований масив всередині документа чату. Колекція також підтримує механізм залучення модератора через поля moderatorId та moderatorRequested, а двосторонні прапорці dealCompletedByOwner і dealCompletedByWinner дозволяють контролювати завершення угоди обома сторонами.

Таблиця 2.11.

Опис колекції chats

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор чату
lotId	ObjectId	так	Посилання на лот, за яким створено угоду
lotTitle	String	так	Назва лота на момент створення чату

продовж. табл. 2.11.

Поле	Тип	Обов'язкове	Опис
lotPhoto	String	ні	Фото лота
winnerId	ObjectId	так	Ідентифікатор переможця
winnerName	String	так	Публічне ім'я покупця
ownerId	ObjectId	так	Ідентифікатор продавця
ownerName	String	так	Публічне ім'я продавця
messages	Array	ні	Масив вбудованих об'єктів-повідомлень
status	String	так	Поточний статус чату/угоди
createdAt	Date	так	Дата та час створення чату
unreadForOwner	Boolean	так	Прапорець наявності непрочитаних повідомлень для продавця
deletedFor	Array (ObjectId)	ні	Масив ідентифікаторів користувачів,
moderatorId	ObjectId	ні	Ідентифікатор модератора
moderatorName	String	ні	Публічне ім'я залученого модератора

продовж. табл. 2.11

Поле	Тип	Обов'язкове	Опис
moderatorRequested	Boolean	так	Прапорець виклику модератора
moderatorRequestedBy	ObjectId	ні	Ідентифікатор користувача який зробив виклик
dealCompletedByOwner	Boolean	так	Прапорець, що угода/отримання коштів підтверджені продавцем
dealCompletedByWinner	Boolean	так	Прапорець, що угода/отримання товару підтверджені покупцем
ownerRatedBuyer	Boolean	так	Прапорець, що вказує, чи залишив продавець відгук покупцю
winnerRatedSeller	Boolean	так	Прапорець, що вказує, чи залишив покупець відгук продавцю

Колекція cart, описана в таблиці 2.12, реалізує функціональність кошика для обох типів користувачів. Для зареєстрованих використовується поле `userId`, для анонімних поле `clientId` з унікальним ідентифікатором, що генерується на клієнті. Свідомим архітектурним рішенням є фіксація поля `price` на момент додавання товару це захищає користувача від ситуації коли адміністратор змінить ціну між додаванням товару до кошика та оформленням замовлення.

Таблиця 2.12.

Опис колекції cart (наведено ключові поля)

Поле	Тип	Обов'язкове	Опис
<code>_id</code>	ObjectId	так	Унікальний ідентифікатор позиції в кошику
<code>userId</code>	ObjectId	так	Посилання на зареєстрованого користувача (ref: users)
<code>clientId</code>	String	так	Ідентифікатор для збереження кошика анонімних користувачів
<code>productId</code>	ObjectId	так	Посилання на документ товару із каталогу
<code>category</code>	String	так	Категорія товару
<code>size</code>	String	ні	Обраний розмір аксесуара або діаметр корпусу

продовж. табл. 2.12

Поле	Тип	Обов'язкове	Опис
count	Number	так	Кількість одиниць товару в цій позиції
price	Number	так	Фіксована роздрібна ціна товару на момент додавання

У таблиці 2.13 наведено колекцію favorites, яка зберігає список бажань користувачів. Колекція зберігає лише посилання на товар через productId і category, не дублюючи жодного атрибуту товару. Якщо адміністратор змінить назву або ціну товару в каталозі, зміни одразу відобразяться скрізь без необхідності оновлювати записи в favorites.

Таблиця 2.13.

Опис колекції favorites (наведено ключові поля)

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор позиції
userId	ObjectId	так	Посилання на zareestrovannogo користувача (ref: users)
clientId	String	так	Унікальний ідентифікатор анонімних користувачів

продовж. табл. 2.13

Поле	Тип	Обов'язкове	Опис
productId	ObjectId	так	Посилання на товар із загального каталогу
category	String	так	Категорія товару
price	Number	так	Актуальна ціна товару

Таблиця 2.14 описує колекцію comparison, яка забезпечує роботу модуля порівняння товарів. На відміну від favorites, ця колекція додатково зберігає вбудований об'єкт characteristics з характеристиками товару. Це дозволяє відображати таблицю порівняння без додаткових запитів до каталогу, що прискорює рендеринг сторінки порівняння.

Таблиця 2.14.

Опис колекції comparison (наведено ключові поля)

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор
userId	ObjectId	так	Посилання на зареєстрованого користувача
clientId	String	так	Унікальний ідентифікатор анонімних користувачів
productId	ObjectId	так	Посилання на конкретний товар із каталогу

продовж. табл. 2.14

Поле	Тип	Обов'язкове	Опис
category	String	так	Категорія товару
characteristics	Object	ні	Об'єкт із характеристиками товару для порівняння

Колекція **topics** наведена в таблиці 2.15, забезпечує роботу модуля спільноти платформи. Повідомлення та коментарі до публікації зберігаються як вбудований масив **messages** всередині документа теми аналогічно до підходу з ставками в **lots**. Поля **likes** та **views** дозволяють відображати активність на публікації без додаткових запитів.

Таблиця 2.15.

Опис колекції **topics** (наведено ключові поля)

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор теми
title	String	так	Заголовок теми
body	String	так	Текст публікації
category	String	так	Категорія теми
userId	ObjectId	так	Посилання на автора (ref: users)
messages	Array	ні	Масив вбудованих об'єктів-коментарів
views	Number	так	Кількість переглядів теми

У таблиці 2.16 наведено структуру колекції notifications, яка реалізує систему сповіщень платформи. Поле type визначає тип події і може приймати значення. Поле href містить готове посилання для швидкого переходу до джерела події, що дозволяє користувачу одразу перейти до відповідного лота чи чату.

Таблиця 2.16.

Опис колекції notifications (наведено ключові поля)

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор
userId	ObjectId	так	Посилання на користувача
type	String	так	Тип події: bid_on_lot, bid_outbid, new_message, lot_restored, account_blocked
actorName	String	так	Публічне ім'я користувача, який ініціював подію
isRead	Boolean	так	Статус прочитання сповіщення
href	String	так	URL-посилання для швидкого переходу до джерела події

Таблиця 2.17 описує колекцію codes, яка використовується виключно для процесу зміни електронної адреси. Після того як користувач підтверджує новий email, документ з кодом автоматично видаляється з колекції. Такий підхід гарантує що одноразові коди не накопичуються в базі і не займають зайвого місця.

Таблиця 2.17.

Опис колекції codes

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор запису верифікації
code	Number	так	Числовий код верифікації / підтвердження
oldEmail	String	так	Поточна електронна адреса користувача
newEmail	String	так	Нова електронна адреса, яка потребує підтвердження

У таблиці 2.18 наведено структуру колекції images, яка використовується адмін-панеллю React Admin для тимчасового зберігання завантажених медіафайлів. Після фінальної обробки зображення передаються до хмарного сервісу Cloudinary, а посилання на них записуються безпосередньо в документи товарів каталогу.

Таблиця 2.18.

Опис колекції images (наведено ключові поля)

Поле	Тип	Обов'язкове	Опис
_id	ObjectId	так	Унікальний ідентифікатор запису
imgId	String	так	Ідентифікатор завантаженого зображення
rawFile.src	String	так	Blob URL зображення для локального відображення
rawFile.title	String	так	Початкова назва файлу зображення

2.3. Проєктування та реалізація алгоритмів роботи системи

Діаграма активності на (рис.2.4) демонструє, що доступно у розділі спільноти від перегляду обговорень до публікації нових тем та коментарів. Кожен може переглядати сторінки, фільтрувати теми за категоріями і читати обговорення. Однак перегляд теми зараховується лише для авторизованих користувачів система оновлює лічильник і додає `userId` до масиву `viewedBy`, щоб уникнути накруток. Для неавторизованих перегляди не зараховуються. Різниця між гостем та зареєстрованим користувачем виникає при створенні теми або написанні коментаря. Система перевіряє JWT-токен, і якщо його не знайдено, користувача перенаправляють на сторінку входу. Коли користувач створює тему він заповнює вводиться, текст, категорію, теги та може додати до чотирьох фотографій. Зображення завантажуються на Cloudinary, а в колекцію `topics` зберігається новий документ із порожніми даними `messages`, `likes` та `viewedBy`. Коментарі можна написати окремо або у відповідь іншому користувачу через поля `replyToId` та `replyToUserName`. Після надсилання

коментаря сервер перевіряє, щоб поле не було порожнім, і додає повідомлення до масиву messages. Також користувач може ставити лайки, якщо перший клік додає userId до масиву likes, то повторний - видаляє.

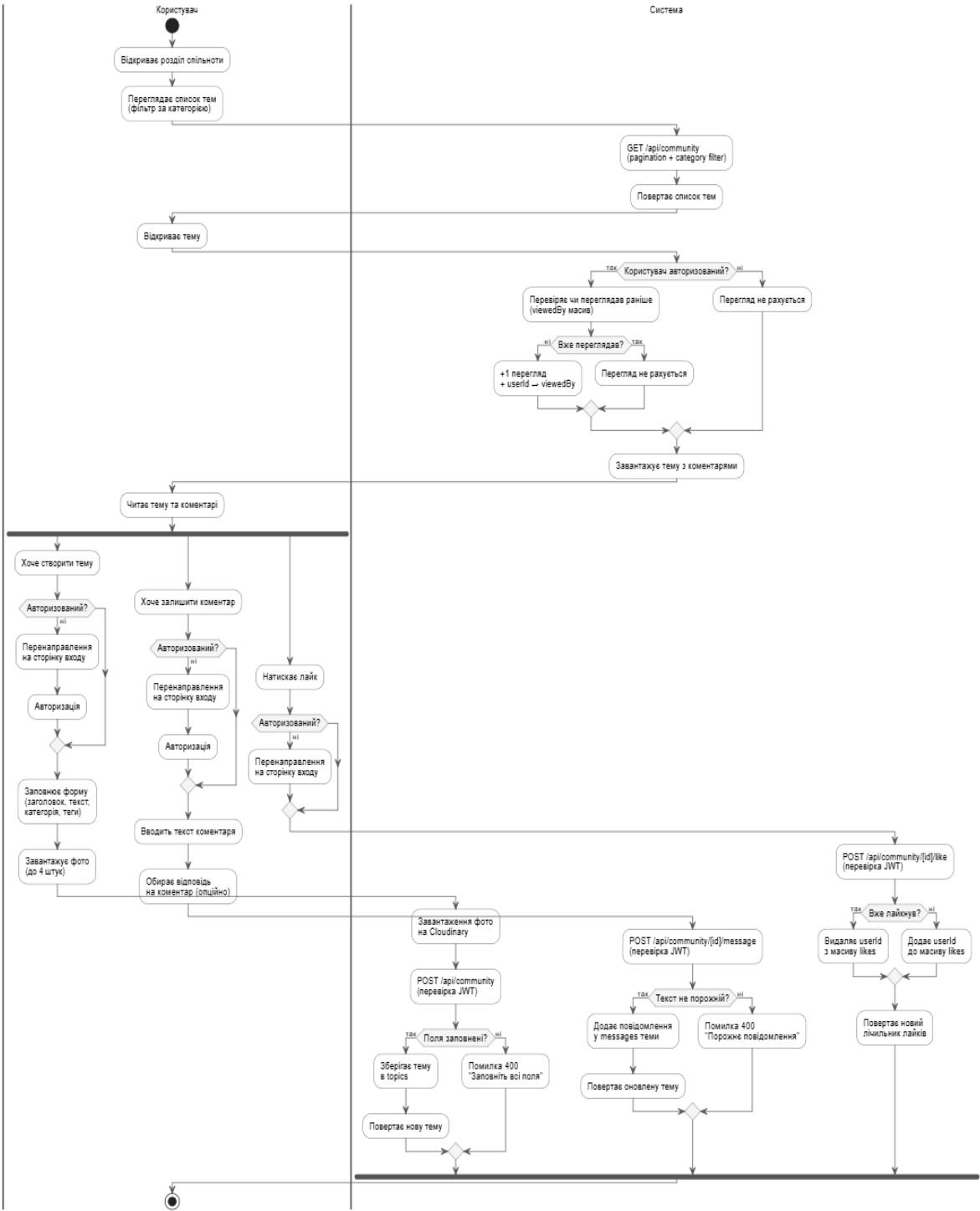


Рис. 2.4. Діаграма активності для створення обговорення в розділі спільноти

Діаграма активності на (рис.2.5) демонструє, як влаштований пошук товарів та оформлення замовлення - від перегляду каталогу до отримання листа на пошту.

Користувач відкриває каталог, де може фільтрувати товари за кількома параметрами категорією, ціною, розміром, кольором та сортуванням. Система збирає ці параметри і робить запит GET /api/goods. Він будує фільтр для MongoDB і повертає список товарів. Також є звичайний пошук через рядок, який працює від двох символів. Система шукає збіги за назвою, описом, артикулом чи категорією одразу в усіх чотирьох колекціях бази даних. Коли користувач відкриває картку товару, він обирає розмір і додає його до кошика. Тут є два варіанти. Якщо користувач не увійшов в акаунт, товар просто зберігається в localStorage і сервер не чіпається. Якщо ж він авторизований, хук useCartAction перевіряє кошик. Якщо такий товар з цим розміром вже є, кількість збільшується через PATCH /api/cart/update. Якщо товару ще немає, створюється новий запис через POST /api/cart/add.

Окремо в системі працює механізм для синхронізації кошика. Якщо користувач додавав товари в кошик ще до того, як увійшов в акаунт, вони лежать у localStorage. Після успішної авторизації система автоматично переносить їх на сервер. Хедер сайту постійно відстежує зміну стану isAuth. Коли статус змінюється на true, система зчитує весь кошик з localStorage і відправляє його через POST /api/cart/add-many. Точно так само це працює для обраного та порівняння товарів. Завдяки цьому користувач не втрачає свої товари, незалежно від того, на якому етапі він вирішив авторизуватися.

При оформленні замовлення користувач вводить ім'я, прізвище, телефон, email та обирає спосіб доставки. Для Нової Пошти підтягується карта TomTom, де можна вибрати потрібне відділення.

Після підтвердження даних система звертається до POST /api/payment, який робить об'єкт із підписом HMAC-MD5 для WayForPay. Користувача перенаправляє на сторінку оплати. Після успішного платежу WayForPay шле callback на POST /api/success-callback. Тоді система надсилає підтвердження замовлення на email через Nodemailer і очищає кошик.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		68

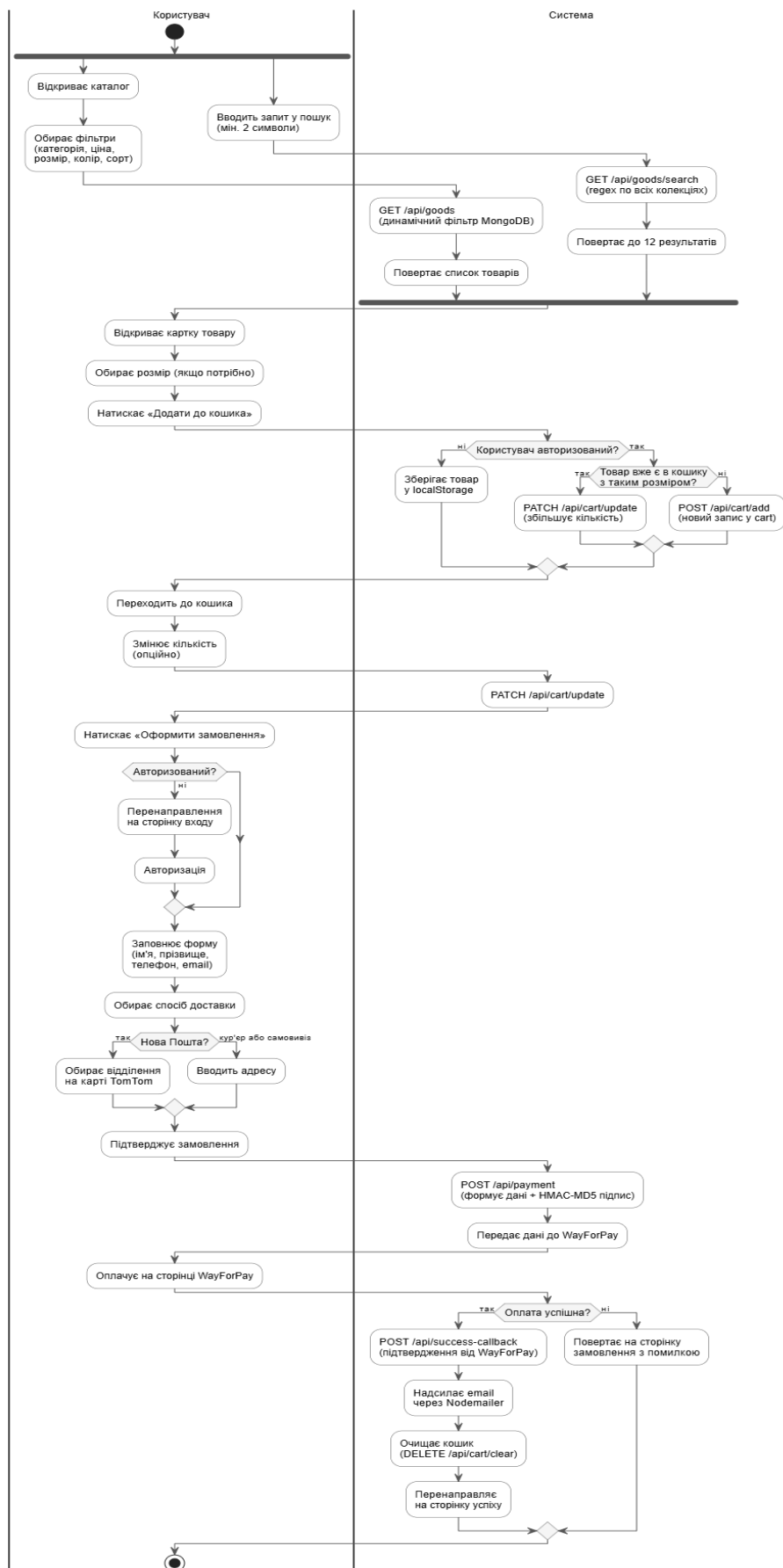


Рис. 2.5. Діаграма активності для пошуку товарів і створення замовлення

Діаграма активності показує два сценарії для аукціонів як виставити свій лот і як брати участь у торгах. Це головна частина всієї платформи. Коли користувач відкриває сторінку аукціону, система запускає два процеси у фоні.

Перший - це POST /api/auction/lots/finalize. Процес шукає лоти, де час закінчився і є хоча б одна ставка. Переможцем стає той, хто поставив останнім. Статус лота змінюється на reserved, і створюється приватний чат, куди продавець одразу отримує вітальне повідомлення.

Другий процес - POST /api/auction/lots/restore-inactive. Він перевіряє чати, які стоять без діла більше трьох днів. Якщо переможець нічого не пише, система сама блокує його акаунт. Лот повертається в активний продаж із новим таймером на сім днів, а користувачі отримують сповіщення. Щоб виставити свій лот, треба увійти в акаунт і мати гроші на балансі для оплати комісії. Система перевіряє баланс ще перед тим, як показати форму. Якщо грошей вистачає, відкривається форма. Там заповнюється назва, опис, стартова ціна, крок ставки, дата закінчення та умови доставки. Також можна вказати ціну для швидкої покупки. Головне фото і ще до чотирьох картинок завантажуються на Cloudinary [19].

Після підтвердження система знімає комісію, а лот зберігається в колекцію lots зі статусом active і пустим масивом bids. Щоб зробити ставку на лот, система робить кілька перевірок по черзі. Перевіряється JWT-токен, статус лота, таймер, чи не заблокований акаунт і чи це не власний лот користувача. Також сума ставки має бути не меншою за поточну ціну плюс крок ставки. Якщо все добре, нова ставка додається в масив bids. Поточна ціна оновлюється, а система надсилає два сповіщення продавцю про нову ставку (bid_on_lot) і минулому лідеру про те, що його перебили (bid_outbid). Цей алгоритм допомагає уникнути проблем, коли кілька людей намагаються поставити ставку в одну і ту саму секунду. Послідовність дій користувача та реакцію сервера під час роботи з торгами наочно показано на діаграмі активності (рис.2.6).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				70
Змн.	Арк.	№ докум.	Підпис	Дата		

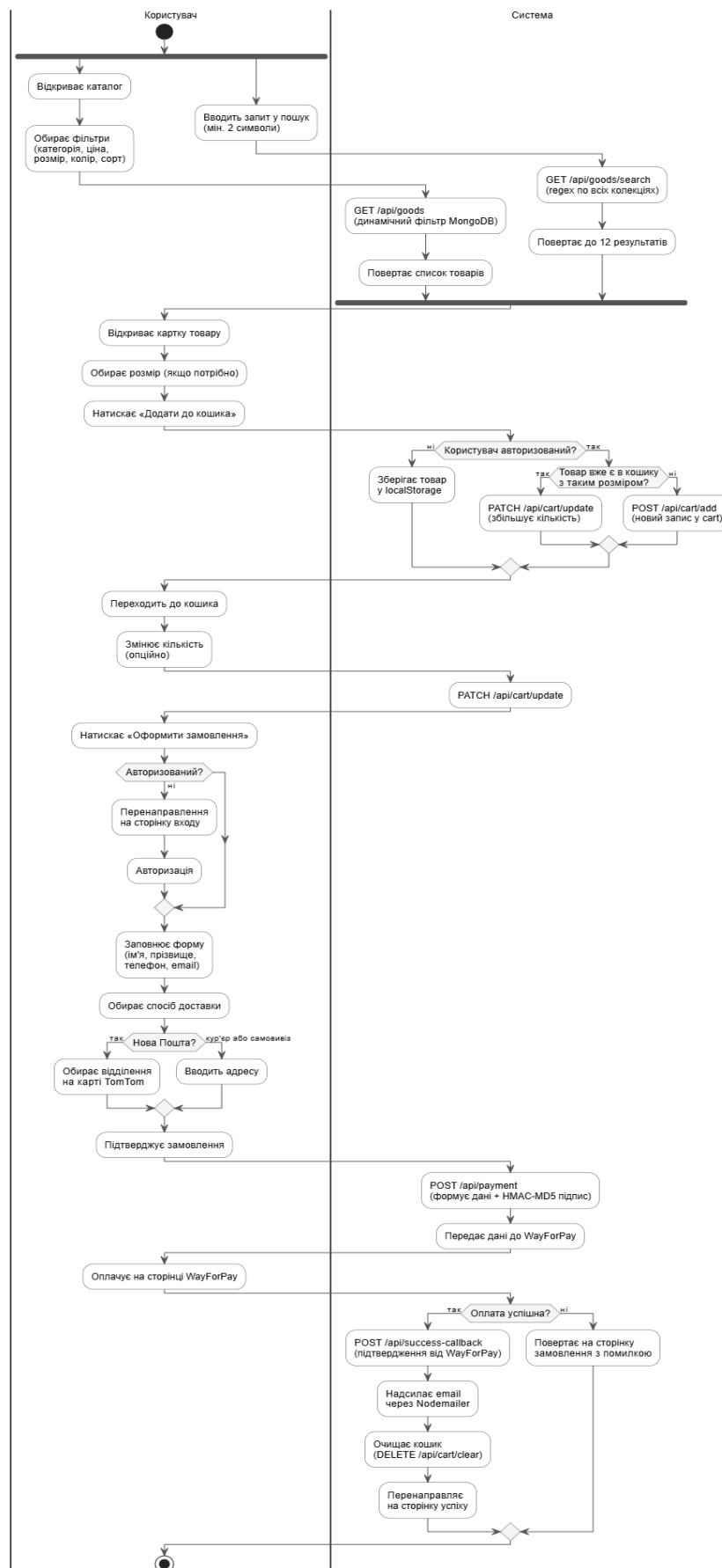


Рис. 2.6. Діаграма активності для виставлення лоту та ставки на лот

Діаграма послідовності на показує, як взаємодіють між собою учасники після закінчення аукціону від створення чату до закриття угоди та виставлення оцінок [7]. Коли таймер лота закінчується, процес `/api/auction/lots/finalize` у фоні знаходить переможця і сам створює чат у колекції `chats`. Туди одразу додається вітальне повідомлення від продавця. Статус самого лота змінюється на `reserved`, після чого обидва користувачі бачать цей чат у своєму списку.

Коли користувач заходить в чат, клієнт робить запит `GET /api/chats/[id]`. Сервер перевіряє, чи є цей користувач учасником або модератором, а потім робить усі непрочитані повідомлення від співрозмовника прочитаними. Коли надсилається нове повідомлення через `POST /api/chats/[id]/message`, сервер записує його як вбудований документ. Далі він знаходить одержувача і шле йому сповіщення `new_message`. Якщо виникає якась суперечка, кожен може покликати модератора через `POST /api/chats/[id]/invite-moderator`, і той отримає доступ до всієї історії переписки.

Чат оновлюється сам кожні три секунди завдяки `setInterval` на клієнті, тому сторінку перезавантажувати не потрібно. Якщо користувач вирішив приховати чат і натиснув кнопку видалення, його `_id` просто додається до масиву `deletedFor`. Однак, як тільки співрозмовник надсилає нове повідомлення, сервер одразу очищає цей масив `deletedFor`, і чат знову з'являється у списку в обох учасників.

Угода закривається за допомогою підтвердження з обох сторін. Кожен учасник має натиснути кнопку завершення. Запит `POST /api/chats/[id]/complete` заповнює потрібне поле `dealCompletedByOwner` або `dealCompletedByWinner`. Тільки коли обидва ці поля стануть `true`, система сама змінить статус чату і лота на `completed`. Після цього користувачі можуть поставити один одному оцінки, які записуються в профілі. Вся послідовність цих кроків, починаючи від моменту завершення таймера і закінчуючи виставленням фінальних відгуків, наочно відображена нижче на (рис.2.7).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		72

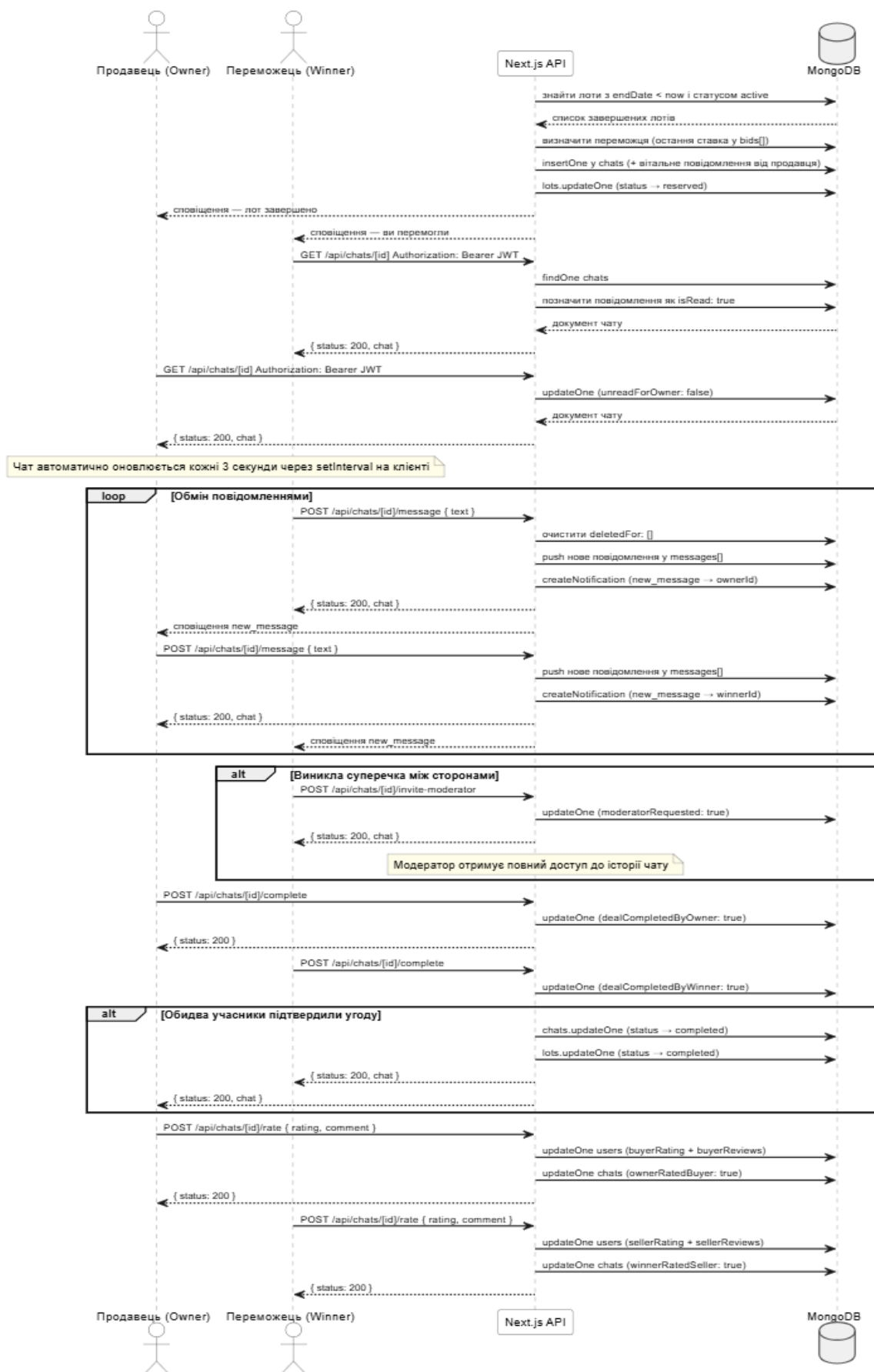


Рис. 2.7. Діаграма послідовності процесу ставки, завершення торгів та приватного чату

Діаграма послідовності показує, як користувач, сервер і платіжна система WayForPay взаємодіють між собою, під час оформлення покупки. Юзер перевіряє кошик де він може змінювати кількість товарів через запит PATCH /api/cart/update або взагалі викидати непотрібні позиції. Після цього відкривається звичайна форма, де треба написати контактні дані й вибрати спосіб доставки. Для Нової Пошти інтегрована карта TomTom, щоб обирати потрібне відділення.

Коли все пройшло успішно, на сервер летить запит POST /api/payment із сумою та інформацією про кошик. Сервер одразу генерує унікальний orderReference. Потім усі параметри збираються в рядок і підписуються через HMAC-MD5 за допомогою секретного ключа продавця. Цей готовий об'єкт перенаправляється назад на фронтенд, і браузер одразу перекидає користувача на платіжну сторінку WayForPay. Потім WayForPay шле callback на ендпоінт POST /api/success-callback. Сервер підписує відповідь і віддає статус асерт, щоб платіжна система зрозуміла, що все пройшло успішно. Якщо оплату відхилили або користувач просто закрити вікно WayForPay, платіжна система присилає callback зі статусом Declined. Сервер отримує його на ендпоінт /api/success-callback, але кошик не чистить і листа на пошту не відправляє. Браузер перекидає користувача назад на сайт і показує помилку. Усі товари в кошику залишаються на місці, тому можна спокійно спробувати оплатити ще раз.

Паралельно з цим вмикається процес POST /api/payment/notify. Він комбінує HTML-лист із усім складом замовлення, адресою та номером транзакції, а Nodemailer відправляє його покупцю на email. В кінці кошик повністю очищається через DELETE /api/cart/clear. Такий підхід гарантує повну синхронізацію даних між кошиком користувача, платіжним шлюзом та базою даних сайту, повністю виключаючи ризик втрати замовлення в разі раптового обриву інтернет-з'єднання. Повний цикл обміну запитом та логіку взаємодії всіх залучених сервісів наочно представлено нижче на діаграмі послідовності (рис.2.8).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		74

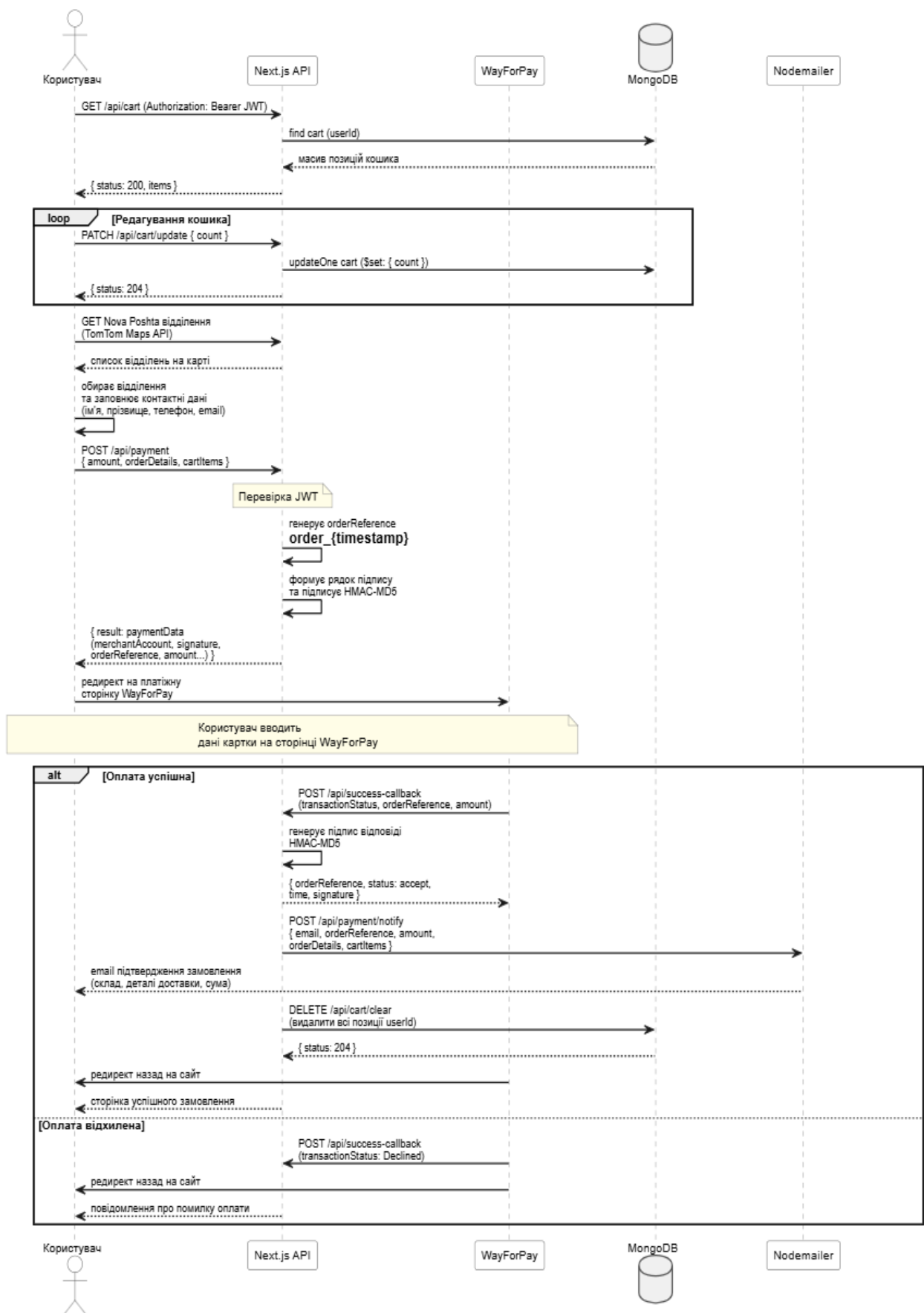


Рис. 2.8. Діаграма послідовності оформлення замовлення та обробка платежу

Діаграма кооперацій на (рис.2.9) показує, як пов'язані між собою різні частини системи, коли обробляється угода по аукціону. Тут головне не час, як на діаграмі послідовності, а те, які саме об'єкти спілкуються один з одним і якими повідомленнями вони обмінюються.

Головним вузлом у цій схемі є колекція lots. До неї звертаються відразу кілька частин системи. Наприклад, API ставок міняє поточну ціну й додає запис у масив bids. Процес фіналізації у фоні змінює статус лота на reserved. А коли обидва користувачі підтверджують угоду, компонент чату переводить лот у кінцевий статус completed.

Сервіс сповіщень працює відразу з трьома різними компонентами. Він шле повідомлення, коли з'являється нова ставка, коли закінчуються торги або коли хтось пише в чат. Колекція users оновлюється у двох випадках якщо треба автоматично заблокувати неактивного переможця, або коли зберігаються оцінки після закриття угоди. Сам чат працює як місток між аукціоном та системою рейтингів.

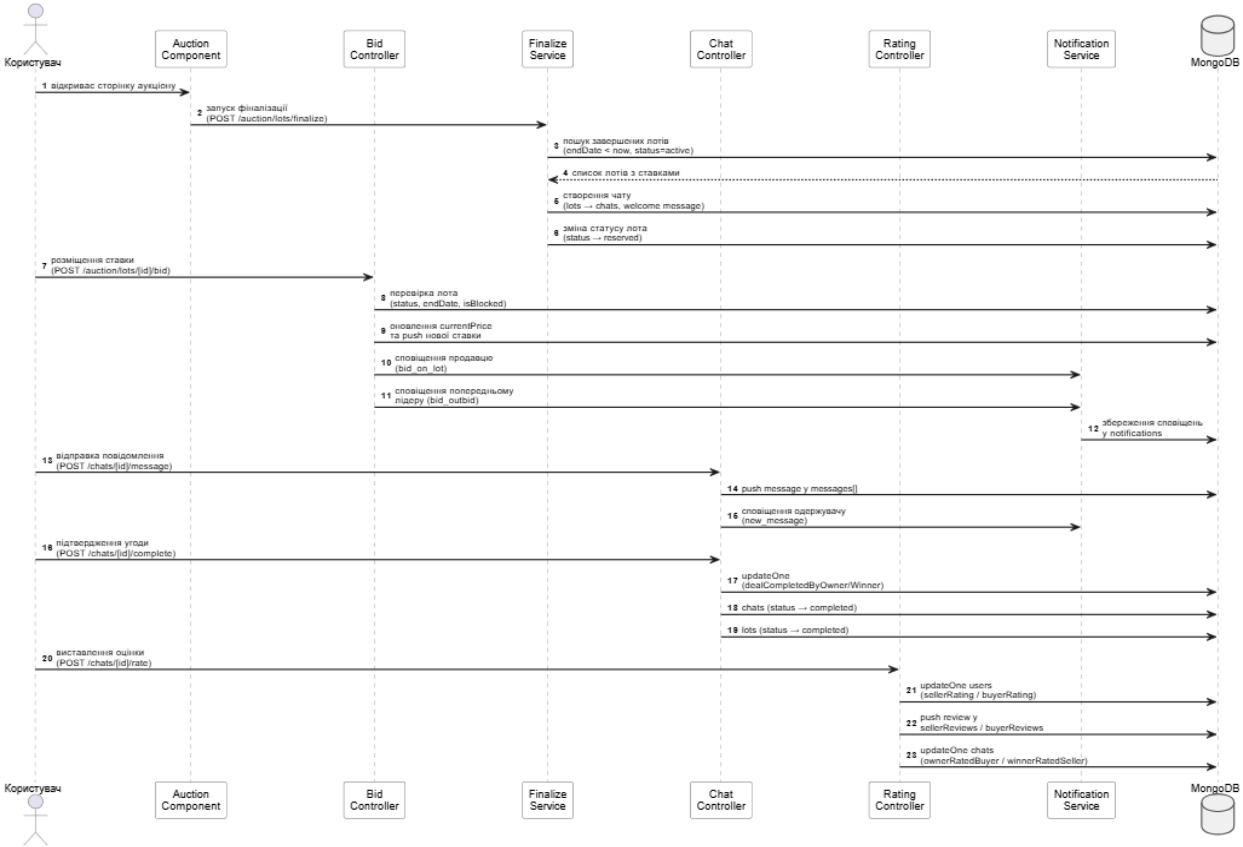


Рис. 2.9. Діаграма кооперацій взаємодія при аукціонній угоді

2.4. Реалізація функціоналу вебплатформи RoyalTick

Уся робота між клієнтом і сервером побудована навколо авторизації [17]. Без неї не вийде зробити жодну захищену дію - наприклад, додати товар у кошик, виставити лот на аукціон, підтвердити акаунт чи оплатити покупку. Сама авторизація працює на парі JWT-токенів, які мають різний термін дії. Коли користувач успішно заходить в акаунт, функція generateTokens робить два токени access token на десять хвилин і refresh token на тридцять днів. Access token зберігає ім'я та email користувача, цей токен додається до кожного захищеного запиту. А refresh token містить тільки email. Він потрібен лише для того, щоб оновлювати сесію, і користувачу не доводилося постійно знову вводити пароль.

Перед тим як видати токени, сервер завжди перевіряє дані користувача. Ендпоінт POST /api/users/login шукає користувача в базі за її email. Далі він порівнює введений пароль із тим хешем,

що вже лежить у базі, за допомогою функції bcrypt.compareSync. Якщо пароль підійшов, сервер повертає обидва токени, а також роль та ім'я користувача. Усі ці дані сайт відразу записує в localStorage.

На кожному захищеному маршруті токен із заголовка Authorization Bearer перевіряється функцією isValidAccessToken. Вона передає цей токен у jwt.verify з тим самим секретним ключем. Якщо є якась помилка - наприклад, токен застарів, підпис неправильний або токена взагалі немає тоді функція відразу повертає помилку { status: 401 }. Ще є функція parseJwt, яка просто декодує корисні дані (payload) токена без нової перевірки. Вона просто розбирає другий сегмент токена, який закодований у base64. Це потрібно для того, щоб швидко дізнатися email користувача і використовувати його далі для запитів до бази.

Отже, JWT - це перший рівень захисту для всіх дій на сайті. Але щоб брати участь в аукціонах, цього мало. Потрібен пройти верифікацію акаунта. Вона підтверджує, що за профілем стоїть реальна людина з унікальним телефоном і карткою. Це заважає створювати купу фейкових акаунтів, щоб

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				77
Змн.	Арк.	№ докум.	Підпис	Дата		

просто накручувати ставки. Сама перевірка номера картки працює через алгоритм Луна. Це звичайний математичний метод, яким перевіряють контрольні суми у всіх платіжних системах. Алгоритм просто перебирає цифри справа наліво і подвоює кожен другу з них. Якщо після подвоєння виходить число більше за дев'ять, то від нього віднімають дев'ять. Наприкінці всі цифри додаються, і якщо загальна сума ділиться на десять без остачі, то з картою все добре.

Після перевірки формату сервер дивиться, чи немає вже таких даних в інших верифікованих акаунтах. Для цього він бере всіх перевірених користувачів і порівнює їхні дані з новими через `bcrypt.compareSync`. Це потрібно для того, щоб один користувач не зміг верифікувати кілька різних профілів однією й тією ж картою чи телефоном. Якщо все унікально, то телефон і картка хешуються з окремою `bcrypt`-сіллю і записуються в базу разом із датою верифікації. У відкритому вигляді ці дані ніде й ніколи не зберігаються.

Тепер, коли авторизація та верифікація захищають систему, можна розібрати саму логіку аукціону. Ендпоінт для ставок `POST /api/auction/lots/[id]/bid` є найскладнішим маршрутом у проекті. Перед тим як записати нову ставку в базу, він по черзі робить ланцюжок із шести різних перевірок [4].

Цей ланцюжок робить перевірки по порядку. Спочатку сервер дивиться на токен, потім перевіряє, чи взагалі існує цей лот і який у нього статус. Далі він перевіряє статус самого користувача, і тільки потім чи правильна сума ставки. Коли ставка успішно записалася, система шле два сповіщення: продавцю про те, що з'явилася нова ставка, і минулому лідеру про те, що його перебили. За це відповідає функція `createNotification`. Вона просто записує новий документ у колекцію `notifications` із типом події, іменем того, хто її зробив, та посиланням для переходу. Після закінчення торгів вебплатформа обробляє всі результати. Для цього є процес `POST /api/auction/lots/finalize`,

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				78
Змн.	Арк.	№ докум.	Підпис	Дата		

який запускається щоразу, коли хтось відкриває сторінку аукціону. Він шукає лоти, у яких час уже вийшов і де є хоча б одна ставка. Після цього система

Завдяки налаштуванню `upsert true` чати повністю захищені від дублікатів. Навіть якщо викликати буде здійснено кілька разів поспіль, система не буде створювати копії, а просто пропустить цей крок, якщо чат уже є. Увесь цей процес логічно закінчується модулем продажів користувач переходить від перегляду каталогу до моменту оплати. Саму платіжну систему WayForPay було підключено через підписи HMAC-MD5. Працює це так сервер спочатку сам генерує унікальний `orderReference`, потім зліплює дев'ять параметрів замовлення в один рядок і закриває все це секретним ключем продавця.

Такий підпис надійно захищає дані від підміни, щоб ніхто не зміг поміняти суму. WayForPay перевіряє цей підпис, коли отримує запит, і якщо щось не збігається, транзакцію буде скасовано. Коли оплата проходить успішно, платіжка шле `callback` назад на наш сервер. Сервер підписує відповідь у відповідь і віддає статус `accept`. Одночасно з цим Nodemailer надсилає покупцю лист із усім складом замовлення, а сам кошик повністю вичищається.

У підсумку весь цей функціонал утворює чітку й безпечну систему. JWT закриває питання з безпечним доступом до функцій сайту. Перевірка карти через алгоритм Луна та `bcrypt` захищає аукціон від злоумисників які будуть використовувати це в своїх цілях. Сама логіка ставок із купою перевірок стежить за чесністю торгів, а автоматична фіналізація сама закриває аукціонні лоти. Ну і наприкінці платіжний шлюз із підписом HMAC-MD5 гарантує, що гроші та замовлення пройдуть безпечно.

Як тільки аукціон добігає кінця, система самостійно відкриває приватний чат між продавцем та покупцем, який став переможцем. За цей процес у фоні відповідає запит `POST /api/auction/lots/finalize`. При цьому новий запис у базі даних створюється із налаштуванням `upsert: true`, що повністю застраховує систему від появи дублікатів чатів, навіть якщо запит випадково

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				79
Змн.	Арк.	№ докум.	Підпис	Дата		

пройде кілька разів поспіль. Щоб користувачам було простіше зорієнтуватися, у діалог одразу падає автоматичне привітання, яке підтверджує успішне завершення торгів та описує наступні кроки.

Сам чат працює дуже спритно і не вимагає постійного перезавантаження сторінки користувачем оновлення даних відбувається автоматично кожні три секунди через звичайний polling на клієнті. Коли людина заходить у діалог, спрацьовує запит GET /api/chats/[id]. На цьому етапі сервер обов'язково перевіряє «чужих»: читати переписку дозволено лише безпосереднім учасникам угоди або модератору. Крім того, під час кожного відкриття вікна чату система сама перевіряє потік повідомлень і миттєво позначає всі нові репліки від співрозмовника як прочитані.

Звісно, на аукціонах бувають різні ситуації, тому на випадок суперечок чи конфліктів ми передбачили кнопку виклику арбітра через запит POST /api/chats/[id]/invite-moderator. Після її натискання для чату вмикається статус moderatorRequested, і цей діалог одразу з'являється у робочій панелі адміністрації. Модератор, який береться за вирішення проблеми, отримує повний доступ до поточної історії повідомлень та може спілкуватися з учасниками від свого імені, щоб допомогти знайти компроміс.

Фінал угоди побудований на принципі взаємної згоди. Щоб закрити продаж, і продавець, і покупець мають підтвердити, що все пройшло успішно. Натискання кнопки завершення відправляє запит POST /api/chats/[id]/complete та фіксує згоду у відповідних полях бази даних dealCompletedByOwner або dealCompletedByWinner. Система терпляче чекає на обидва підтвердження, і як тільки обидва прапорці стають true, статус лота та самого чату автоматично змінюється на completed. Лише після цього відкривається доступ до взаємного оцінювання. Користувачі можуть залишити один одному відгуки, які напрямую записуються в їхні профілі як sellerRating чи buyerRating і формують загальний рейтинг репутації на майданчику.

Також ми врахували ризик того, що переможець торгової сесії може просто зникнути. Для боротьби з такими «мовчунами» було створено окремий

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				80
Змн.	Арк.	№ докум.	Підпис	Дата		

фоновий скрипт `POST /api/auction/lots/restore-inactive`. Він регулярно сканує базу даних на наявність чатів, де немає жодного руху вже понад сім днів. Якщо покупець так і не вийшов на зв'язок, система діє рішуче: автоматично заморожує його профіль за порушення правил, повертає лот назад на торги з новим чистим таймером та розсилає усім учасникам сповіщення про перезапуск аукціону.

2.5. Контейнеризація та автоматизація розгортання

Щоб усе працювало стабільно і проект запускався, обидва застосунки платформи було запаковано в Docker-контейнери та підключено до CI/CD пайплайну в GitLab [21]. Завдяки цьому код, який пройшов перевірку в пайплайні працює скрізь однаково як локально, так і на сервері. Основний застосунок на Next.js збирається за допомогою багатоетапного Dockerfile (multi-stage build). Збірка розбита на три послідовні кроки. Кожен етап робить свою роботу і передає далі лише ті артефакти, які дійсно потрібні для запуску, прибираючи все зайве.

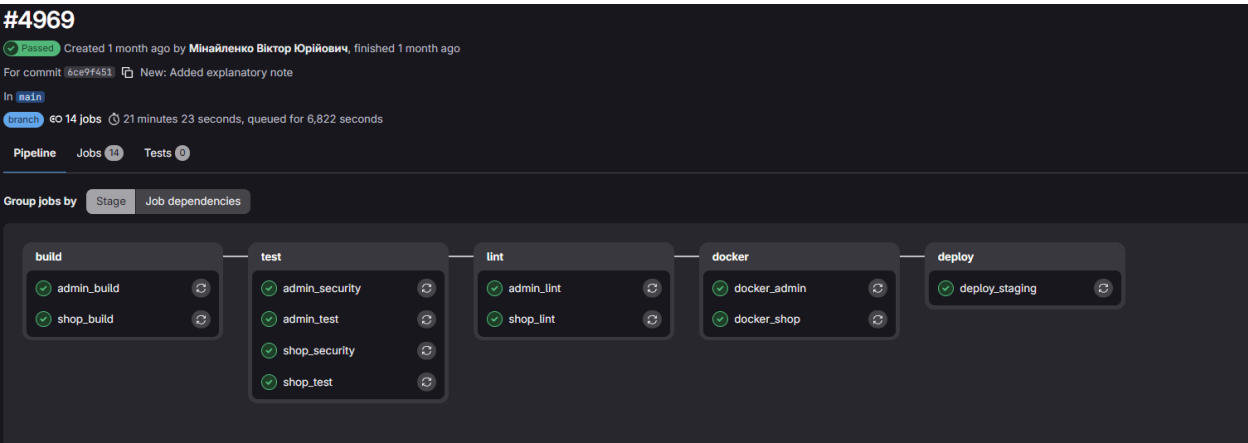


Рис. 2.10. GitLab CI/CD пайплайн

На першому етапі `deps`, ставляться всі залежності. Для цього береться мінімальний образ `node:20-alpine`, щоб усе ставилося в ізольованому середовищі. Також туди додається пакет `libc6-compat` він потрібен, щоб нормально працювали деякі нативні модулі Node.js.

Далі компілюються готові залежності, змінні середовища через ARG записуються у файл `.env`. Усі важливі ключі та секрети (як-от для WayForPay,

Cloudinary чи Firebase) передаються саме як параметри під час складання, тому вони не відображаються у відкритому вигляді в кінцевому образі. Також відключаємо надсилання звітів у Next.js, щоб під час збірки проєкт не віддавав зайвої статистики.

Далі переходимо до runner. Він потрібен для того, щоб зібрати максимально легкий образ для роботи сайту. Копіюються туди лише три найважливіші речі публічні файли, готову standalone-збірку та статичні ресурси (assets). Для безпеки весь цей процес запускається не від суперкористувача (root), а від імені звичайного системного користувача nextjs. Це стандартне правило, якщо раптом контейнер зламають, у зловмисника просто не буде прав нічого зіпсувати всередині системи. Що стосується адмінки на Vite, то вона лежить в окремому, набагато простішому контейнері. Там ми не робили складну багатоетапну збірку, тому що вона запускається в режимі розробки й не потребує якихось супер-оптимізацій.

Автоматизація збірки і деплою реалізована через GitLab CI/CD пайплайн з п'ятьма послідовними стадіями.

На етапі збірки (build) готуються до роботи обидві частини сайту і сам магазин, і адмінка. Проєкт завантажує всі потрібні пакети й збирає їх у готові робочі файли. Ці файли зберігаються одну годину, щоб система могла передати їх на наступні перевірки. Потім запускається етап тестів (test). Тут автоматично перевіряється, чи все працює правильно, і окремо сканується безпека проєкту, щоб у коді не було критичних вразливостей. Наприкінці працює етап перевірки коду (lint). Переглядається весь написаний код, щоб усе було написано акуратно, без сміття і за правилами розробки. Якщо дрібних зауважень менше ніж сто, то система пропускає код далі. На етапі роботи з Docker збирається фінальний образ проєкту. Усі важливі налаштування та секретні ключі GitLab автоматично підставляє із захищених змінних. Готовий образ підписується унікальним кодом (хешем коміту), щоб завжди було чітко видно, яка саме це версія, а також отримує мітку latest. Цей етап запускається не завжди, а лише тоді, коли зміни вносяться в головні гілки - main або develop.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		82

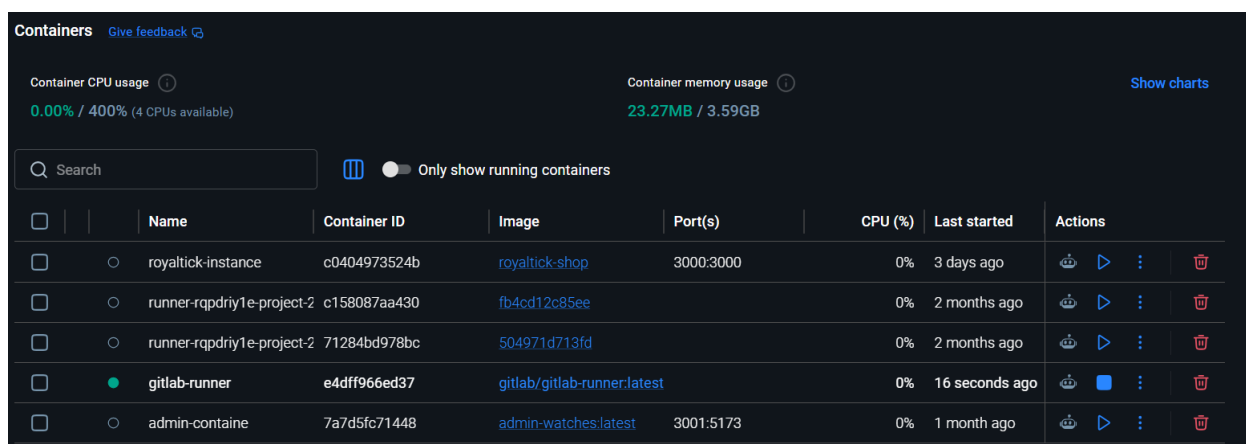


Рис. 2.11. Docker Desktop з контейнерами

Розглянемо кожну стадію цього пайплайну окремо. Перший етап це build, де система паралельно збирає обидва проєкти магазин на Next.js та панель адміністратора на Vite. На цьому кроці підтягуються всі потрібні залежності, а вихідний код компілюється у готові до роботи артефакти. Зібрані файли залишаємо в пам'яті лише на одну годину. Цього якраз вистачає, щоб спокійно передати їх далі по ланцюжку. Завдяки такому рішенню не доводиться щоразу наново встановлювати npm-пакети на наступних кроках, що непогано економить час і прискорює весь процес збірки.

Після білду настає крок tests тут за це відповідають тести на Jest, які перевіряють, чи не зламалася серверна логіка та утилітарні функції після нових завантажень. Паралельно система запускає аудит безпеки через команду npm audit. Вона сканує всі пакети на наявність помилок або вразливостей. Якщо скрипт знайде щось критичне, пайплайн одразу зупиниться, а деплой заблокується, поки розробник не виправить проблему.

За правильність коду відповідає стадія lint. ESLint ретельно сканує весь TypeScript та JSX код, перевіряючи його на відповідність стандартам чи правильно написані хуки React, чи немає змінних, які ніде не використовуються, і чи скрізь порядок із типізацією. Щоб не стопорити розробку через дрібниці, було виставлено ліміт у триста попереджень. Якщо помилок менше цього порогу, стадія вважається пройденою і код пропускається далі.

Фінальні етапи це docker та deploy. На стадії роботи з Docker збираємо фінальний образ системи, але робиться це виключно для головних гілок main та develop. Тратити ресурси сервера на збірку контейнерів для кожної поточної робочої гілки просто немає сенсу. Ну і на самому фініші, як тільки образ успішно готовий, автоматично стартує деплой.

Висновки до другого розділу

Було проведено аналіз функціональних вимог і визначено п'ять рівнів доступу з чітким розмежуванням повноважень між гостем, зареєстрованим користувачем, верифікованим користувачем, модератором та адміністратором.

Побудовано діаграму варіантів використання і діаграму класів що охоплює чотирнадцять колекцій MongoDB. Визначено функціональні та нефункціональні вимоги до системи. Спроектовано документно-орієнтовану базу даних на MongoDB Atlas.

Описано структуру всіх колекцій з таблицями полів і типів даних. Для моделювання динаміки роботи системи розроблено шість UML-діаграм три діаграми активності що описують взаємодію зі спільнотою, оформлення замовлення та аукціонні торги, дві діаграми послідовності для аукціонного чату і платіжного флоу та діаграму кооперацій що відображає структурні зв'язки між компонентами.

Реалізовано п'ять ключових механізмів платформи JWT-автентифікація з парою токенів різного терміну дії, верифікація акаунту через алгоритм Луна з bcrypt-хешуванням реквізитів, платіжна інтеграція з WayForPay через HMAC-MD5 підпис та автоматична фіналізація торгів.

Контейнеризацію реалізовано через Docker multi-stage build з запуском процесу від непривілейованого користувача. Налаштовано GitLab CI/CD пайплайн з автоматичною перевіркою якості коду і деплой на Vercel при кожному коміті в основну гілку.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		84

РОЗДІЛ 3. РОЗГОРТАННЯ, ІНТЕРФЕЙС ТА ТЕСТУВАННЯ ВЕБПЛАТФОРМИ ROYALTICK

3.1. Порядок розгортання та налаштування системи

Вебплатформа RoyalTick побудована як модульний моноліт, де сам сайт і його серверна логіка об'єднані в один застосунок на Next.js.

Адмінка - це Vite-застосунок на базі React Admin. Такий підхід дає нам змогу вносити зміни небоючись зламати існуючий функціонал. Щоб усе працювало стабільно і запускалося, обидва застосунки запаковані в Docker-контейнери [20].

Загальну схему розгортання системи показано на (рис.3.1).

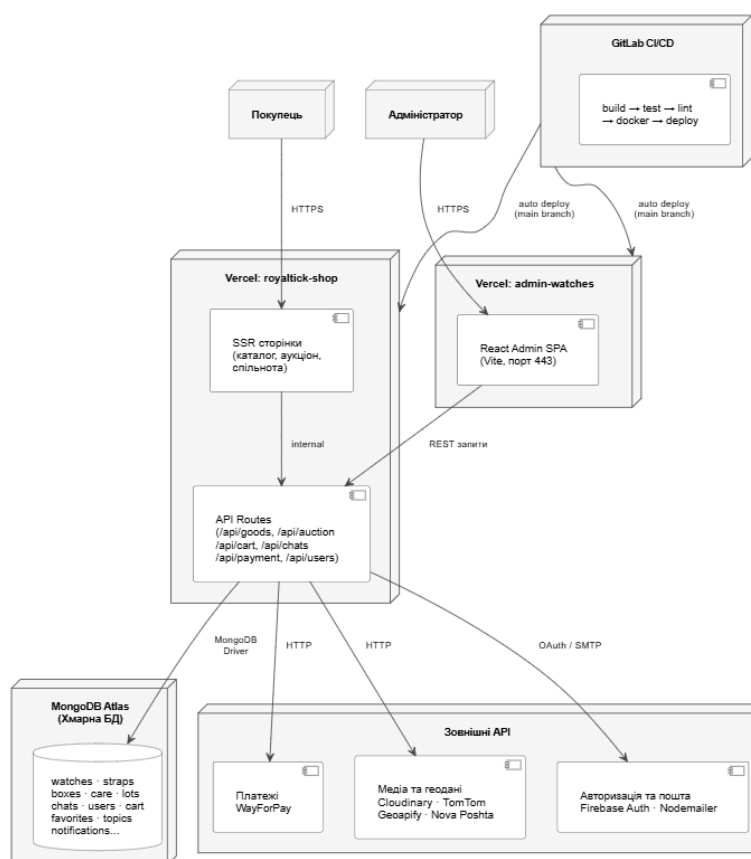


Рис. 3.1. Схема розгортання системи

Коли людина переходить по сайту, її браузер надсилає запити на хостинг Vercel, де розгорнуто Next.js застосунок. Сервер реагує на дії користувача або миттєво збирає сторінку на своїй стороні (SSR) і віддає її в готовому вигляді, або ж звертається до хмарної бази даних MongoDB Atlas, щоб завантажити чи зберегти свіжі дані. Якщо дія стосується зовнішніх сервісів завантаження

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		85

картинок товарів, проведення оплати чи швидкий вхід через Google, то до Next.js підключено відповідні зовнішні API. На схемі 3.1 відображено повний цикл взаємодії користувача з маркетплейсом RoyalTick.



Схема 3.1 - Взаємодія користувача з платформою

Усі деталі про те, як влаштований сайт зсередини, показано на схемі 3.2. На ній чітко видно, з яких частин складається система і як вони пов'язані між собою.

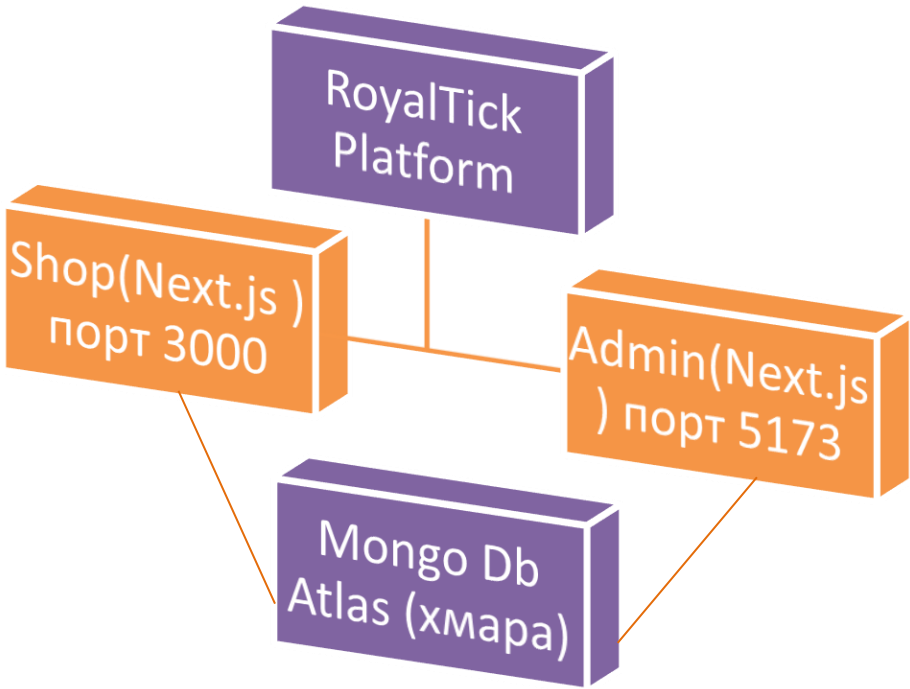


Схема 3.2 - Компоненти системи

Вебплатформа RoyalTick ділиться на дві частини. Спочатку сам сайт аукціонного магазину на Next.js. Сторінки каталогу, аукціонів та блогу створюються прямо на сервері, через це сайт відкривається швидко. Усі запити від користувачів та адмін панелі проходять через API Routes. Панель адміністратора зроблена на React Admin, працює як окремий застосунок на Vite і просто підключається до загального сервера. Уся інформація надійно зберігається в хмарі MongoDB Atlas. Застосунки виставлено на Vercel.

Таблиця 3.1.

Системні вимоги для розгортання платформи

Параметр	Мінімальні вимоги	Рекомендовані вимоги
CPU	2 ядра, 2.0 GHz	4 ядра, 3.0 GHz+
RAM	4 ГБ	8 ГБ
Накопичувач	20 ГБ SSD	50 ГБ SSD
Мережа	10 Мбіт/с	100 Мбіт/с
ОС	Ubuntu 20.04 / Windows 10	Ubuntu 22.04 LTS
Node.js	V15+	v20 LTS
Docker	v24+	v26+

Розгортання платформи можливе трьома способами залежно від середовища і потреб.

Таблиця 3.2.

Варіанти розгортання платформи RoyalTick

Через Docker	Вручну	Vercel (хмара)
Встановити Docker v24+	Встановити Node.js v20	Підключити репозиторій GitLab до Vercel
Клонувати репозиторій і заповнити .env	Виконати npm install та npm run build	Вказати змінні середовища в панелі Vercel
Виконати docker build з усіма змінними середовища	Скопіювати .next/standalone на сервер	При коміті в main деплой відбувається автоматично
Запустити контейнер на порту 3000	Запустити через PM2 або systemd	Перевірити статус у розділі Deployments

Особливу увагу приділено безпеці всі ключі й паролі від Firebase, WayForPay, Cloudinary і MongoDB сховані в захищених змінних середовища. У відкритому коді репозиторію їх немає. Весь обмін даними між користувачем і сервером йде через безпечне HTTPS-з'єднання, а спеціальні налаштування CORS стежать, щоб до внутрішнього API не міг достукатися ніхто сторонній. Сайт чудово працює на будь-яких пристроях і в усіх популярних браузерях таких як Google Chrome, Mozilla Firefox, Microsoft Edge і Safari. Інтерфейс повністю адаптивний, тому все відображається навіть на невеликих смартфонах із шириною екрану від 320 пікселів.

3.2. Структура інтерфейсу вебплатформи

Вебплатформа для з продажу годинників з аукціонним модулем працює в браузері через локалхост та розміщено на сервері у відкритому доступі.

Посилання на сайт(на захист): <https://royal-tick-mu.vercel.app/>

При вході на сайт користувач бачить головну сторінку магазину з великим рекламним банером. На ньому видно рекламний слоган і рекламні товари. Зверху шапка магазину з кнопками пошуку, обраного, кошику, порівняння і профілю. Також є кнопка меню для взаємодії з іншим контентом сайту. Знизу cookie-попап з кнопкою прийняти або можна натиснути хрестик щоб закрити цей попап[30]. На рис. 3.1. можна побачити детально як виглядає ця сторінка.

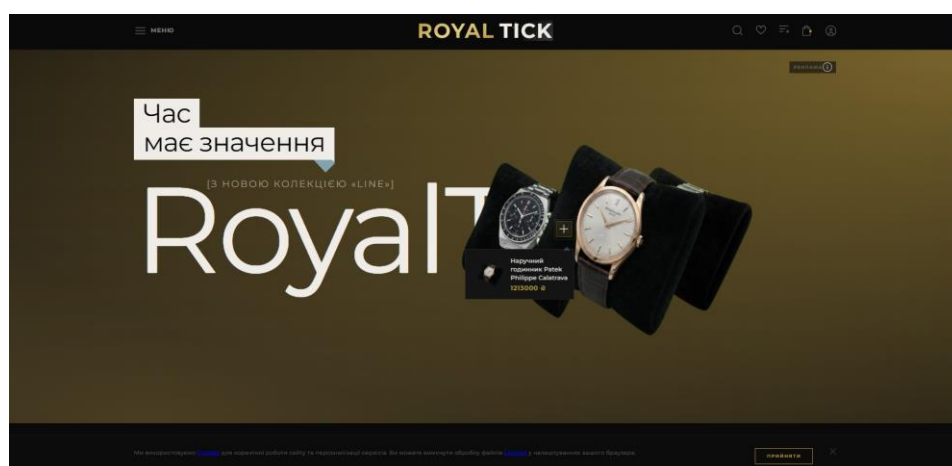


Рис. 3.1. Вигляд головної сторінки сайту

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				88
Змн.	Арк.	№ докум.	Підпис	Дата		

Прогорнувши нижче користувач бачить блок з категоріями де може обрати доступну категорію або настинути кнопку усі, щоб перейти на каталог. Ще ниж зникає блок новинок (рис.3.2).

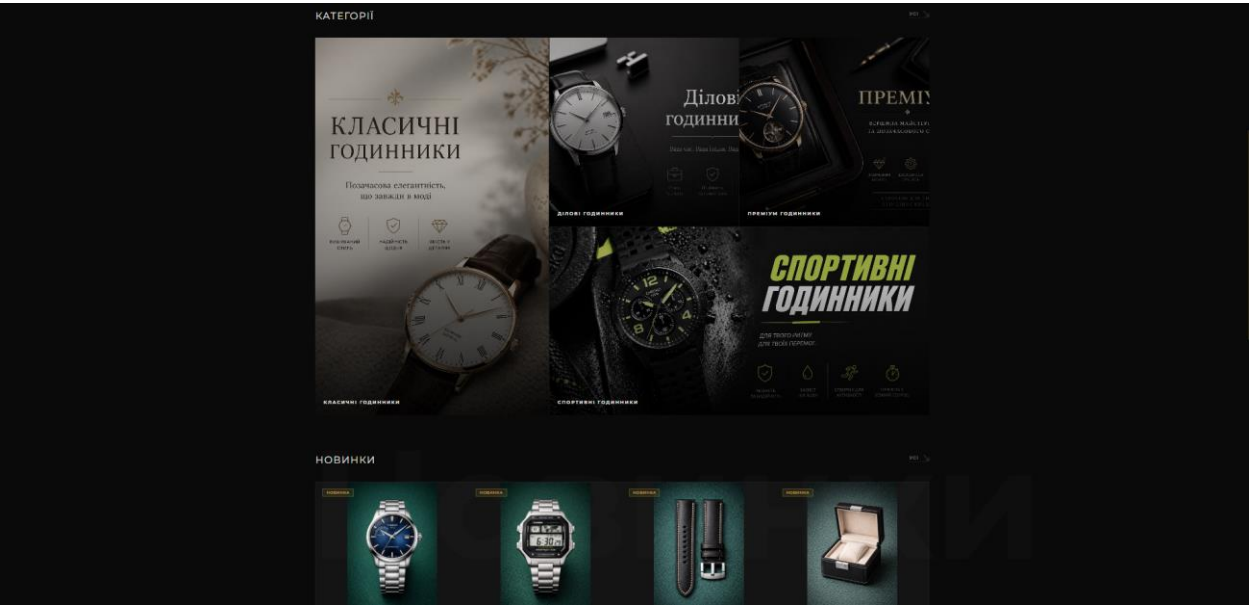


Рис. 3.2. Блок категорій і новинок на головній сторінці

Далі користувач бачить секцію «Хіти продажу» з картками товару. По кліку на картку користувач переходить на сторінку товару. Також присутня кнопка переходу на каталог з переглядом всім товарів які є хітами. Внизу футер з логотипом, контактами, посиланнями на соц. мережі та політику конфіденційності (рис.3.3).

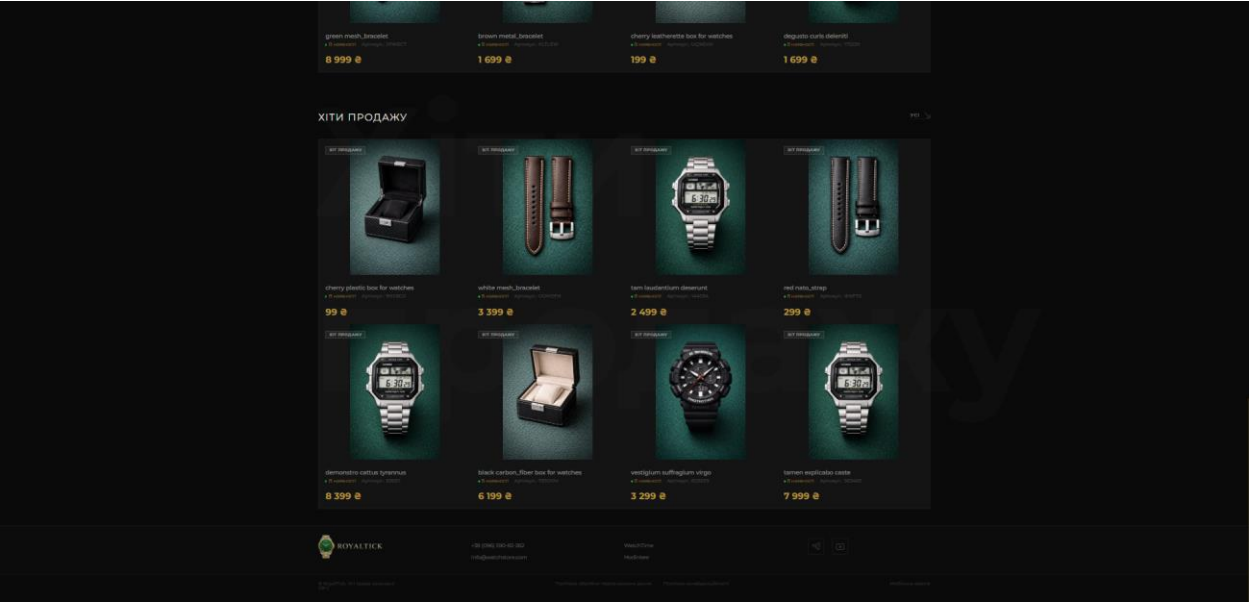


Рис. 3.3. Блок хіти продажу і футер на головній сторінці

Навівшись на картку товару з’являється три іконки додати в обране, додати до порівняння, швидкий перегляд. Також з’являється кнопка «Додати в кошик». По кліку на кнопку чи іконку вона змінюється це видно по іконці додати в обране (рис.3.4). Перехід на сторінку товару відбувається по кліку на саму картку.

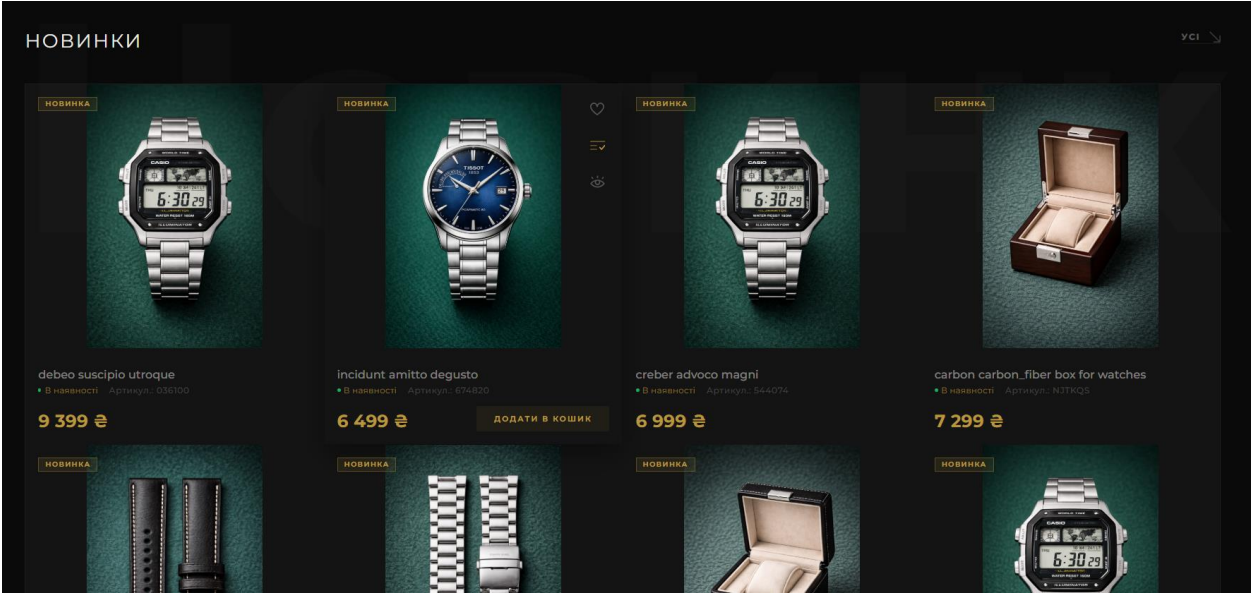


Рис. 3.4. Вигляд катки товару

Користувач спробує додати в кошик товар, який має розмір, то в нього відкриється модальне вікно з таблицею розмірів. Тут він може дізнатися детально інформацію і побачити які розміри є в продажу, вибраний розмір підсвічується і кнопка стає клікабельною. Якщо товар немає розміру то товар додається в кошик і текст кнопки змінюється на додано і її видно без наведення на товар (рис.3.5).

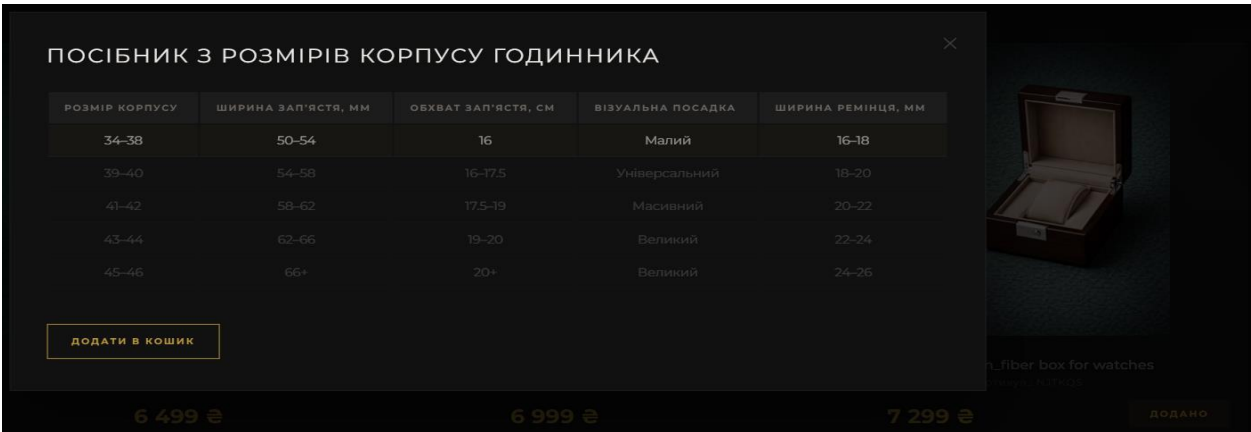


Рис. 3.5. Таблиця розмірів товару і кнопка додати

Натиснувши на кнопку швидкого перегляду на катрці товару можна побачити попап з основною інформацією про товар (рис.3.6). Можна змінити кількість товару, перейти на сторінку товару, погортати слайдер з картинками, якщо товар містить розмір відкрити таблицю розмірів, додати в обране або в порівняння. Користувач може обрати розмір відразу в цьому попапі і на кнопці відобразиться бейдж с цифрою скільки товарів цього розміру знаходиться в кошику.

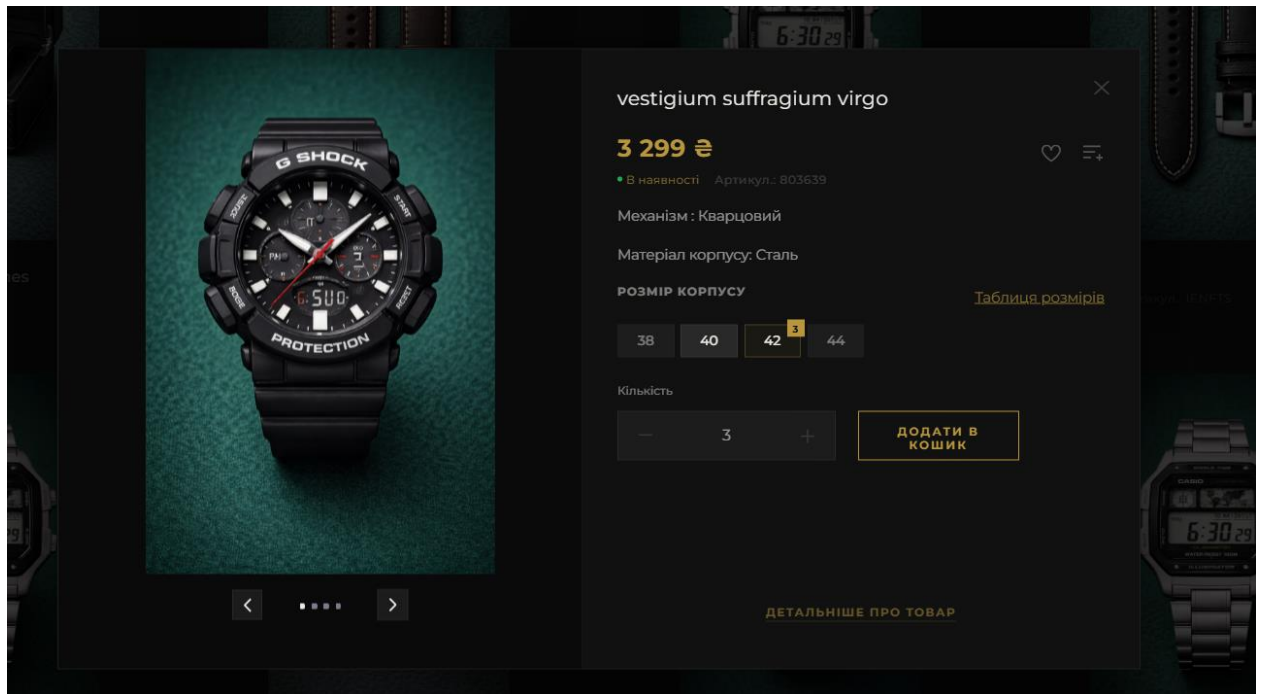


Рис. 3.6. Попап швидкого перегляду товару

Відкривши сторінку товару користувач може побачити фото колаж товару, хлібні крихти, списки з характеристикою, опис, кнопки додати в обране та поділитися. Також пристуній весь функціонал що на швидкому прегляду товару. По хлібніим крихтам можна перейти на загальний каталог або на каталог з годинниками. При натисканні на кнопку «Додати в обране» іконка перефарбовується в червоний колір. Якщо натиснути на іконку поділитися з'явиться попап з можливістю поділитися товаром в watsap. Також видно, що товар відноситься до двох типів, а саме до новинок і хітів продажу (рис.3.7).

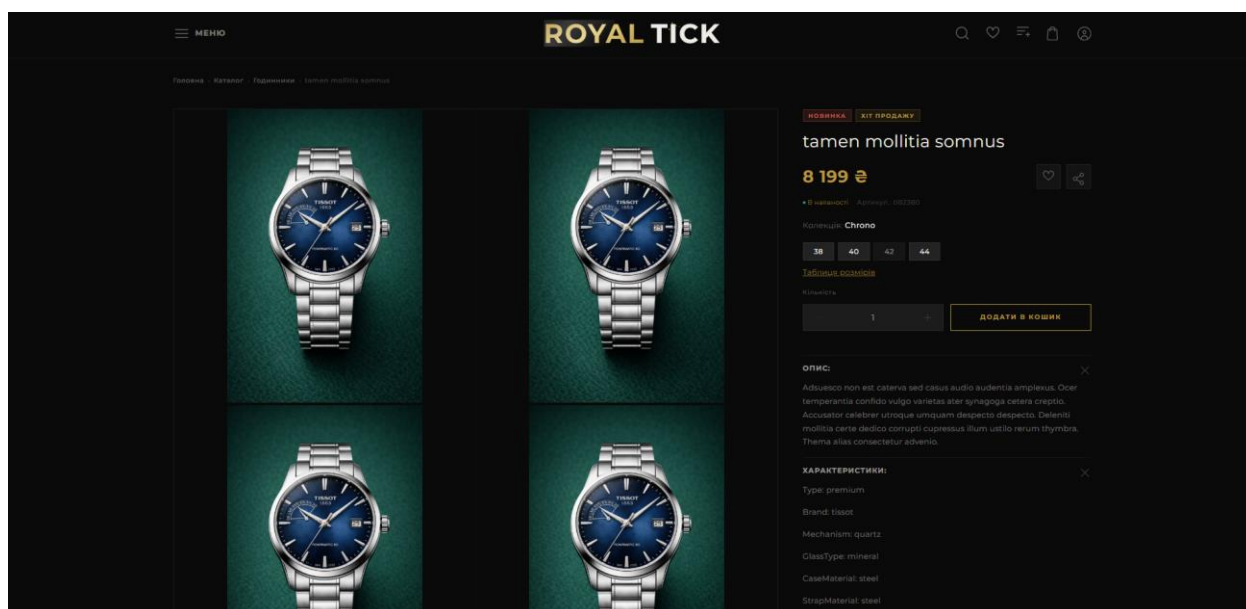


Рис. 3.7. Сторінка товару

Якщо прокрутити сторінку товару нижче користувач зможе побачити товари тієї самої колекції, що і обраний товар (рис.3.8). Також ще нижче список товарів який формується з переглянутих товарів користувачем раніше. Кнопка «Усі» перенаправляє в каталог цієї колекції або переглянутих товарів.

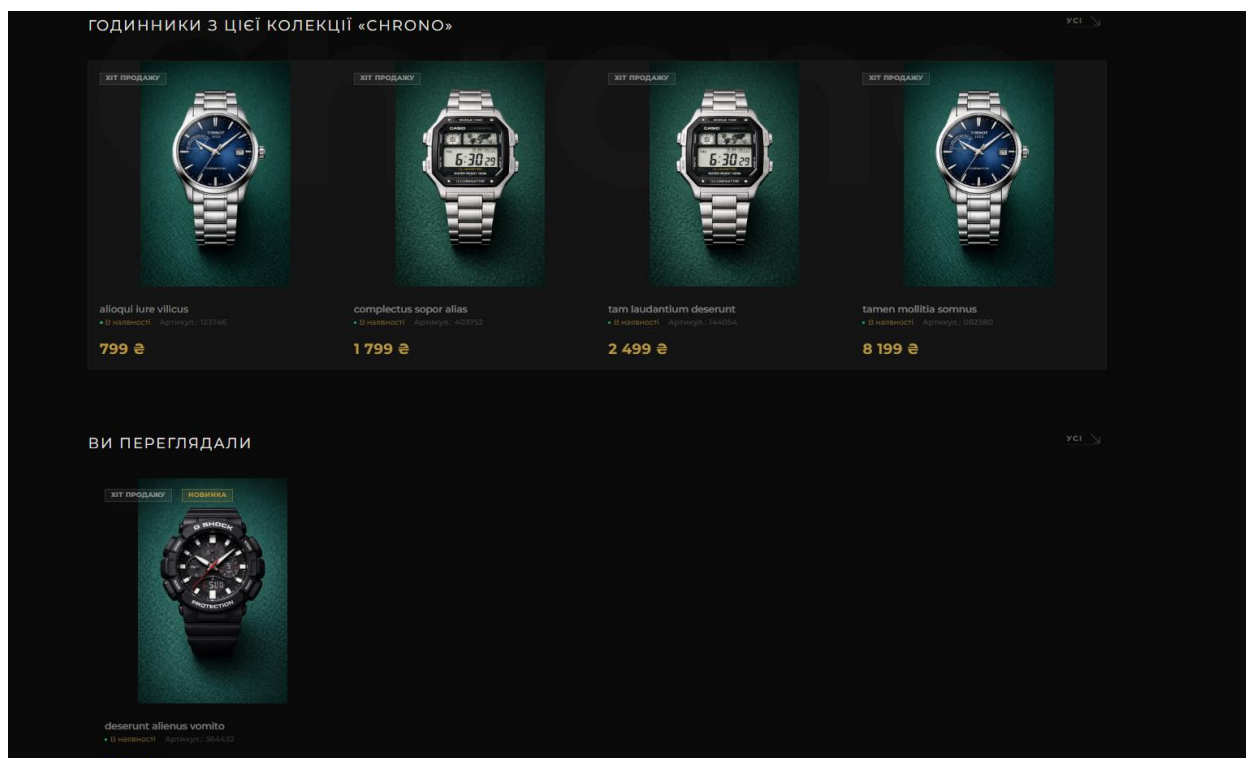


Рис. 3.8. Товари однієї колекції і блок ви переглядали на сторінці товару

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				92
Змн.	Арк.	№ докум.	Підпис	Дата		

Попап поділитися товаром (рис.3.9).

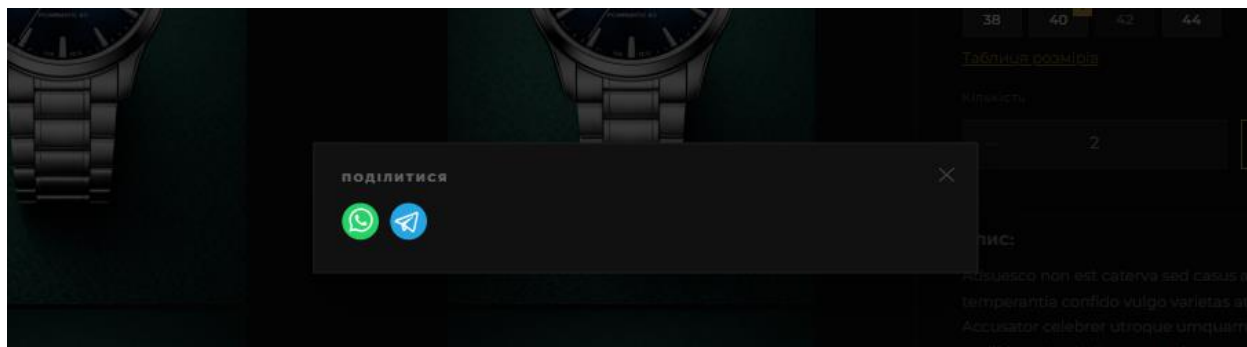


Рис. 3.9. Попап поділитися товаром

Модалка пошуку товару (рис.3.10). На ній можна знайти товар за категорією, типом, назвою, колекцією. Також можна обрати одну з чотирьох категорій і перейти на каталоог.

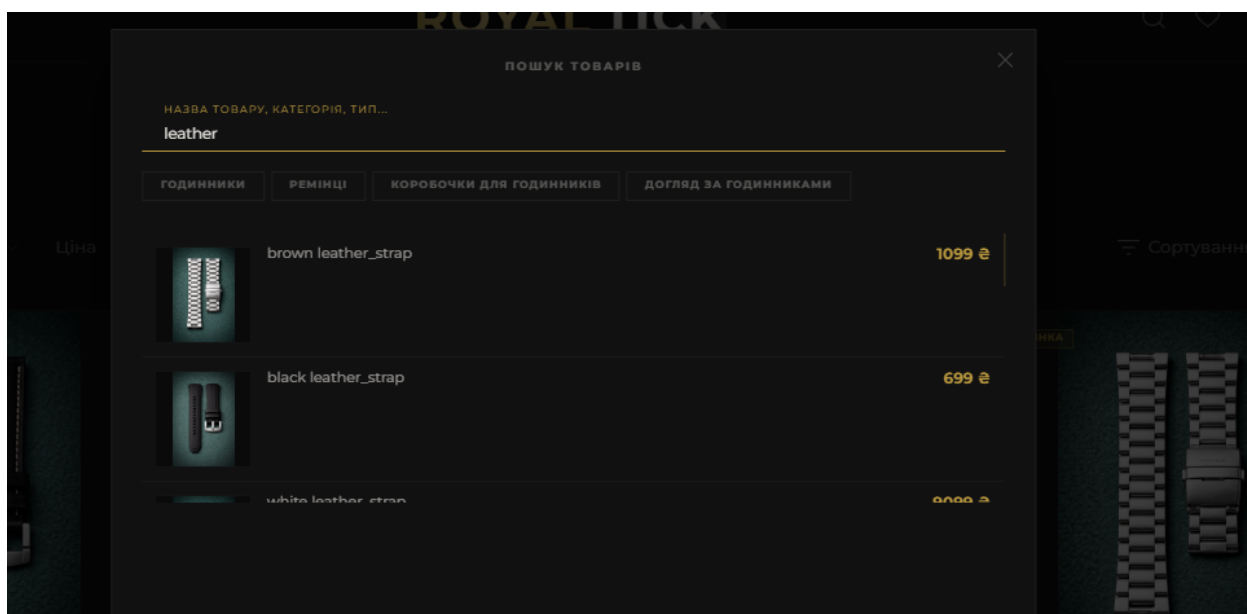


Рис. 3.10. Модалка пошуку товару

Перейшовши на сторінку улюблених товарів користувач бачить самі товари і їх кількість. З цієї сторінки він може перейти на товар, видалити товар, або додати в кошик. Все відбувається асинхронно тому додавши товар в кошик в хедері одразу загориться позначка на іконці кошику, що додали товар. Такий підхід забезпечує плавний та безшовний користувацький досвід, адже інтерфейс миттєво реагує на дії людини без жодних дратуючих перезавантажень сторінки (рис.3.11).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				93
Змн.	Арк.	№ докум.	Підпис	Дата		

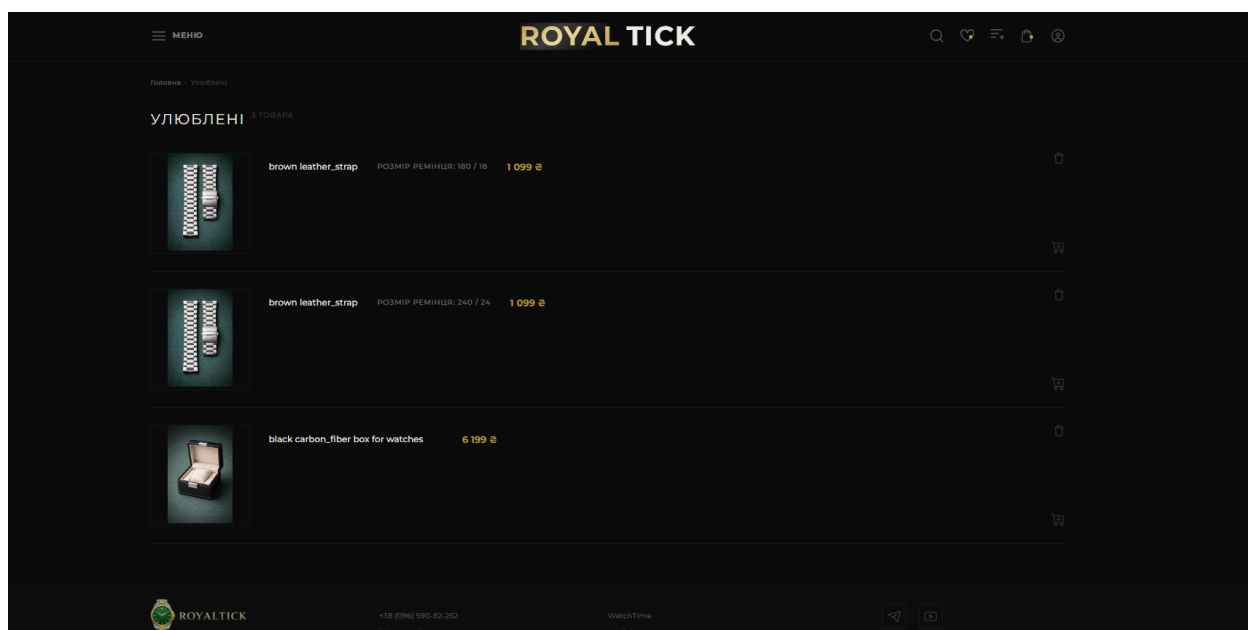


Рис. 3.11. Сторінка улюблених товарів

Якщо сторінка улюблених товарів пуста, то користувач побачить відповідну картинку і повідомлення, що сторінка порожня і має можливість перейти до каталогу або на головну (рис.3.12).

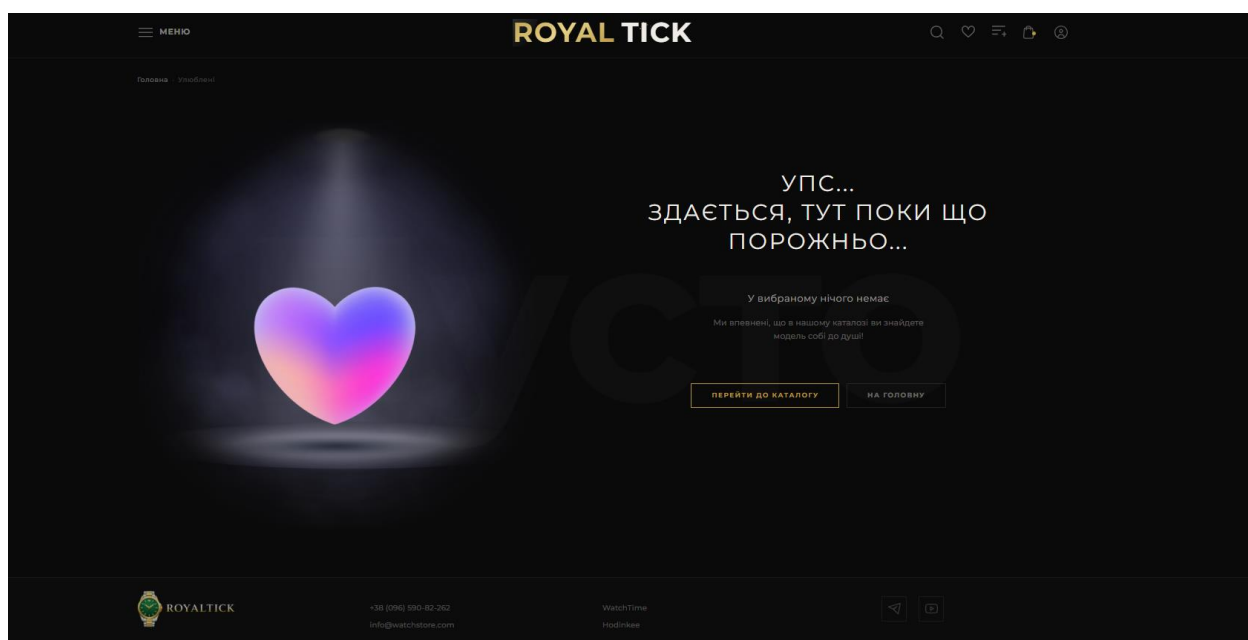


Рис. 3.12. Пуста сторінка улюблених товарів

Перейшовши на сторінку порівняння і маючи там товари користувач бачить категорії товарів, які він додавав. Він може натиснути на категорію і перейти до товарів цієї категорії для порівняння (рис.3.13).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				94
Змн.	Арк.	№ докум.	Підпис	Дата		

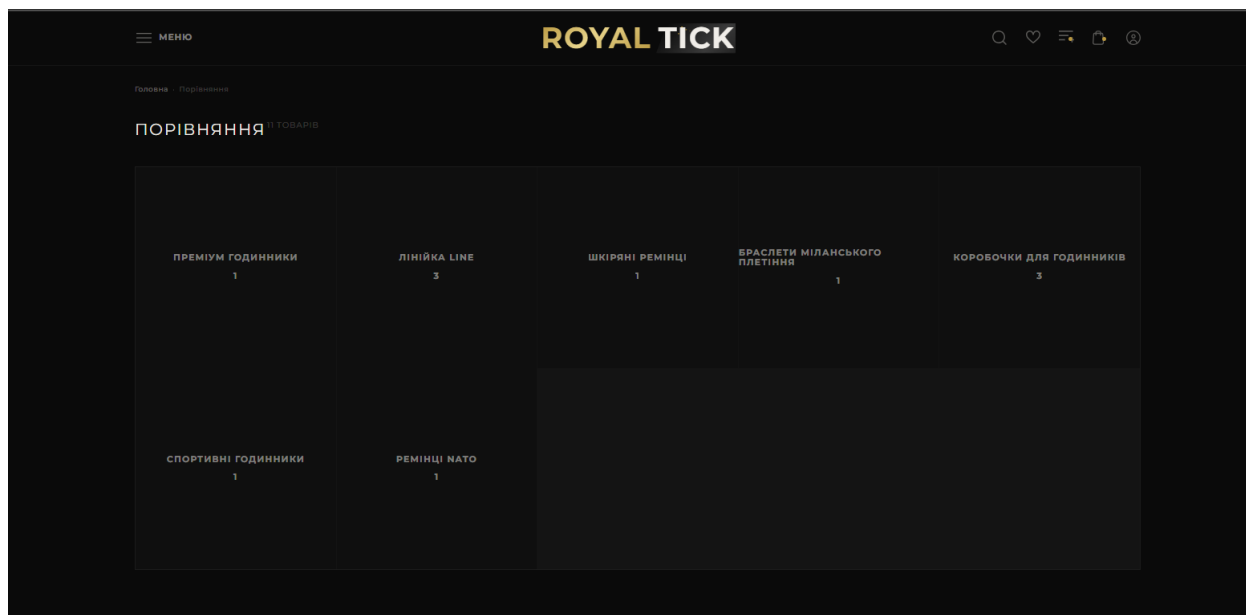


Рис. 3.13. Сторінка порівняння

Обравши категорію для порівняння товарів користувача перенаправляє на іншу сторінку. Тут він бачить товари з всіма характеристиками, може додати товар в кошик або видалити з порівняння. Також має можливість перейти на іншу категорію або скористатися хлібними крихтами для повернення назад (рис.3.14).

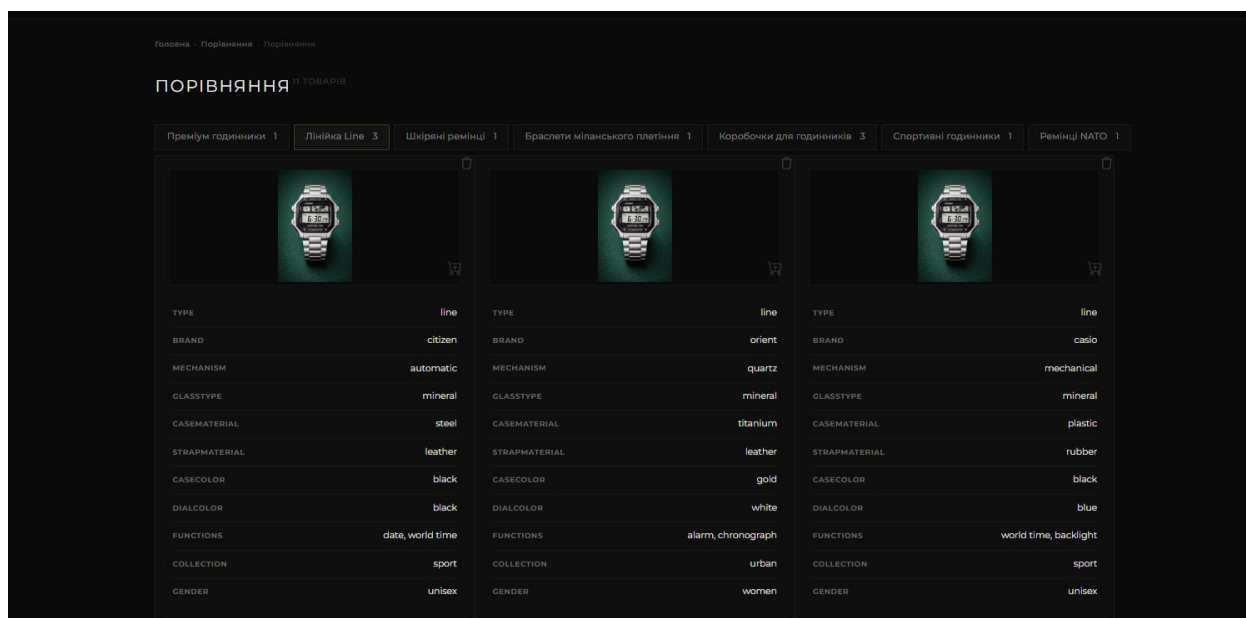


Рис. 3.14. Процес порівняння товарів

Так як і на сторінці улюблених товарів, якщо порівняння не мають ні одного товару, то висвічується повідомлення відповідна картинка і кнопки (рис.3.15).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				95
Змн.	Арк.	№ докум.	Підпис	Дата		

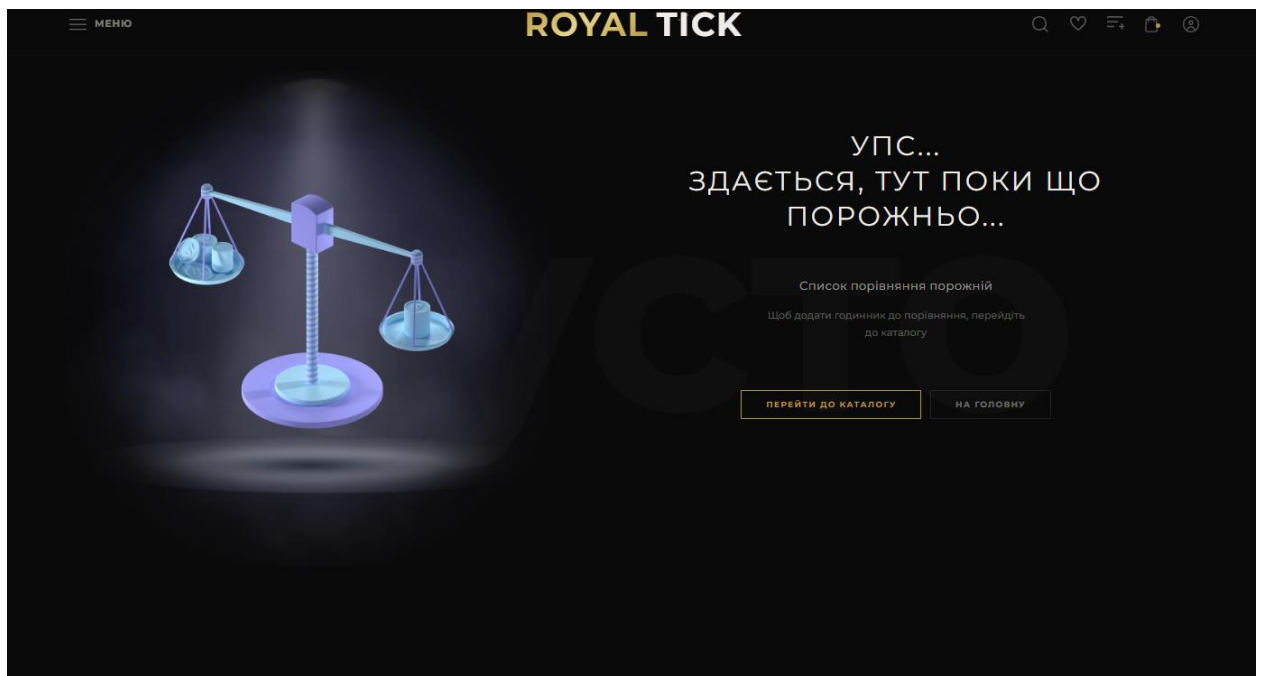


Рис. 3.15. Пуста сторінка порівняння товарів

Коли користувач додає товар в кошик, в хедері з'являється позначення того, що кошик поповнився. Навівшись на іконку у користувача з'являється попап кошик, де він бачить товар назву, сумму, загальну сумму і розмір годинника. Також користувач може збільшити або зменшити кількість і видалити товар (рис.3.16).

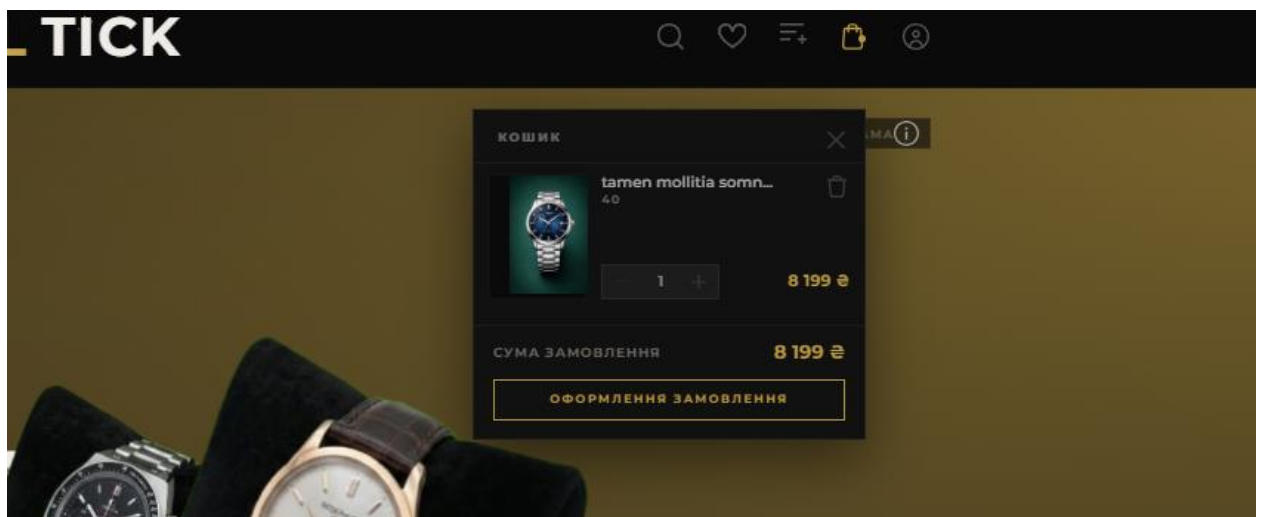


Рис. 3.16. Попап кошика

Натиснувши на іконку людини в хедері у користувача відкривається попап реєстрації, користувач може зареєструватися увівши дані або увійти за допомогою інших сервісів. Якщо аккаунт зареєстровано потрібно натиснути кнопку «Увійти!» (рис.3.17)

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				96
Змн.	Арк.	№ докум.	Підпис	Дата		

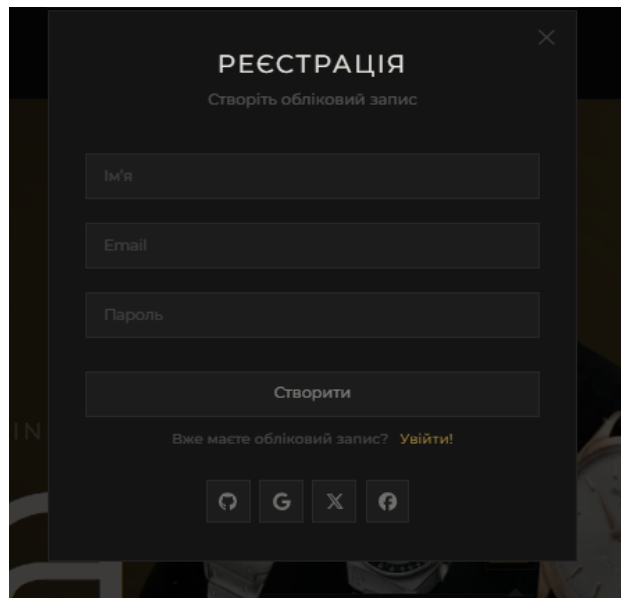


Рис. 3.17. Попап реєстрації

Попап увійти відрізняється тільки кнопкою «Забули пароль?», яка веде на сторінку відновлення паролю (рис.3.18).

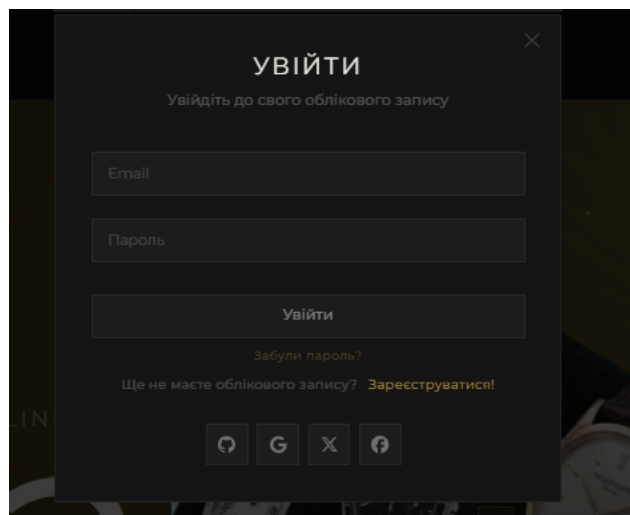


Рис. 3.18. Попап входу в акаунт

Коли користувач натиснув на кнопку «Забули пароль?» його перенаправляє на цю сторінку. Він вводить email на який повинен прийти код підтвердження. Після цього користувача попросять увести новий пароль і повторити його, щоб упевнитись в правильності пароля. Це дозволяє надійно захистити акаунт від несанкціонованого доступу, водночас надаючи користувачеві простий і зрозумілий інструмент для швидкого відновлення контролю над своїм профілем (рис.3.19).

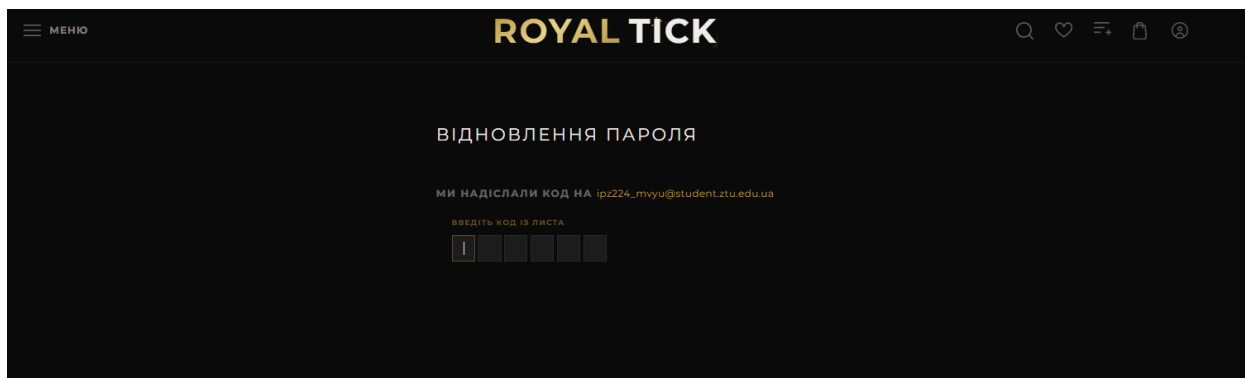


Рис. 3.19. Сторінка відновлення паролю

На email користувачу приходить код для зміни пароля, а також дані для входу в систему при реєстрації (рис.3.20).

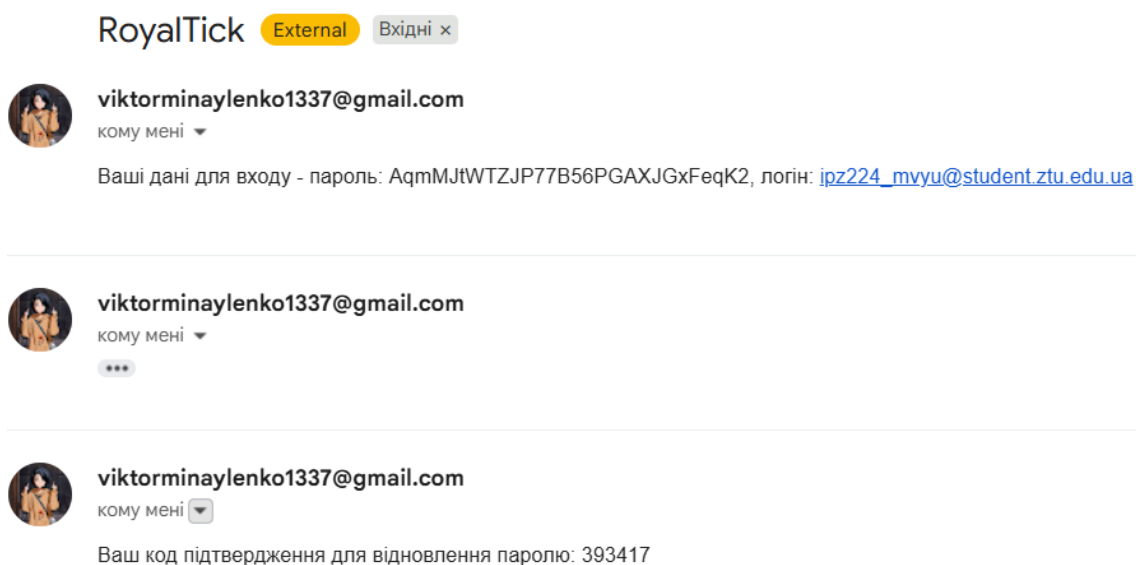


Рис. 3.20. Результат надходження повідомлень на email

Створивши акаунт користувач бачить інформацію про свій аккаунт. Може змінити аватарку, ім'я, email. Користувач має можливість поповнити свій баланс, увівши суму і натиснувши кнопку поповнити. Може верифікувати аккаунт, щоб у майбутньому робити ставки на лоти в розділі аукціону. В центрі бачить інформацію, кількість підписників, лотів, рейтинг, баналанс. А також місце для лотів. З самого низу знаходяться відгуки, які йому можуть залишити інші користувачі після покупки чи продажу лоту (рис.3.21).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				98
Змн.	Арк.	№ докум.	Підпис	Дата		

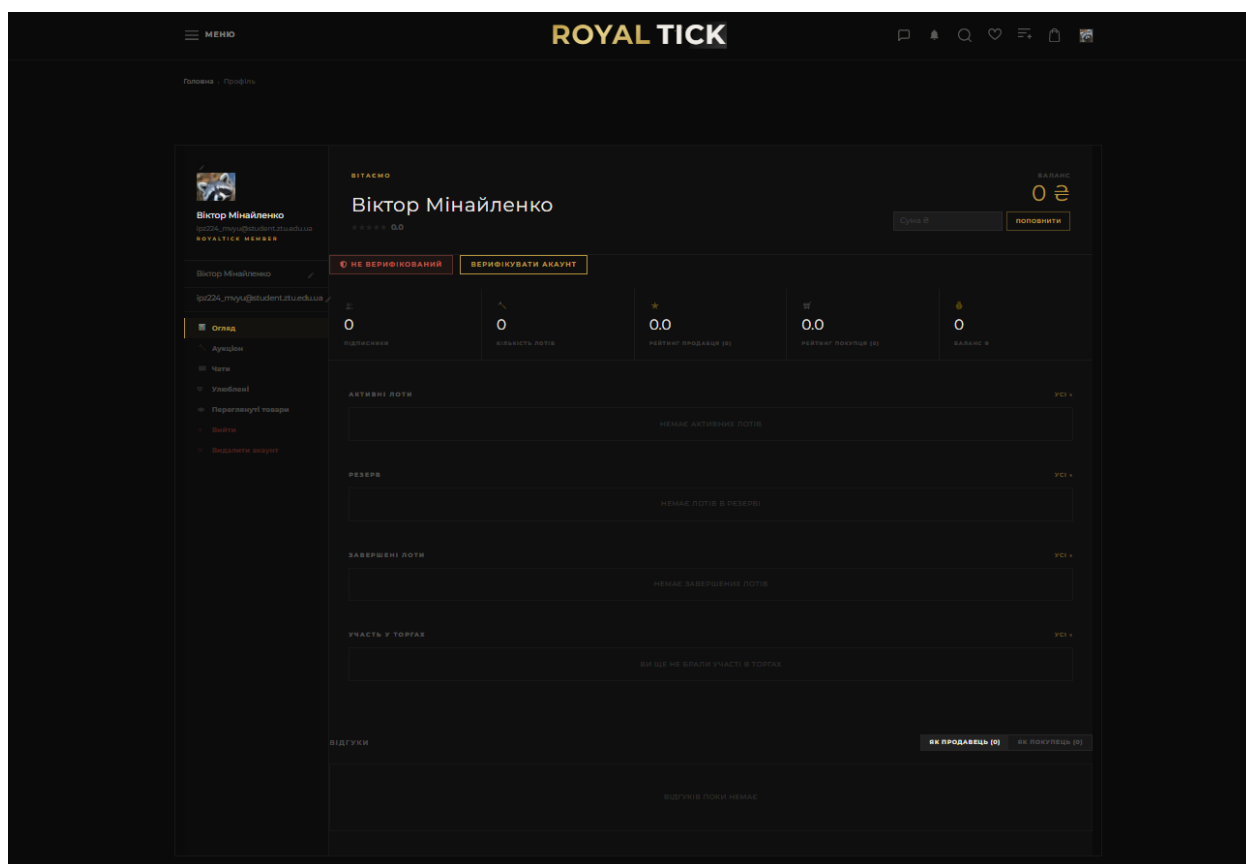


Рис. 3.21. Сторінка власного аккаунту

Натиснувши кнопку верифікувати аккаунт, у нас з'являється модалька де ми вводим реальний номер карти та номер телефону для верифікації (рис.3.22). Після цього статус аккаунту зміниться на верифікований.

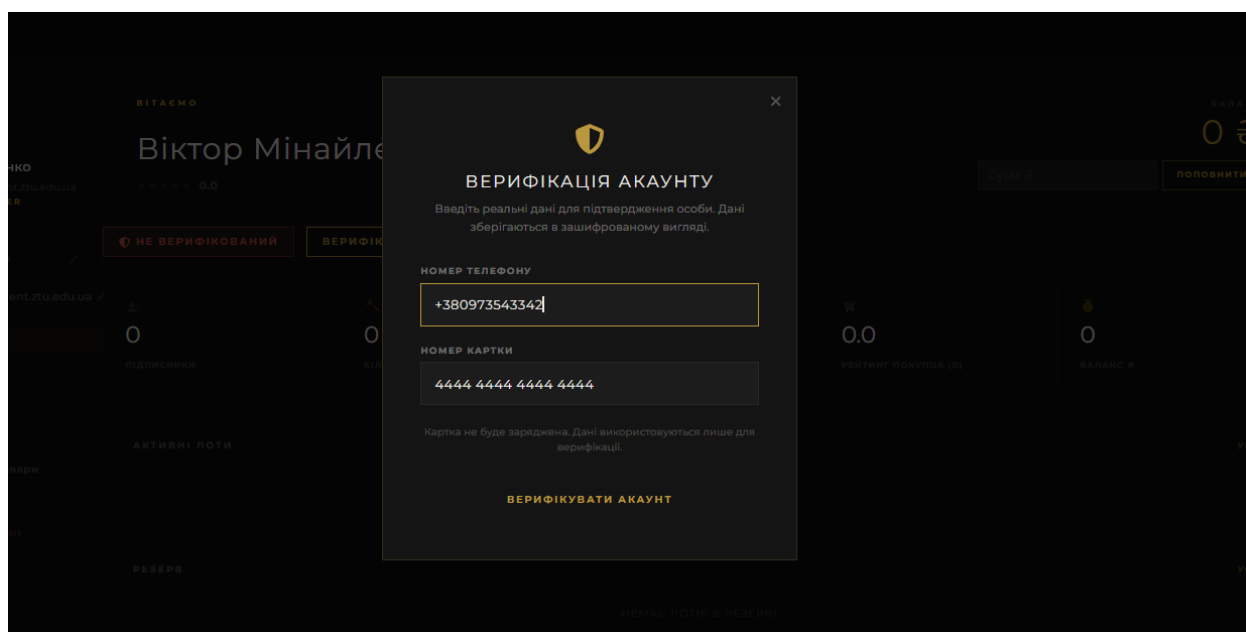


Рис. 3.22. Модалька для верифікації аккаунту

		Мінайлєнко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				99
Змн.	Арк.	№ докум.	Підпис	Дата		

Сторінка на якій користувач може ознайомитися та прочитати політику конфіденційності та обробки персональних даних (рис.3.23).

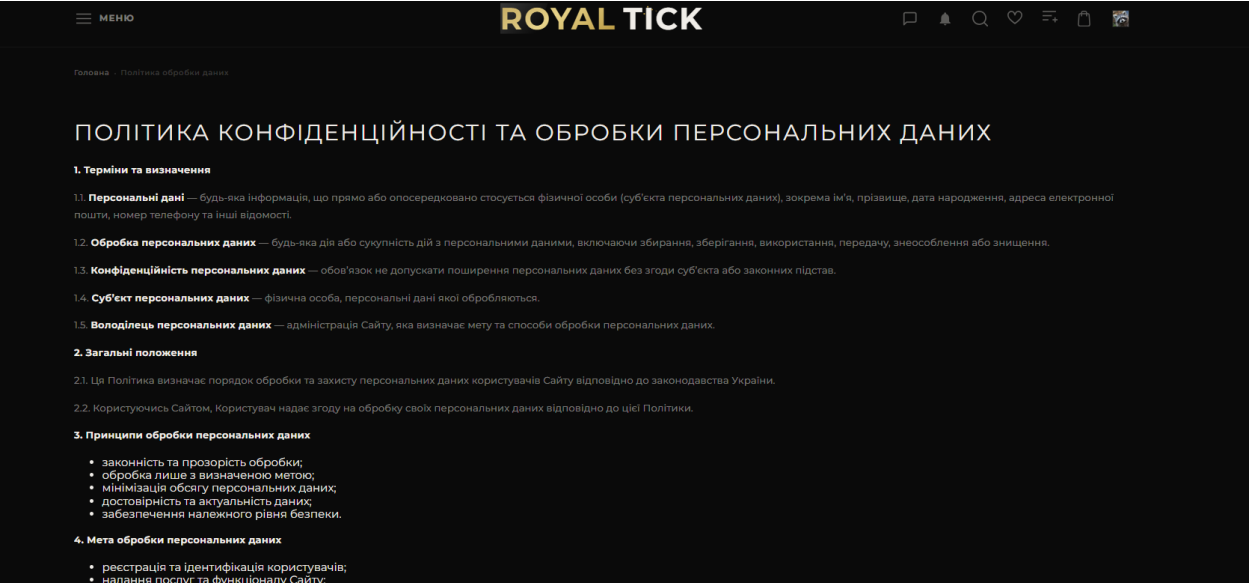


Рис. 3.23. Сторінка політики конфіденційності

Меню платформи RoyalTick через яке користувач може повернутися на головну, змінити мову на сайти (UA / EN) або піти далі по доступному контенту. Також видно випадаючий список з каталогу, де видно 4 категорії товарів доступні на сайті (рис.3.24).

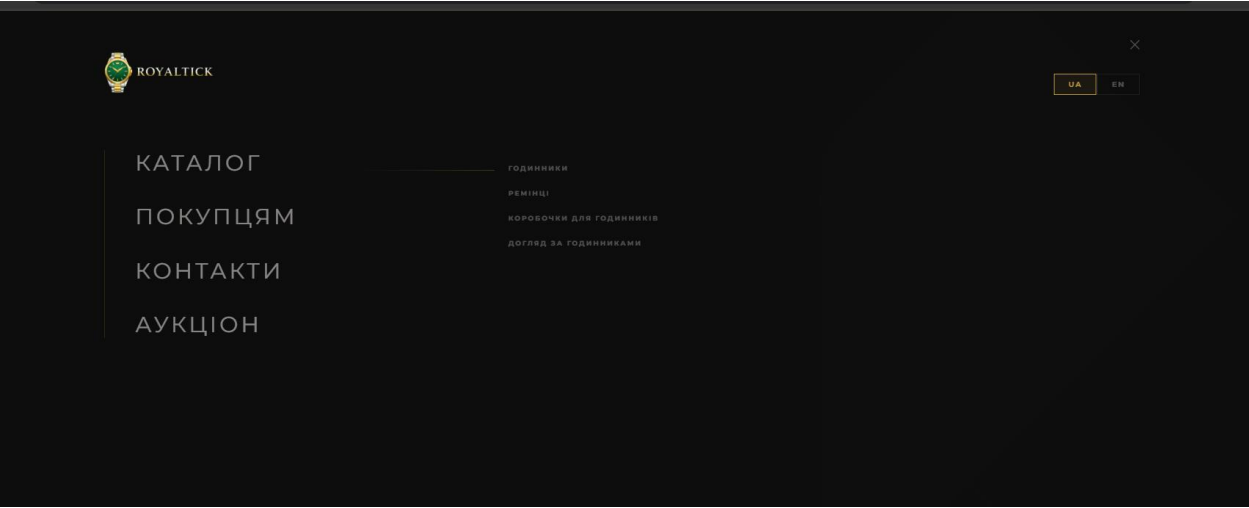


Рис. 3.24. Головне меню платформи RoyalTick

Перейшовши до каталогу користувач бачить декілька фільтрів залежно від того на якому типу товарів він знаходиться. На (рис.3.25) видно фільтрацію за категорією, ціною, розміром корпусу, кольором та сортування. Також користувач може побачити товари за фільтрами та рекламні товари. Також

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				100
Змн.	Арк.	№ докум.	Підпис	Дата		

видно скільки товарів знаходиться на цій категорії (12 твоарів). За допомогою хлібних крихт користувач може повернутися на каталог або головну сторінку.

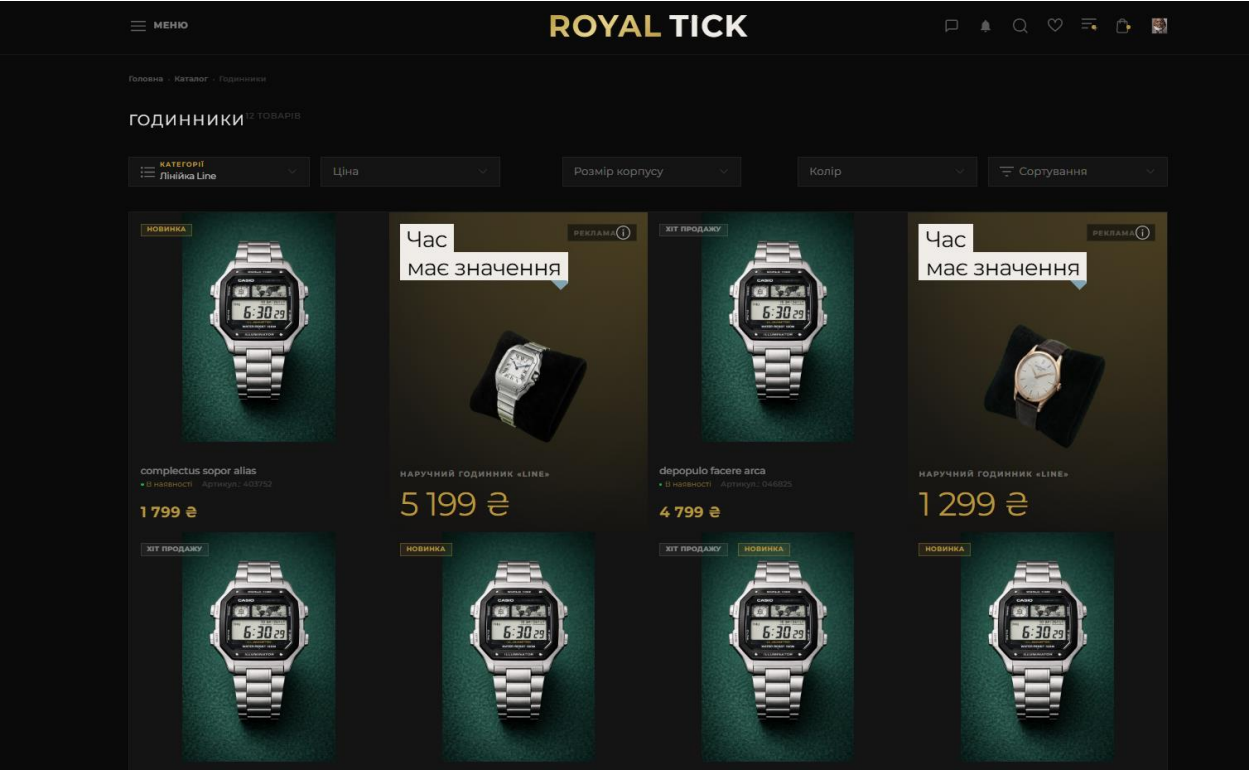


Рис. 3.25. Сторінка каталогу товарів

Прокрутивши сторінку нижче користувач побачить пагінацію та відомий блок «Ви переглядали» (рис.3.26).

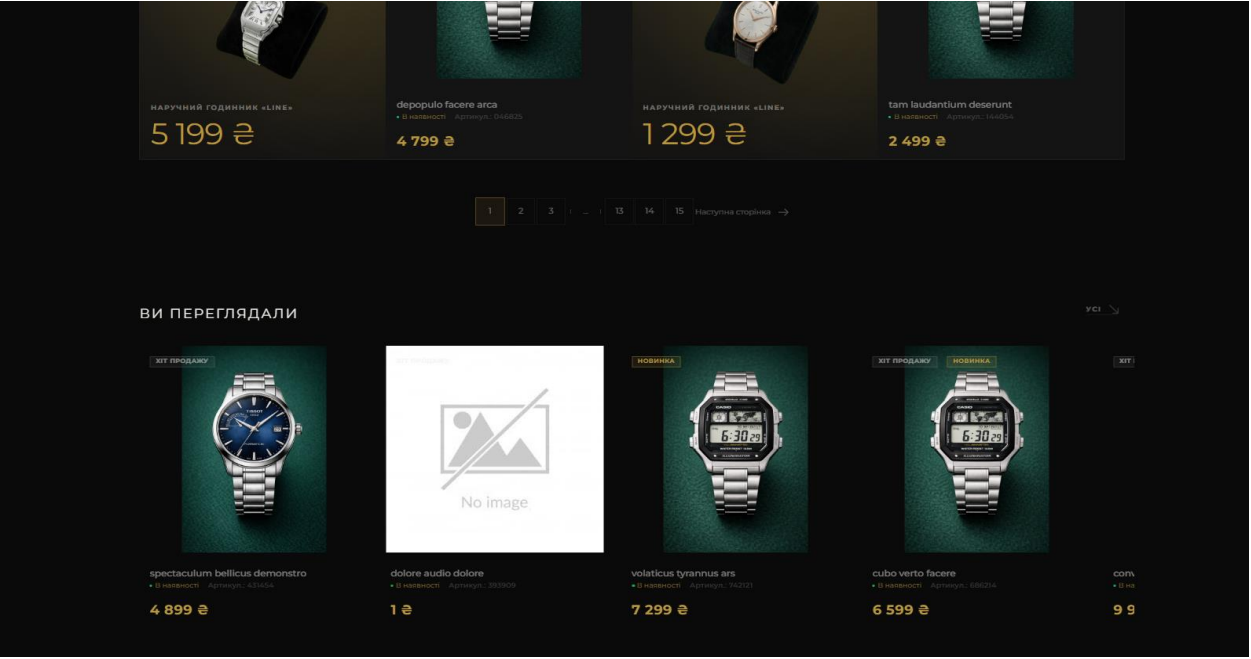


Рис. 3.26. Пагінація на сторінці каталогу

Коли користувач визначився з товаром і перейшов на сторінку кошику він бачить обрані товари. Користувач має можливість змінювати кількість товару або видалити його з кошику, також можна ввести промокод для отримання знижки. Після того як користувач дав згоду на обробку персональних даних може натиснути на “оформити замовлення” (рис.3.27).

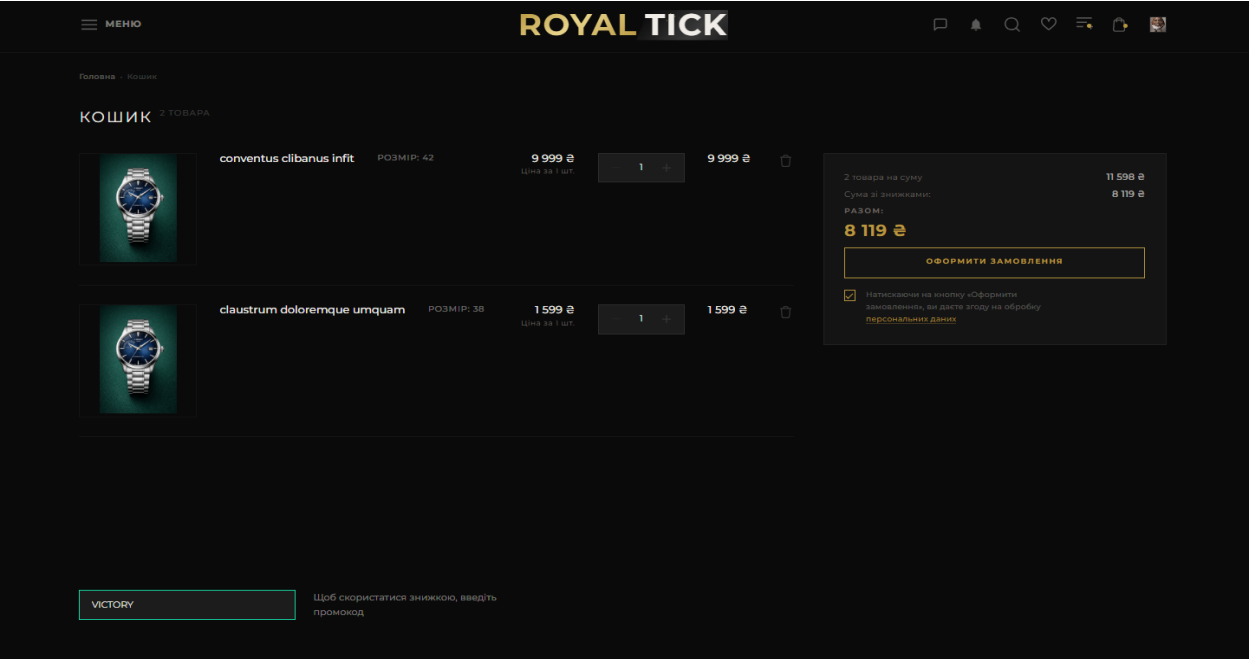


Рис. 3.27. Сторінка кошику

Після натискання кнопки «Оформити замовлення» можна побачити 4 етапи оформлення (рис.3.28). На першому користувач може переглянути свої обрані товари, також праворуч знаходиться блок з додатковою інформацією та кнопкою оформити замовлення.

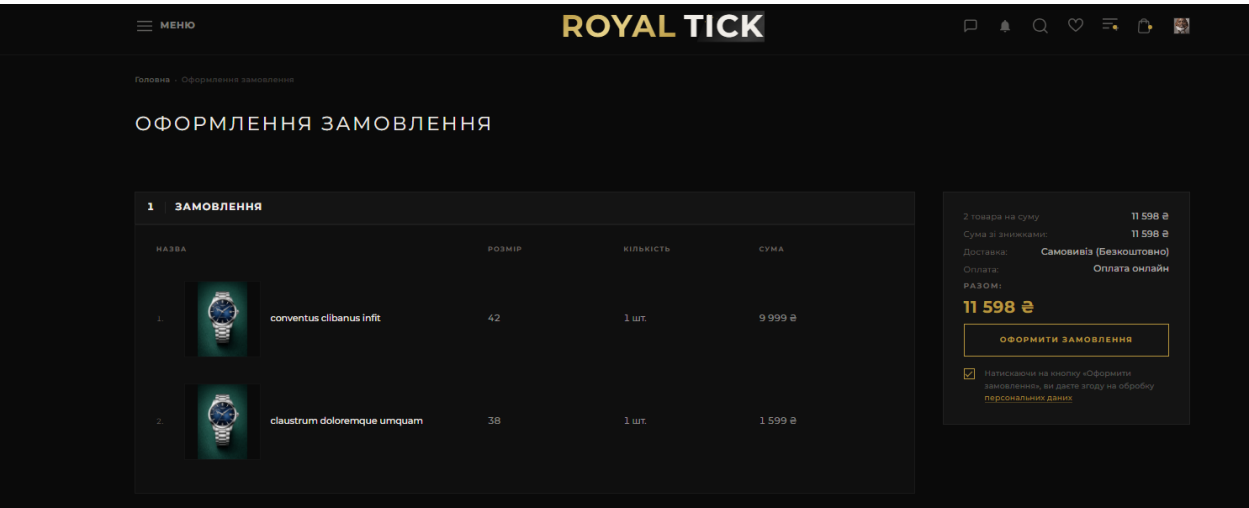


Рис. 3.28. Сторінка оформлення замовлення

Другий етап це доставка, тут користувач може обрати 3 види доставки спмовивіз, кур’єрська доставка та нова пошта (рис.3.29). Третій етап це оплата, можна сплатити онлайн або при отриманні. Четвертий етап це дані отримувача, вони є обов’язковими бо надають детальні інформацію про отримувача. На доставці ми можемо вписати місто і нам видасть список віділень, або перенаправить на карту, якщо ми обрали кур’єра.

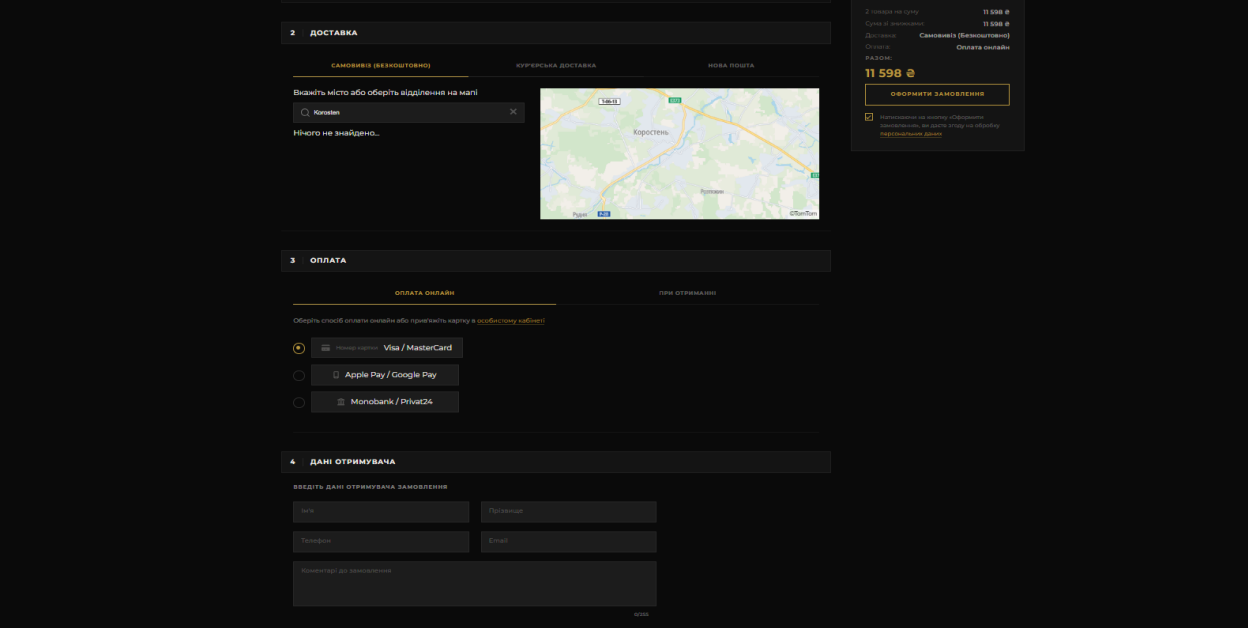


Рис. 3.29. Сторінка оформлення замовлення та інтерактивна карта

На інтерактивні карті можна обрати доставку кур’єром і вказати місце куди потрібно доставити товар (рис.3.30). Обране місце позначається і в інформацію виводять назву вулиці довготу і широту.

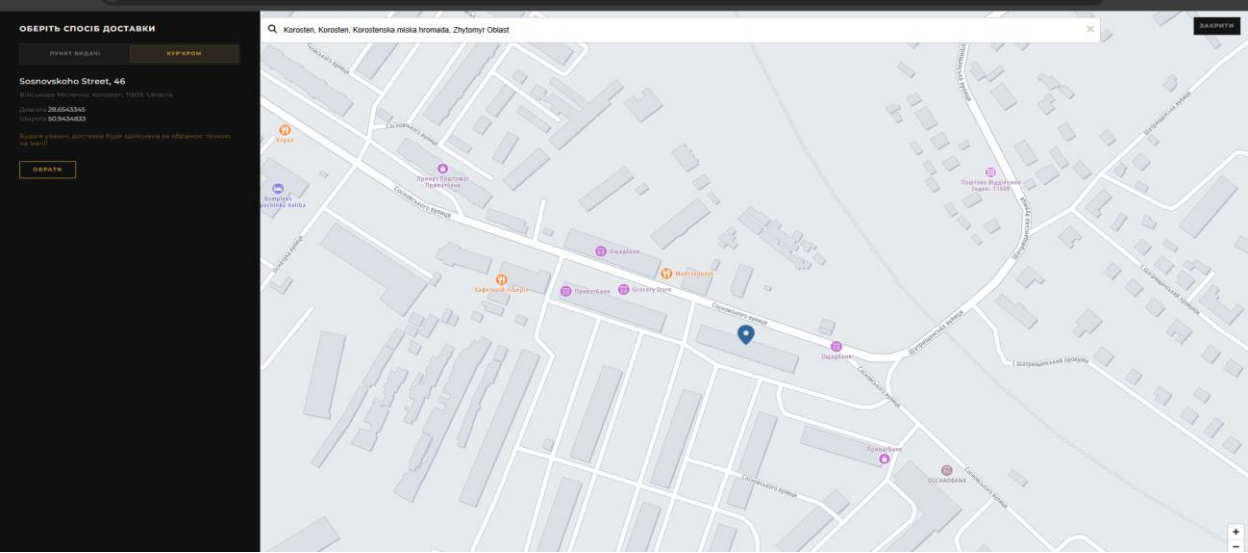


Рис. 3.30. Інтерактивна карта доставки

В головному меню перейшовши розділ аукціон можна потрапити на сторінку з правилами. Користувач може ознайомитися як працює аукціон, прочитати посібник продавця, посібник покупця, переглянути інформацію про чати та угоди, систему рейтингів, поради для надійного аккаунту, а також ознайомитися з правилами та покараннями на сайті(рис.3.31).

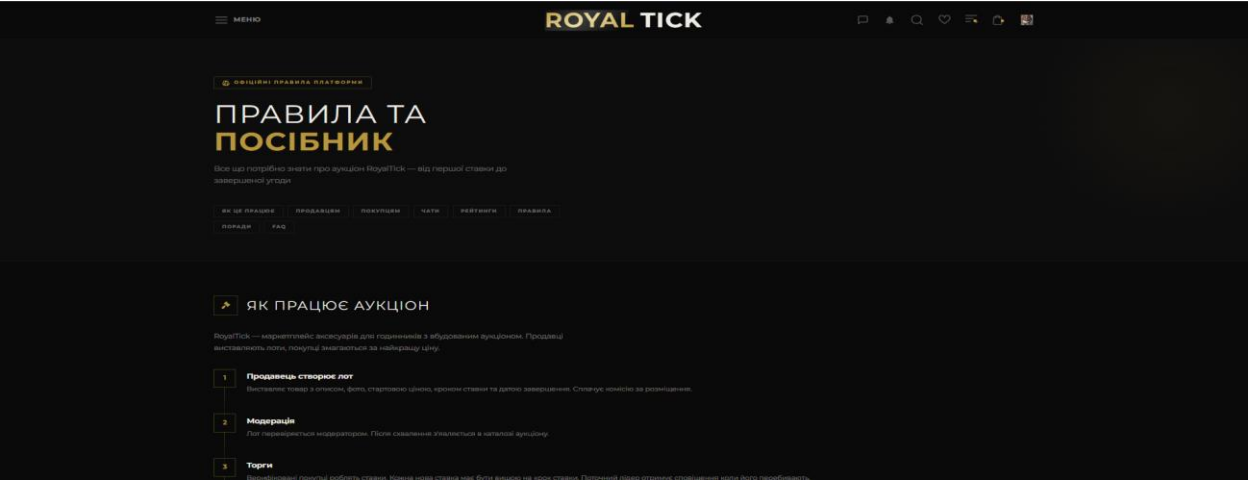


Рис. 3.31. Сторінка з правилами сайту

Йдуть далі по блоку аукціону користувач може натрапити на модуль спільноти. Тут можна створити нову тему а також переглянути існуючі теми, які створили інші користувачі. Теми можуть бути на різні тематики, починаючи від оцінювання закінчуючи обговоренням. Теми можна фільтрувати. Під заголовками видно кількість переглядів, лайків та коментарі до того чи іншого обговорення(рис.3.32).

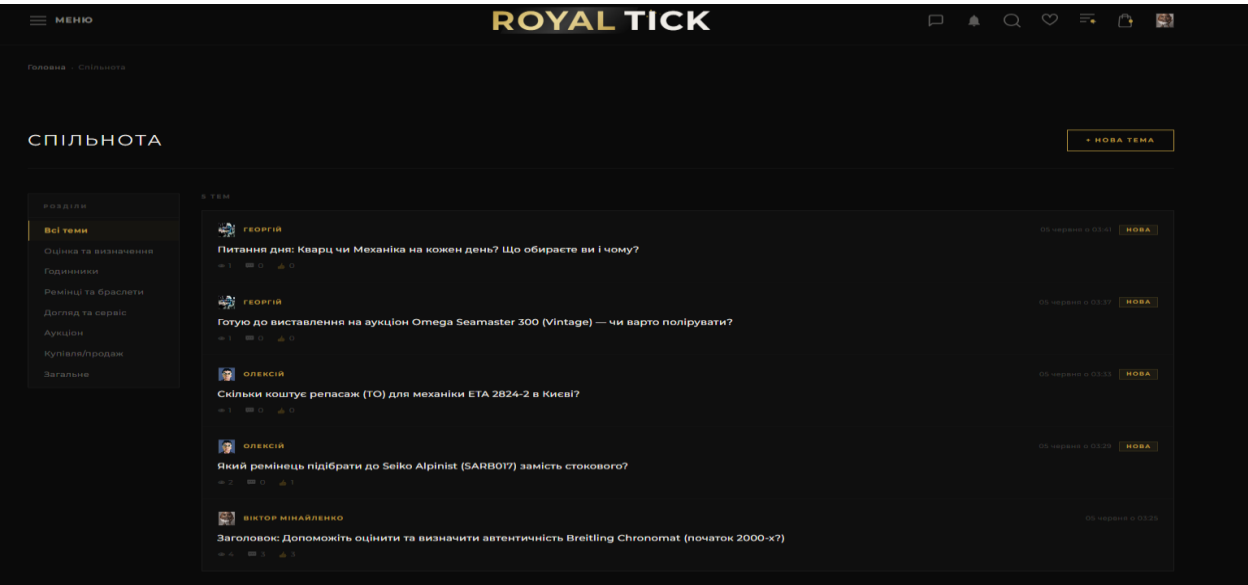


Рис. 3.32. Сторінка з спільноти з темами

Натиснувши на кнопку нова тема, користувач може створити своє обговорення. Юзер повинен написати заголовок , обрати категорію, написати опис, а також може вставити до 4 фото і може додати свої теги (рис.3.33). Проворуч знаходяться рекомендації щодо оформлення теми. Після цих дій кнопка опублікувати стане клікабельною.

Рис. 3.33. Створення обговорення

Можна переглянути вже створену тему. На цій сторінці видно заголовок, опис а також декілька фотографій. Окрім цього, інтерфейс відображає ім'я автора публікації, точну дату створення, кількість переглядів та спеціальну мітку, як-от «Оцінка та визначення». Нижче під основним текстом виводиться лічильник лайків та стрічка відповідей, де інші користувачі можуть залишати свої розгорнуті коментарі з порадами. Також можна ставити лайки, писати коментарі або відповідати на коментарі інших. На цьому і будується розділ спільноти. Завдяки такій структурі обговорень колекціонери або зацікавлені користувачі можуть легко вести тематичні діалоги, ділитися досвідом та допомагати один одному з оцінкою аксесуарів. Усі зміни та нові репліки відображаються в реальному часі, що підтримує високу активність всередині платформи. Це дає можливість проводити час на сайті (рис.3.34).

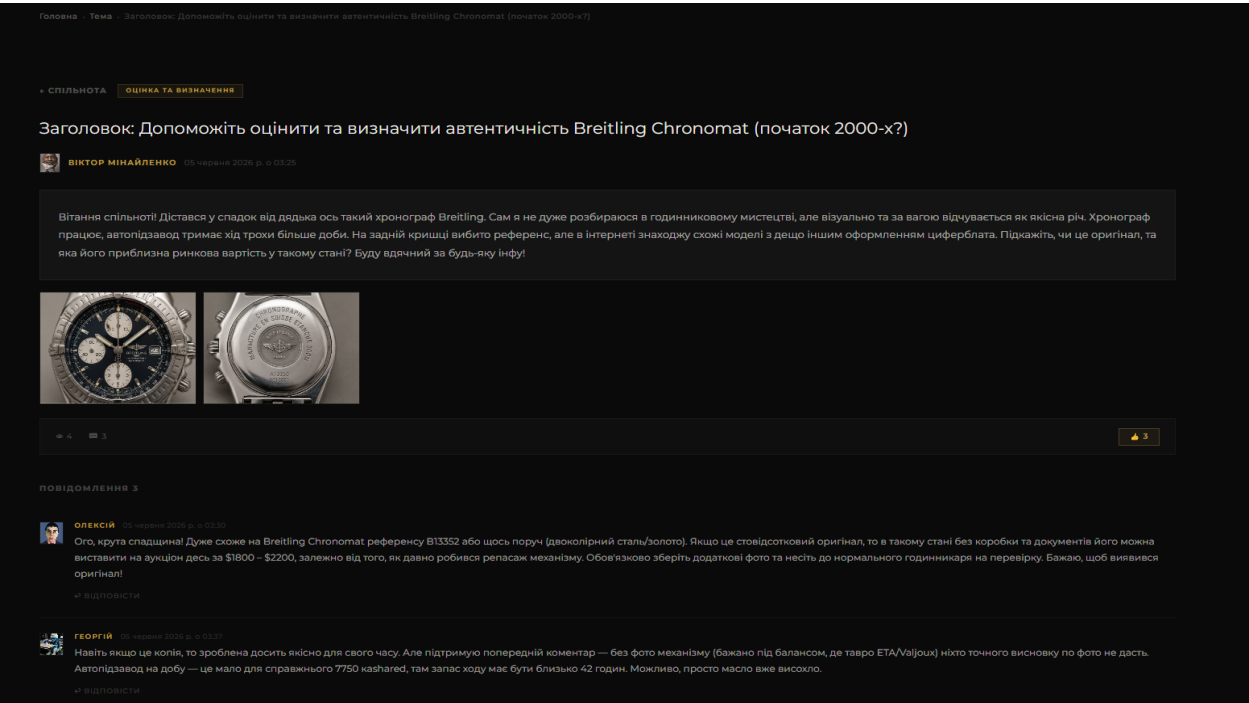


Рис. 3.34. Вигляд готової теми для обговорення

Перейшовши на головний модуль, а саме аукціон користувач може побачити активні лоти (рис.3.35). Також є практичні фільтри які допоможуть орієнтуватися серед потрібних лотів. Кнопка «Створити лот» перенаправить нас на форму створення лоту. На картці лоту написано заголовок а також поточну ціну, при наведенні на картку стає видним коли лот закінчується.

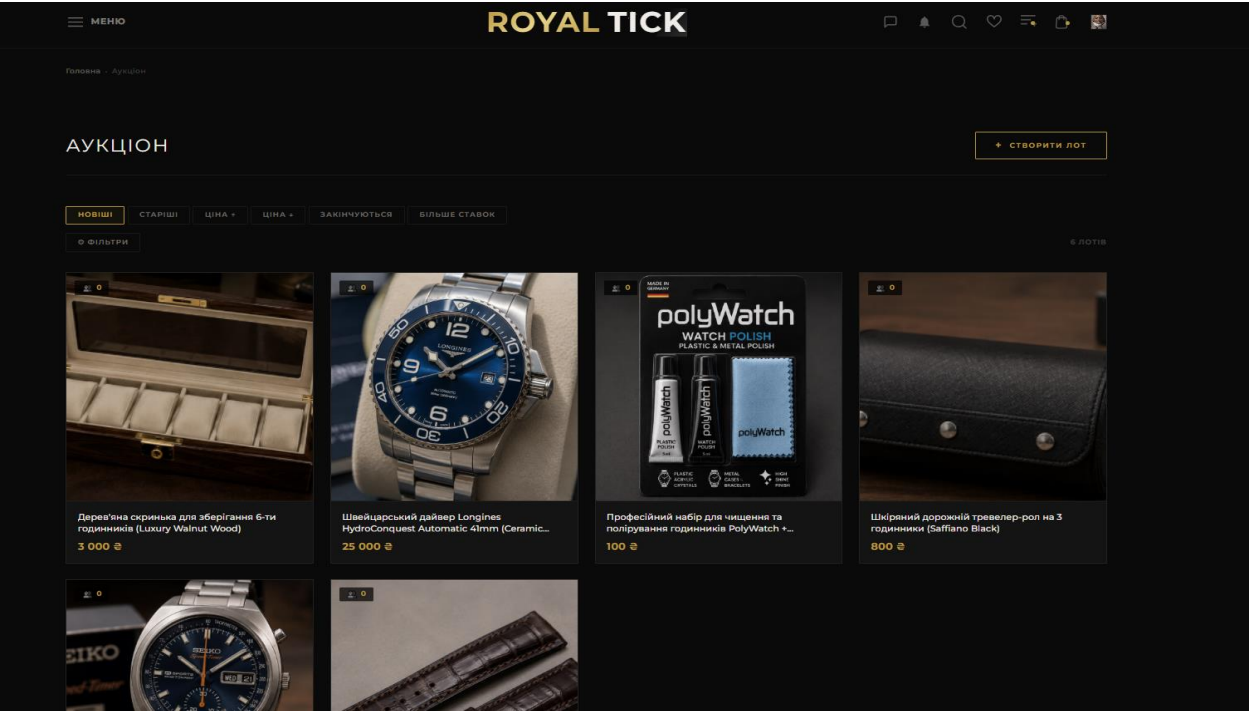


Рис. 3.35. Сторінка з активними лотами

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Євдемюв Ю.М.				106
Змн.	Арк.	№ докум.	Підпис	Дата		

Перейшовши на конкретний лот, користувач може побачити фото лоту, назву, кількість ставок, максимальна ставка, дата додавання і продаця на якого можна клікнути (рис.3.36). Справа знаходиться блок з таймером зворотнього відліку, який говорить про закінчення аукціону, поточна ціна, поле введення для своєї ставки з кроком який вказано і кнопка «Зробити ставку». Також є кнопка «Купити зараз» за фіксованою ціною без участі у торгах, якщо таку можливість додав власник.

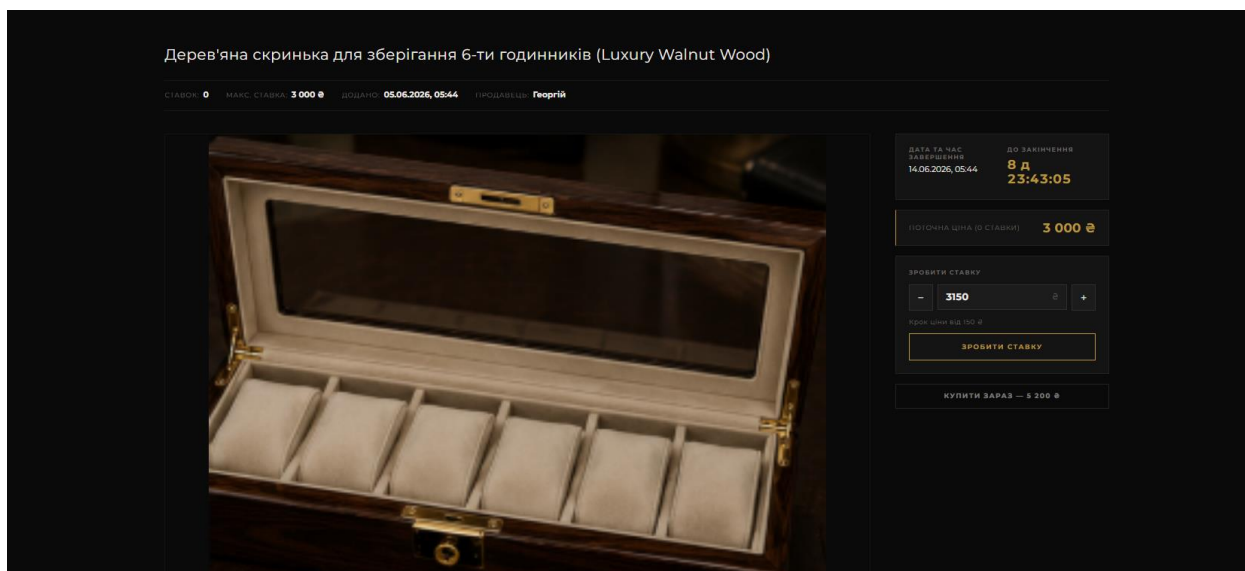


Рис. 3.36. Сторінка конкретного лоту

Прогорнувши нижче користувач може побачити вкладку опис і фотографії які додав продавець. Вкладка «Опис» містить структуровану таблицю з ключовими параметрами товарної позиції, де вказано її стан, місцезнаходження лота, доступні способи доставки, інформацію про оплату транспортних послуг та можливість повернення. Крім того, тут відображаються окремі текстові блоки з гарантійними зобов'язаннями продавця та його особистими коментарями для потенційних покупців. Під вкладками написаний детальний опис цього лоту. Ще нижче блок з лотами цього продавця. Наявність такого блоку інших пропозицій дозволяє відвідувачу сайту швидко ознайомитися з усім асортиментом конкретного колекціонера та, за бажанням, придбати кілька супутніх аксесуарів у межах однієї угоди (рис.3.37).

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		107

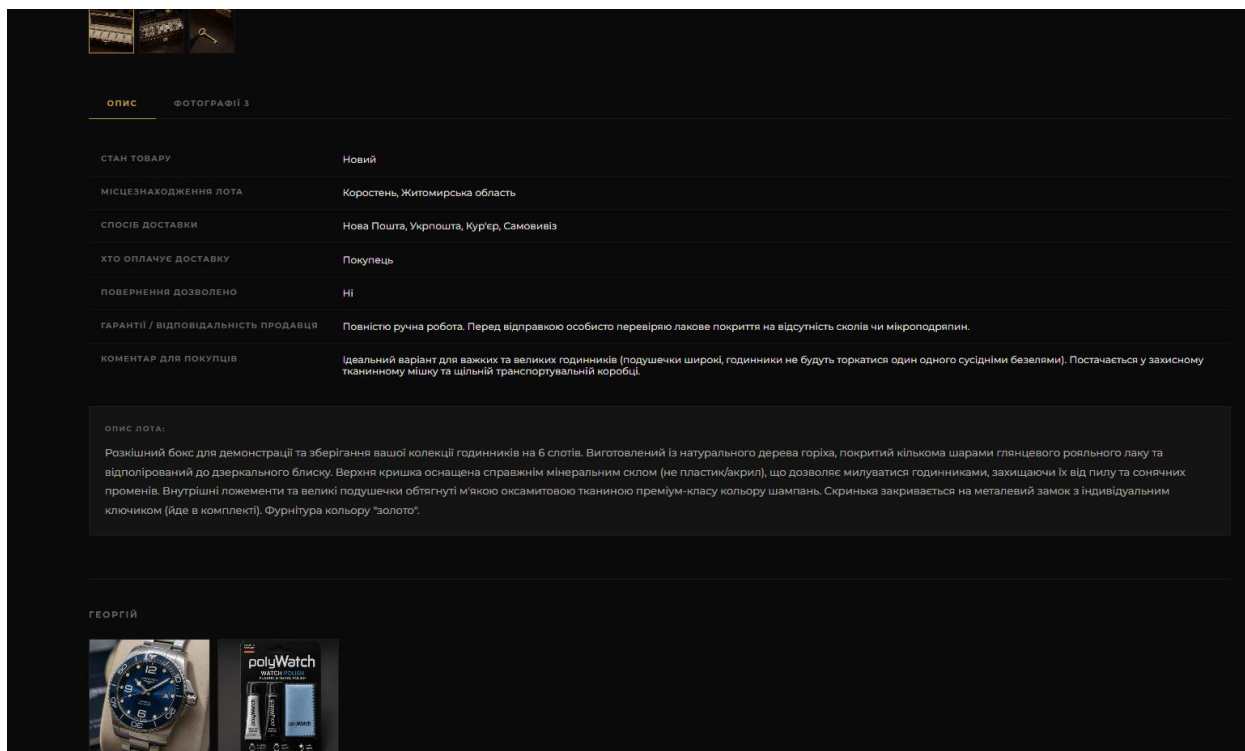


Рис. 3.37. Сторінка лоту з описом і блоком лотів цього продавця

Користувач бачить секцію з коментарями під лотом. Є поле для написання коментаря і кнопка «Надіслати». Нижче відображаються коментарі з ім'ям автора, датою і текстом. Автор лоту позначений окремим бейджем «Продавець» (рис.3.38).

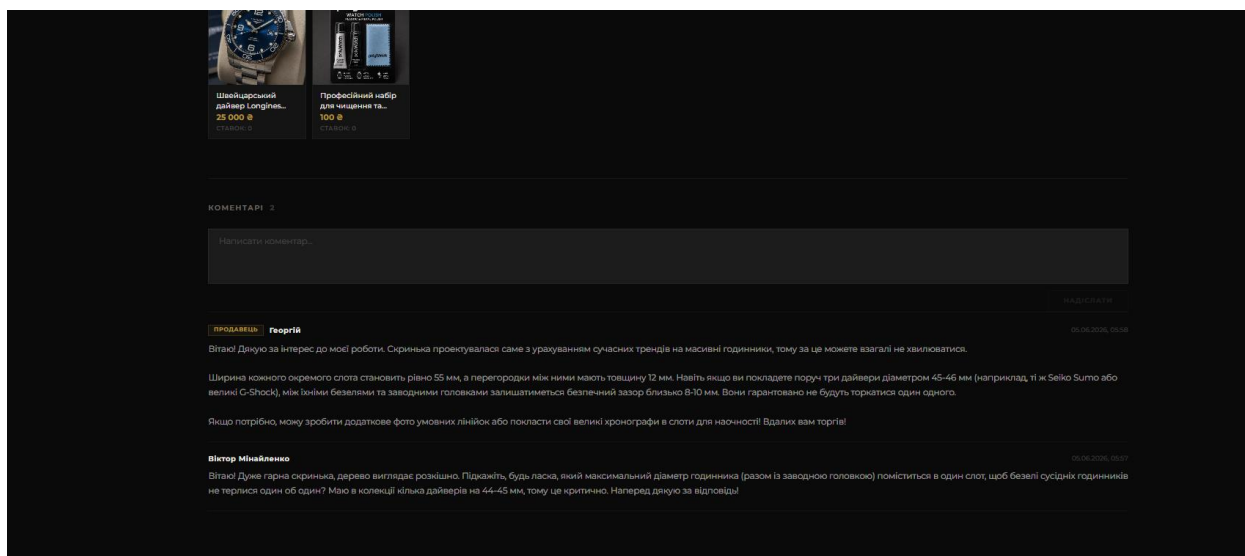


Рис. 3.38. Сторінка лоту з блоком коментарів

Перейшовши на сторінку до іншого користувача можна побачити інформацію про нього. Відображається кількість активних лотів і самі лоти, к-сть підписників та підписок, рейтинг продавця та рейтинг покупця. Також

		Минайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				108
Змн.	Арк.	№ докум.	Підпис	Дата		

можна підписатися або відписатися від нього. Також видно статус користувача чи верифікований чи ні. Це зроблено для того щоб проаналізувати користувача і зрозуміти чи можна з ним продовжувати розмову (рис.3.39).

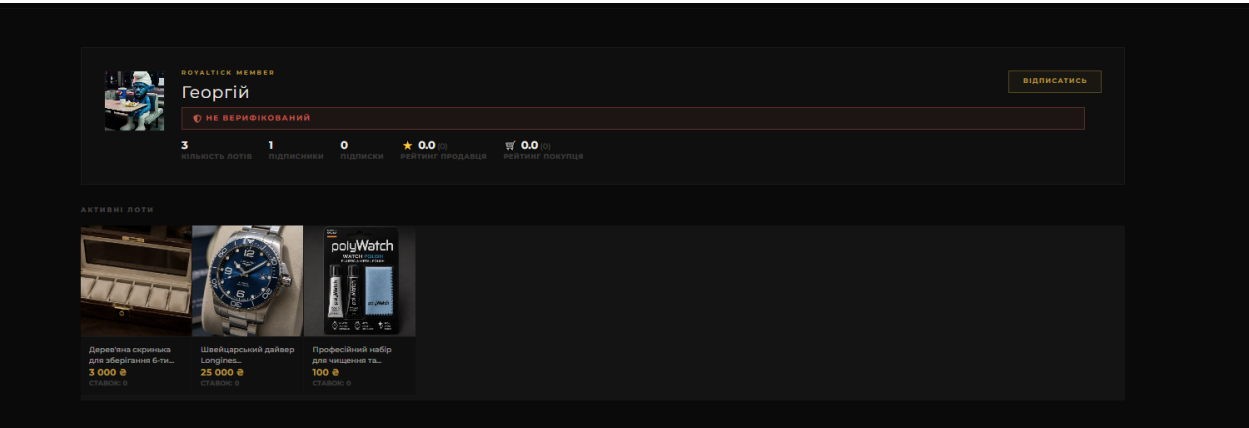


Рис. 3.39. Сторінка іншого користувача

Якщо користувач авторизований і активно користується функціоналом аукціону, то він буде отримувати сповіщення за ставку на його лот, коли йому напишуть в чат, коли його ставку переб'ють. В розділі сповіщення ми бачимо саме сповіщення, прочитати всі сповіщення, очистити. Також сповіщення клікабельні і переводять на лот або конкретний чат (рис.3.40).

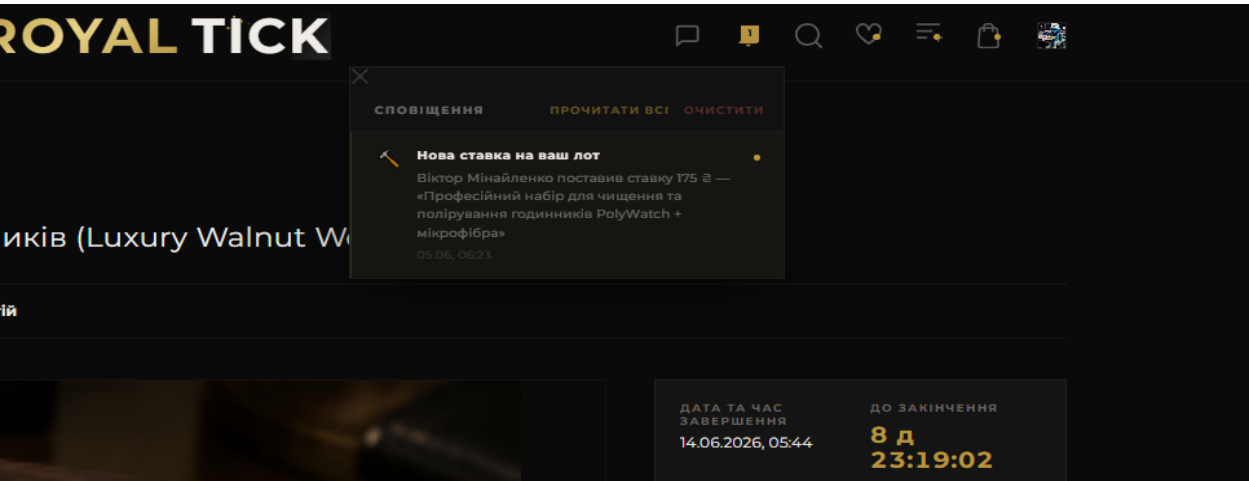


Рис. 3.40. Функціонал сповіщень

Якщо користувач захоче придбати лот, натиснувши кнопку «Купити зараз», йому висвітиться попап з підтвердженням покупки. Користувач отримує інформацію, може скасувати або підтвердити покупку. Цей елемент виступає захистом, який запобігає можливості випадково придбати лот одним кліком. Окрім фінальної ціни, у вікні чітко видно назву товару та умови

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				109
Змн.	Арк.	№ докум.	Підпис	Дата		

оплати, тому покупець одразу розуміє, за що саме і скільки він платить (рис.3.41).

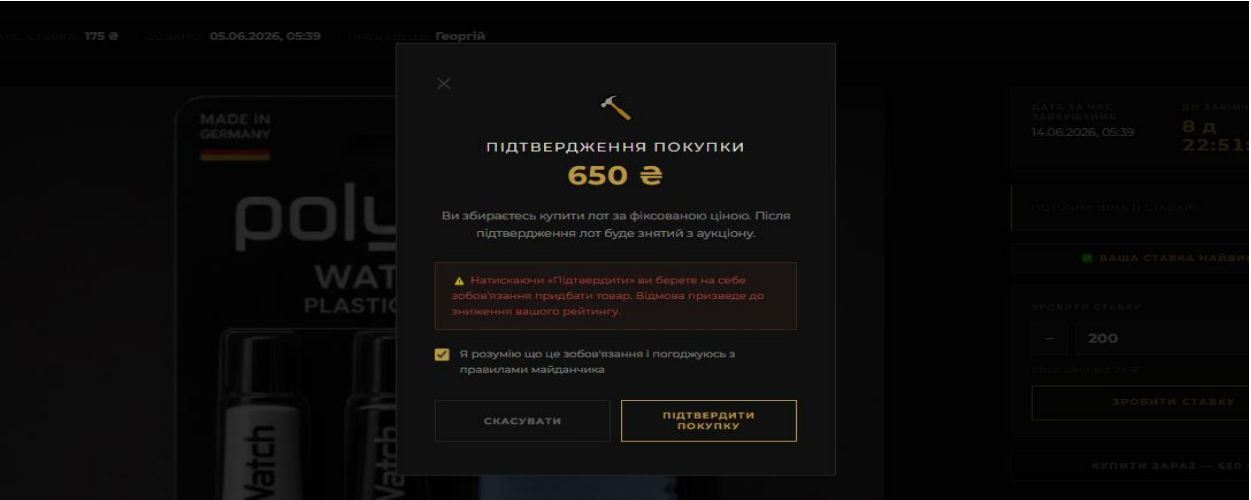


Рис. 3.41. Попап підтвердження покупки

Після того як користувач виграє в лоті або придбає його, то йому прийде автоматично згенероване повідомлення від продавця про перемогу в торгах(рис.3.42).

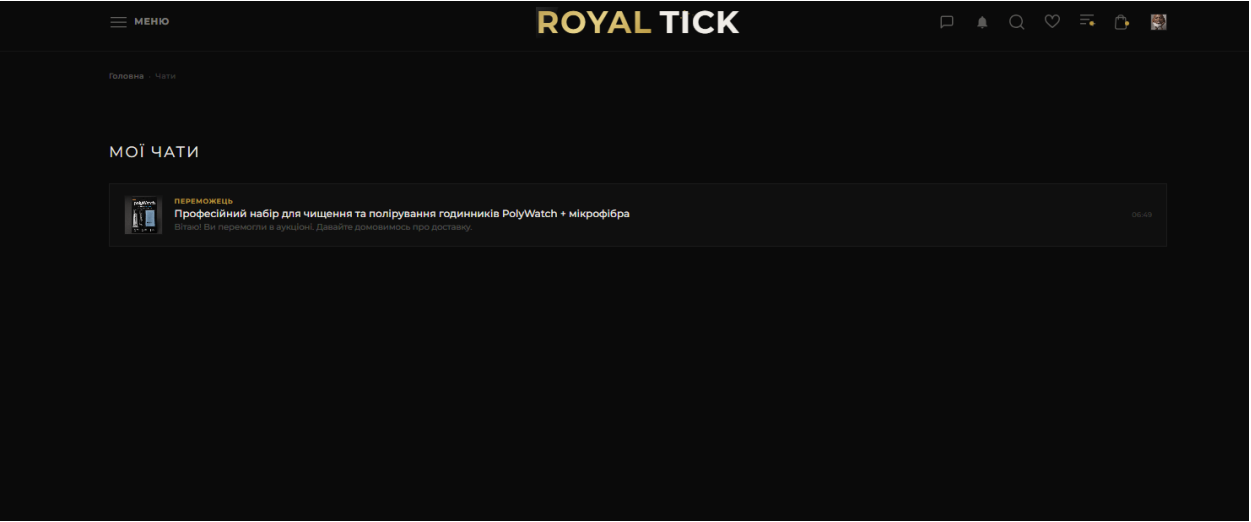


Рис. 3.42. Список чатів

Відкривши чат можна побачити повідомлення, а також переможця і продавця. Можна надіслати повідомлення і натиснути кнопку завершити угоду, ця кнопка дуже важлива і просто так її натискати не треба, вона має бути натиснута з обох сторін, щоб перейти далі. Спеціальні текстові індикатори у лівому нижньому кутку екрана чітко показують поточний статус угоди та інформують, чи підтвердила її кожна зі сторін. Червона іконка щита,

розташована у верхній панелі праворуч, дозволяє миттєво звернутися за допомогою, якщо між учасниками виникли розбіжності під час обговорення доставки. Також є значок щита, завдяки цьому можна позвати модератора до чату для вирішення якихось питань (рис.3.43).

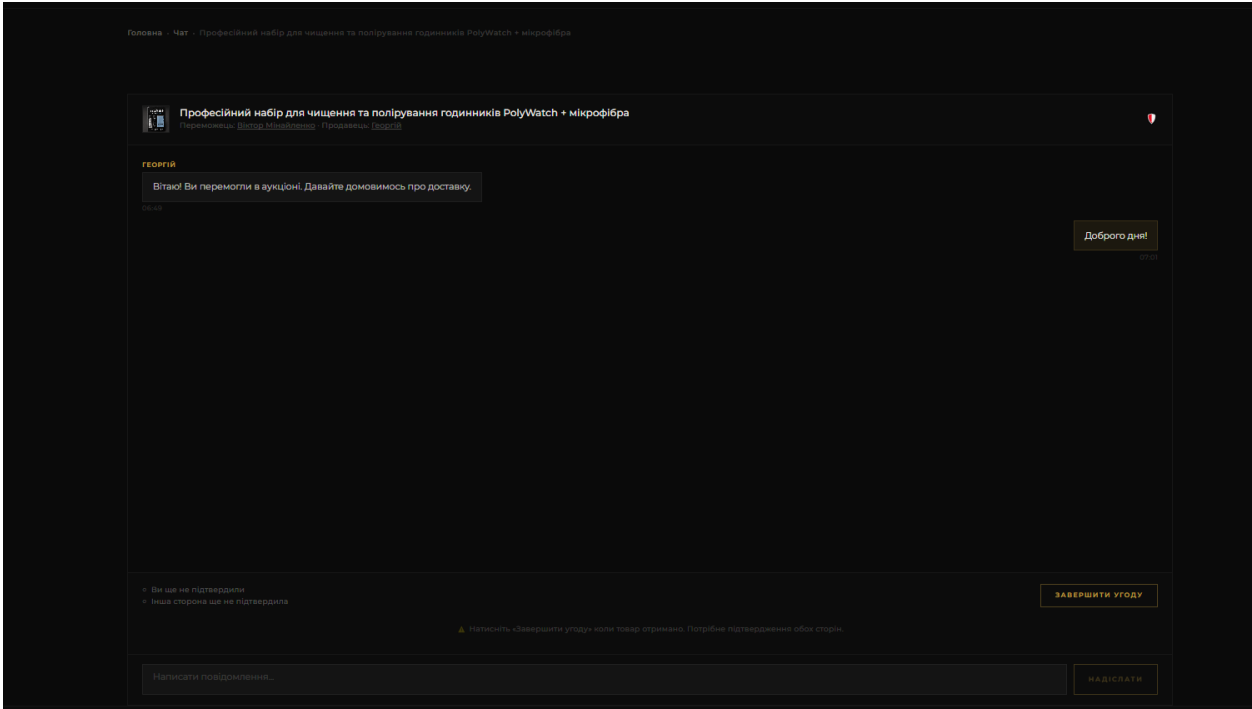


Рис. 3.43. Конкретний чат

Попап який допоможе викликати модератора. Користувач бачить повідомлення, також має дві кнопки «Викликати» та «Скасувати». Після натискання чат отримує статус очікування на модератора (рис.3.44).

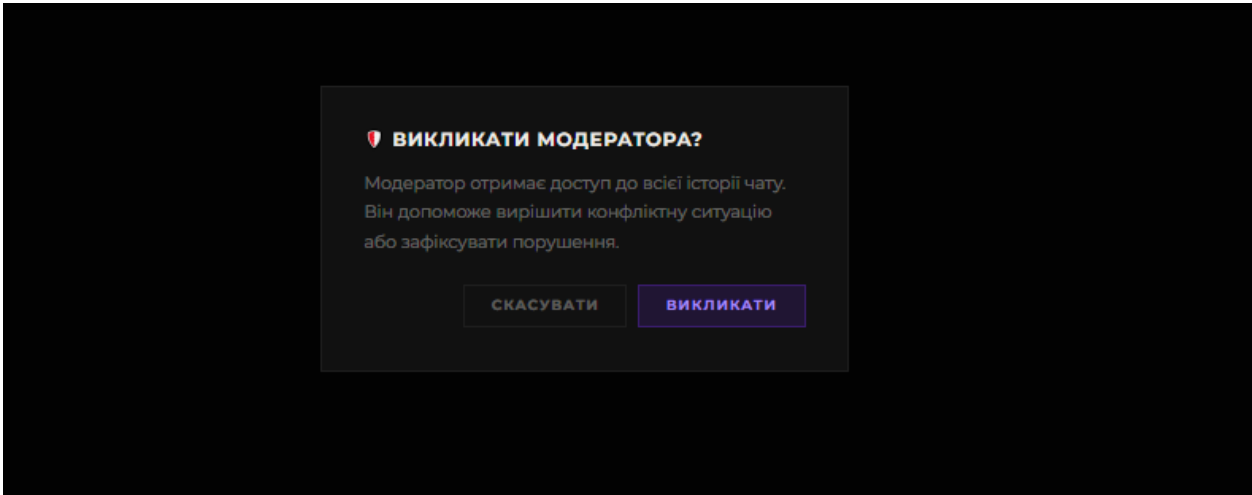


Рис. 3.44. Попап виклику модератора

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				111
Змн.	Арк.	№ докум.	Підпис	Дата		

Після того, як користувачі натиснули кнопку завершити угоду, з'являється можливість оцінити і залишити коментар. Продавець може оцінити собеседника як покупця, а покупець навпаки (рис.3.45).

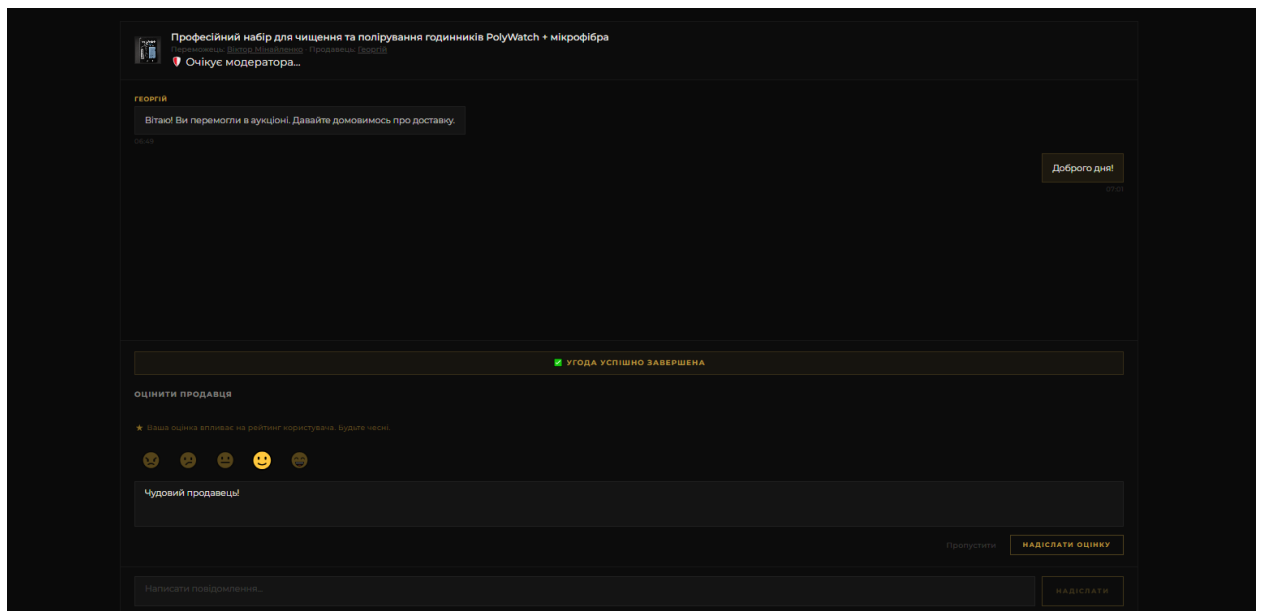


Рис. 3.45. Оцінка і написання коментаря в чаті

Після того як користувачі виставили оцінки або вирішили пропустити цей етап, у чат автоматично приходять сповіщення про дію кожного учасника. Як видно на прикладі угоди щодо професійного набору для чищення годинників PolyWatch, система фіксує текстові підтвердження наприклад, повідомлення про залишений відгук із оцінкою та коментарем «Чудовий продавець!», а також плашку про те, що інший учасник пропустив виставлення балів. Після цього з'являється фінальний статус «Угода успішно завершена», інпут для введення тексту блокується, а сам чат за бажанням можна видаляти. Окремо в системі реалізовано жорсткі часові обмеження для захисту від недобросовісних користувачів: якщо переможець аукціону ігнорує повідомлення і не виходить на зв'язок протягом 7 днів, алгоритм автоматично блокує його аккаунт. При цьому заморожений лот знімається зі статусу резерву й автоматично повертається в каталог як активний, щоб продавець не втрачав час. Крім того, передбачено механізм вирішення конфліктів. Якщо покупець чи продавець порушують правила платформи або намагаються використати баги системи, модератор може підключитися прямо до діалогу

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		112

(на скріншоті відображається мітка «Очікує модератора...»). Маючи повний доступ до історії листування, він може швидко розібратися в ситуації, врегулювати суперечку або видати покарання порушнику (рис.3.46).

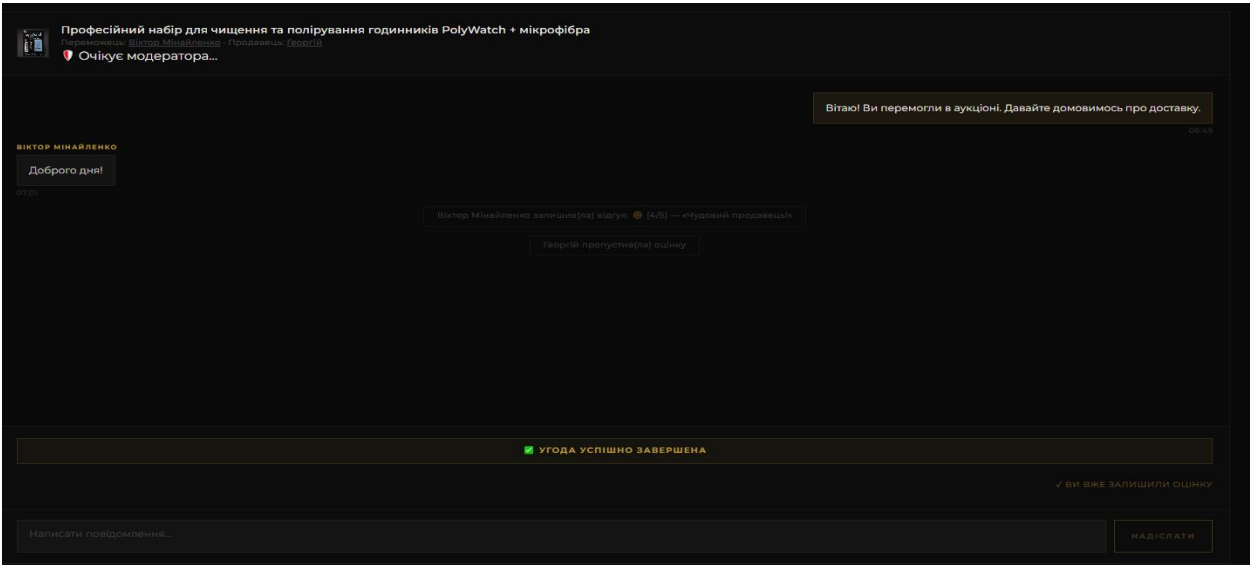


Рис. 3.46. Повідомлення в чаті і завершення угоди

Так як користувач покликав модератора, то у користувача з роллю “moderator” в адмін панелі з’явився запит. Модератор бачить кількість повідомлень в чаті, учасників, а також має кнопку приєднатися (рис.3.47).

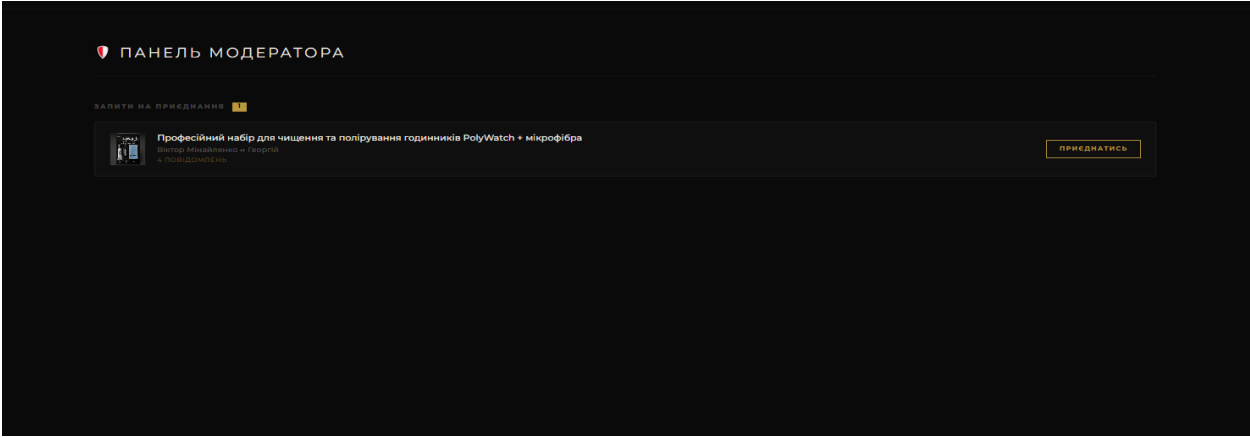


Рис. 3.47. Панель модератора

Приєднавшись модератор бачить всю історію чатів та може писати в чат. В чат виводиться інформація, що модератор приєднався. Також вісить позначка, що свідчить що перегляд відбувається з аккаунту модератора. Також чат отримує статус і видно хто модерує цей чат (рис.3.48).

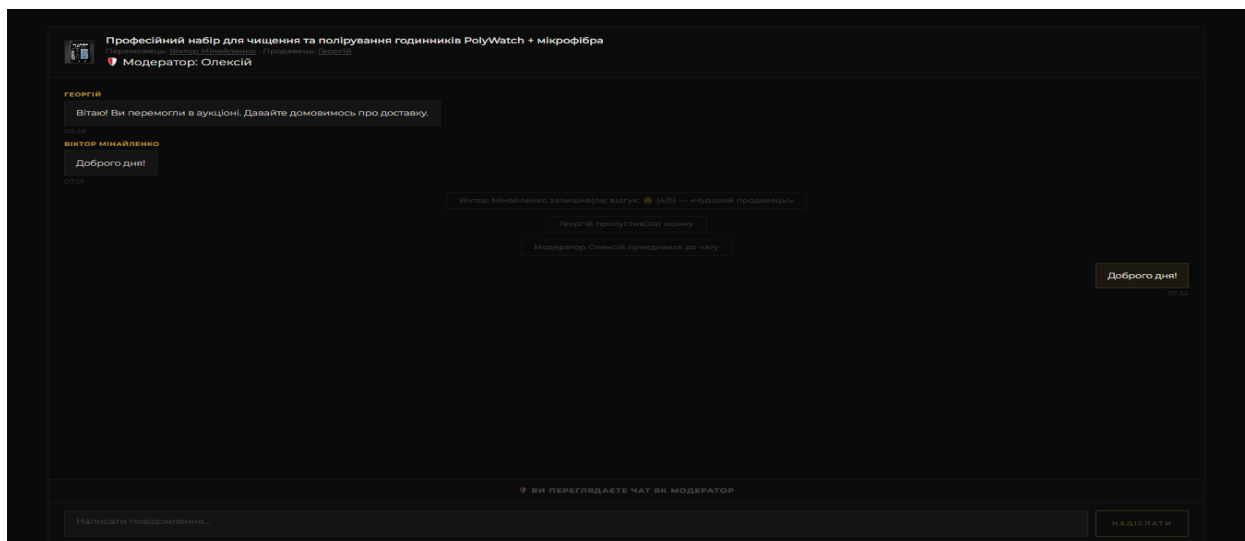


Рис. 3.48. Вигляд чату від лиця модератора

Перейшовши на сторінку користувача, користувач може натиснути на кнопку знизити рейтинг (рис.3.49). Після цього вилазить попап, де користувач може видати покарання у формі пониження рейтингу користувачеві. Модератор має можливість обрати, який рейтинг знизити, наскільки у відсотках та вказати причину покарання. Для зручності налаштування відсоток штрафу регулюється за допомогою інтуїтивного повзунка, а наявність обов'язкового поля для введення причини гарантує об'єктивність та прозорість модерації.

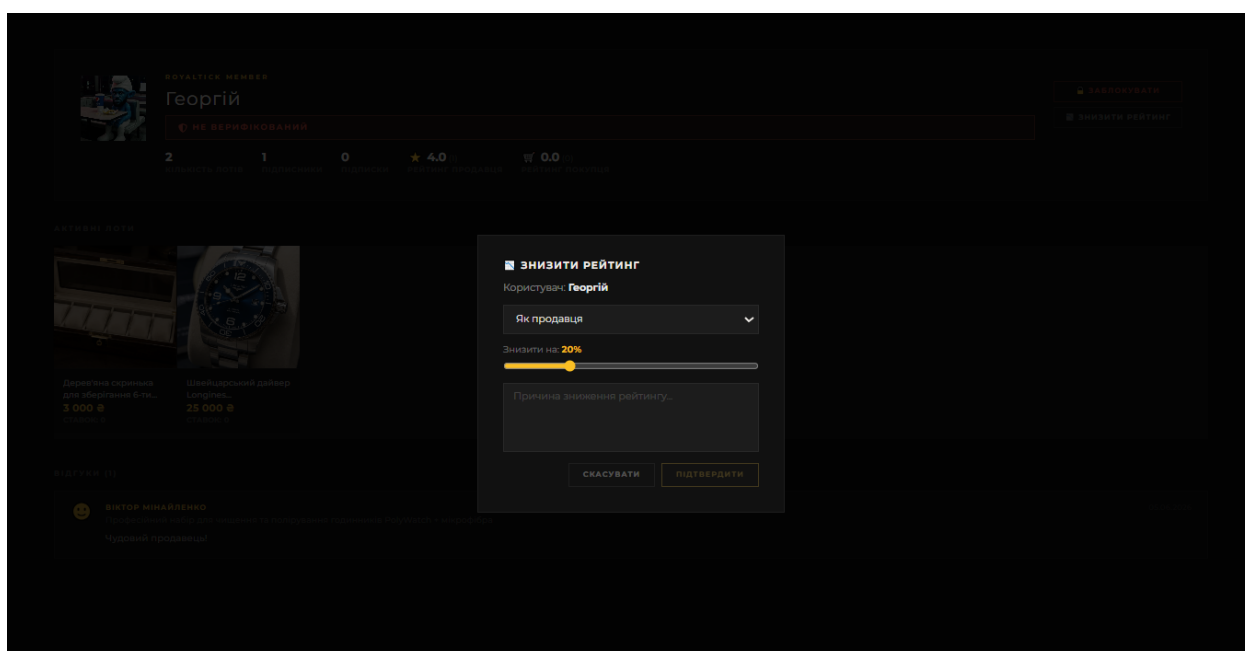


Рис. 3.49. Попап зниження рейтингу користувачу модератором

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Євдемов Ю.М.				114
Змн.	Арк.	№ докум.	Підпис	Дата		

Аналогічний попап, який дає можливість заблокувати користувача. Через це юзер не зможе приймати участь в аукціонах і робити ставки (рис.3.50).

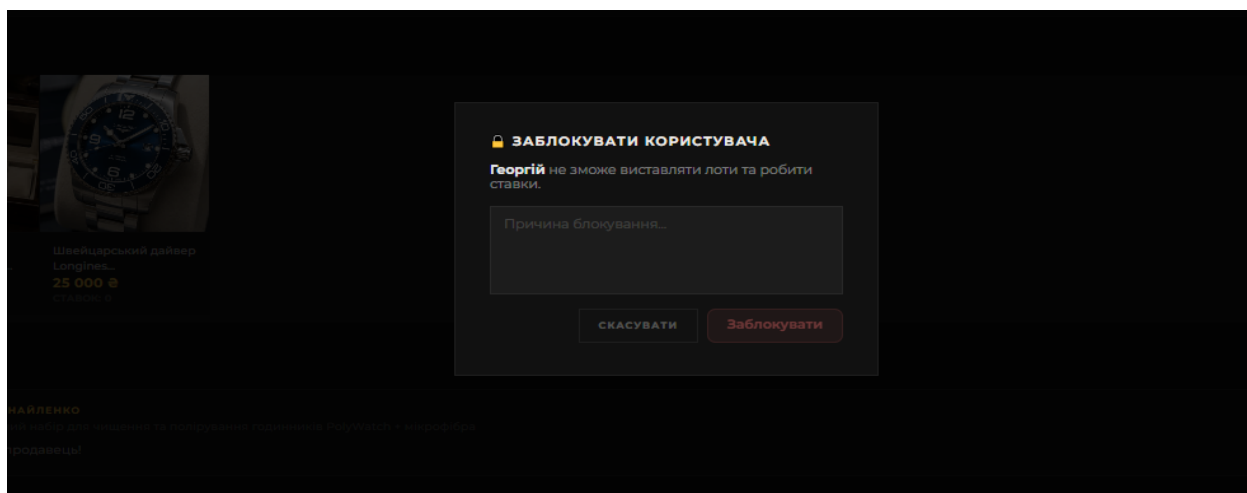


Рис. 3.50. Попап блокування користувача

На сайті пристуня сторінка 404, яка свідчить про захист роутів, а також неправильних URL. На сторінці видно повідомлення, а також можливість перейти на каталог або головну. Унікальне тематичне оформлення з інтегрованим фоновим текстом «Зона 404» та тематичним підписом про зупинку часу допомагає згладити негативне враження відсутньої сторінки. Наявність помітних навігаційних кнопок «До покупок» та «На головну» дозволяє миттєво повернути відвідувача до взаємодії з платформою без потреби використовувати стрілки браузера (рис.3.51).



Рис. 3.51. Сторінка 404

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				115
Змн.	Арк.	№ докум.	Підпис	Дата		

Стосовно адмінки, це звичайна React admin панель. Для входу треба ввести ім'я та пароль (рис.3.52).

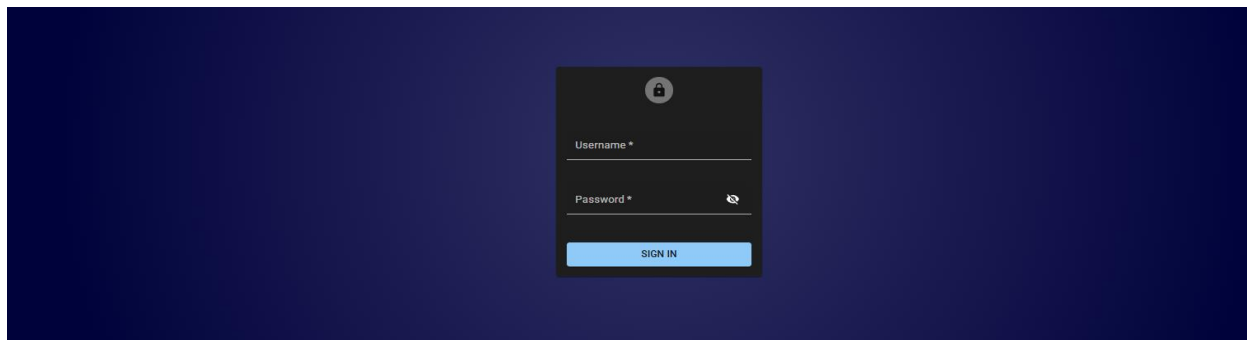


Рис. 3.52. Сторінка авторизація для входу в адмін панель

В адмін панелі можна помітити списки юзерів а також товарів, які присутні на сайті. Кнопка створити об'єкт, виділити декілька об'єктів, або перейти на його сторінку (рис.3.53).

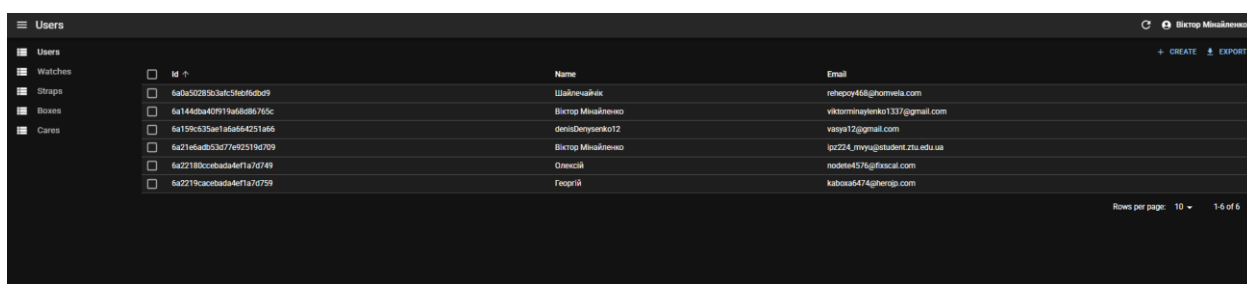


Рис. 3.53. Адмін панель

Перейшовши наприклад на якоесь юзера ми можемо редагувати його або відправити йому повідомлення на пошту про необхідність змінити пароль. Також пристуні кнопка перегляду всієї інформації про юзера або товар. Видати об'єкт можна перейшовши на нього або обрати на головній панелі адміністратора декілька (рис.3.54).

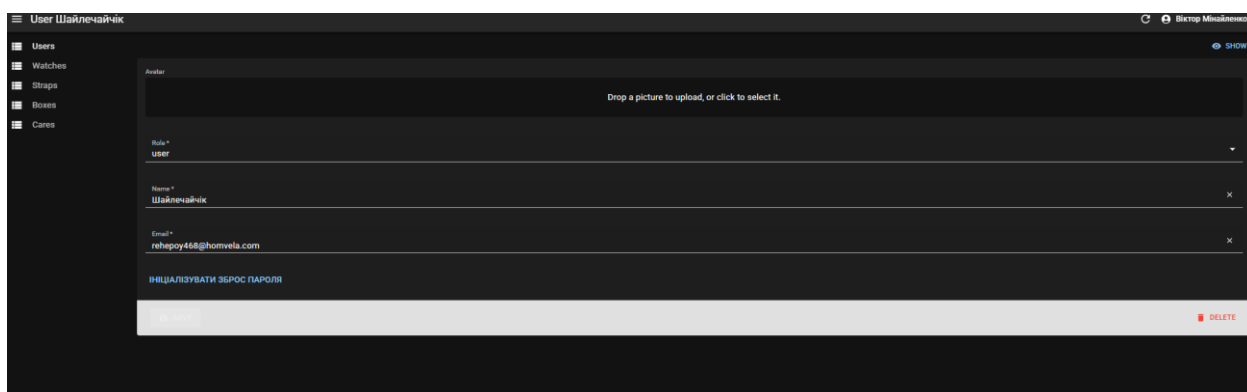


Рис. 3.54. Перегляд інформації про юзера

Присутній функціонал створення товарів або юзера. Адміністратор бачить відповідні поля необхідні для заповнення і створення об'єкту(рис.3.55).

Рис. 3.55. Створення товару

Загалом, така адмінка дає повний контроль над сайтом, вона дуже легка в освоєнні, а також допомагає швидко розбиратися в програмі, автоматизує рутинну роботу модераторів і дозволяє оновлювати всі дані на платформі буквально за декілька хвилин. В майбутньому планується розширення цього модуля.

3.3. Тестування вебплатформи та аналітика

Взагалі, етап тестування став чи не найважливішою частиною всієї розробки проєкту, адже без детальної перевірки випустити готовий маркетплейс просто неможливо. Саме завдяки тестам вдалося вчасно відловити купу дрібних багів у логіці та переконатися, що фінальна система працює саме так, як спочатку планувалося за завданням. Коли було закінчено основний код для платформи RoyalTick, постало питання про те, як правильно оцінити її стабільність, тому було прийнято рішення провести повне комплексне тестування всіх модулів. Jest відповідав за модульні та інтеграційні тести, Playwright дуже виручив у наскрізних сценаріях, PageSpeed Insights допоміг оцінити швидкість роботи інтерфейсу, а SonarCloud провів статичний аналіз самого коду[13]. Якщо говорити про модульне та інтеграційне тестування, то для перевірки логіки на сервері та різних дрібних утилітарних функцій було написано автотести на фреймворку Jest[23]. Перший набір це чисті unit-тести, де перевіряється, як працюють окремі дрібні функції

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		117

системи. Сюди ж увійшла перевірка функції `showCountMessage`, яка має красиво відмінювати іменники залежно від кількості товарів, причому як для української, так і для англійської мови з урахуванням усіх їхніх граматичних відмінностей. Окрім цього, тести для функції `shuffle`, яка перемішує масиви, утиліти `parseJwt` для декодування токенів, а також функцій `isItemInList` та `getCartItemCountBySize`, які відповідають за базову роботу з кошиком. Функції валідації параметрів на кшталт `checkPriceParam`, `getCheckedPriceFrom`, `getCheckedPriceTo`, а разом з ними й допоміжні `checkOffsetParam` та `getCheckedArrayParam`, які обробляють параметри запитів. Другий набір тестів вийшов уже інтеграційним, бо тут мені треба було подивитися, як функції кошика взаємодіють із локальним сховищем браузера, тобто з `localStorage`. Було відтворено повний життєвий цикл звичайного покупця спочатку додавав перший товар у повністю порожній кошик, потім перевіряв, чи збільшується кількість при повторному кліку, і чи система нормально розрізняє однакові товари, але різних розмірів. Також було важливо переконатися, що оновлення кількості однієї позиції ніяк не зачіпає інші елементи, а при видаленні кошик знову стає порожнім.

```
D:\My\_Institut\Online store of watches\shop>npm run test

> shop@0.1.0 test
> jest

 PASS  __tests__/unit/utils.integration.test.ts
 PASS  __tests__/unit/utils.unit.test.ts

Test Suites: 2 passed, 2 total
Tests:      61 passed, 61 total
Snapshots:  0 total
Time:       4.475 s
Ran all test suites.
```

Рис. 3.56. Результати виконання Jest тестів

Щодо наскрізного тестування, то для перевірки сценаріїв роботи користувача на сайті було написано end-to-end тести за допомогою фреймворку Playwright. Замість того, щоб просто натискати по кнопкам на сайті було створено скрип, який повністю імітує поведінку реального покупця і проходить по основному функціоналу сайта.

Тести детально перевіряють, чи відкривається каталог товарів, чи підтягуються ціни, а також як працює процес додавання годинника чи аксесуара до кошика, де обов'язково треба вибрати розмір. В цю перевірку також входять тести для списку улюблених товарів, куди можна додавати позиції та видаляти їх звідти, модуль порівняння характеристик, пагінацію при переході між сторінками каталогу та роботу модального вікна швидкого перегляду, коли клікаєш на деталі продукту. Було розраховано, що інтерфейс динамічний і якісь дані вантажаться трохи довше, тому скрипти вміють адаптуватися до стану сторінки. Наприклад, спочатку автоматично обробляють спливаючий банер згоди з файлами cookie, а потім чекають на повне завантаження контенту з бази даних, перш ніж робити наступний крок. Оботи користувача на сайті було написано end-to-end тести за допомогою фреймворку Playwright[22]. Замість того, щоб просто натискати по кнопкам на сайті було створено скрип, який повністю імітує поведінку реального покупця і проходить по основному функціоналу сайту.

✓	RoyalTick E2E Flows	Користувач може додати товар в "Улюблені"	chromium	6.6s
		royal-tick-e2e.spec.ts:156		
✓	RoyalTick E2E Flows	Користувач може переглядати каталог	chromium	2.8s
		royal-tick-e2e.spec.ts:226		
✓	RoyalTick E2E Flows	Користувач може перейти в порівнювання	chromium	9.7s
		royal-tick-e2e.spec.ts:273		
✓	RoyalTick E2E Flows	Користувач може перейти в кошик	chromium	6.9s
		royal-tick-e2e.spec.ts:330		
✓	RoyalTick E2E Flows	Користувач може фільтрувати товари по категоріях	chromium	4.3s
		royal-tick-e2e.spec.ts:367		
✓	RoyalTick E2E Flows	Користувач може переглядати деталі товару	chromium	3.0s
		royal-tick-e2e.spec.ts:396		
✓	RoyalTick E2E Flows	Користувач може видалити товар з улюблених	chromium	6.6s
		royal-tick-e2e.spec.ts:445		
✓	RoyalTick E2E Flows	Користувач може переключатися між сторінками	chromium	7.1s
		royal-tick-e2e.spec.ts:531		

Рис. 3.57. Результати виконання e2e тестів

Для оцінки того, наскільки швидко працює клієнтська частина маркетплейсу і чи взагалі інтерфейс зручний для звичайних користувачів, було вирішено провести тестування через сервіс Google PageSpeed Insights. Аналізу піддали версію платформи RoyalTick, яку заздалегідь розгорнули на хостингу

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		119

Vercel[29]. Загалом сервіс проаналізував вебсторінку за чотирма основними напрямками й видав досить непогані підсумкові цифри, які якраз зафіксовано на (рис.3.58) Оцінка за впровадження оптимальних методів розробки склала 96 балів, показник доступності інтерфейсу зупинився на 86 балах, оптимізація для пошукових систем взагалі вибила максимальні 100 балів, а загальна ефективність швидкодії продемонструвала результат у 89 балів. Якщо детальніше розібрати окремі метрики перформансу, то вимальовується досить цікава картина. Наприклад, перше відображення контенту (метрика FCP) займає всього 0.4 секунди, що за суворими критеріями Google впевнено входить у зелену зону «добре».

Повне завантаження найбільшого елемента контенту (LCP) триває близько 1.2 секунди, а загальний час блокування основного потоку інтерфейсу (TBT) склав усього 190 мілісекунд. Також дуже хорошим виявився кумулятивний індекс зміщення макета (CLS), який зупинився на позначці 0.018 це підтверджує, що під час рендерингу сторінки блоки сайту не стрибають хаотично по екрану. Що стосується загальної швидкості відображення всього видимого контенту (Speed Index), то вона підтягнулася до 1.8 секунди. Проте проведений аудит чітко підсвітив головне слабе місце в продуктивності платформи, яке безпосередньо пов'язане з графічним контентом. З'ясувалося, що зображення товарів зберігаються у звичайному важкому форматі PNG та ще й у надто великій роздільній здатності (аж до 1024×1536 пікселів), хоча для адекватного відображення в картках каталогу достатньо значно менших розмірів. Через це загальна вага графіки на сторінці роздулася майже до 30 мегабайт. Якщо перевести всі ці картинки на сучасні адаптивні формати типу WebP або AVIF та обмежити їхні вихідні розміри, це дозволить заощадити близько 30 574 KiB мережевого трафіку, тому таку оптимізацію визначили як пріоритетне завдання для наступних оновлень системи.

Стосовно інших оцінок, то максимальний бал за SEO підтвердив, що маркетплейс повністю відповідає вимогам пошукових роботів. А от над

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		120

доступністю в 86 балів ще доведеться попрацювати. Аналіз показав, що частина іконкових кнопок не має обов'язкових текстових міток aria-label, через що програми-екранні диктори не зможуть озвучити їх для людей із порушеннями зору. Крім того, контрастність деяких елементів тексту та фону виявилася недостатньою, що теж потребує виправлення в майбутньому.

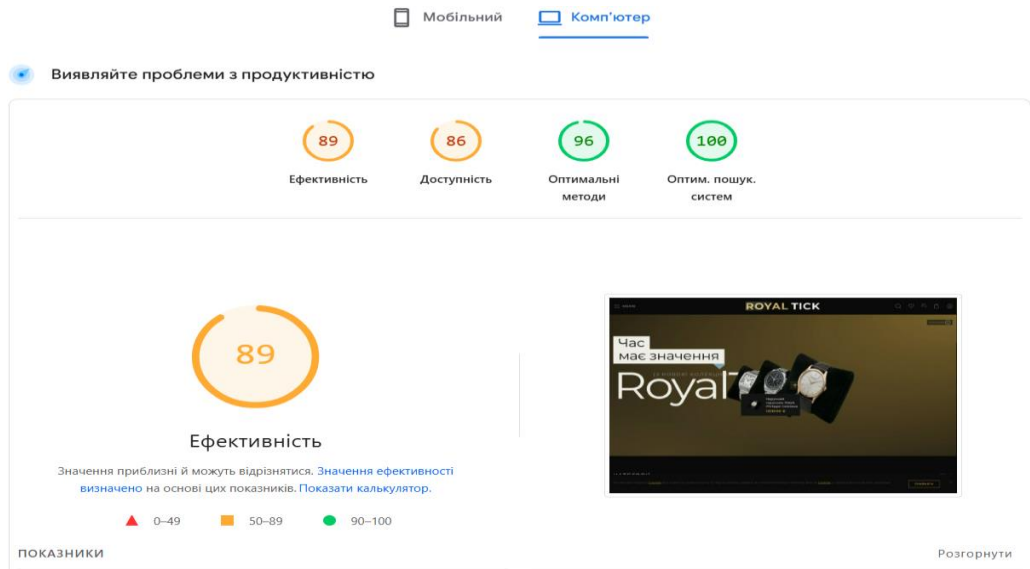


Рис. 3.58. Результати тестування PageSpeed Insights

Щоб переконатися, що інтерфейс маркетплейсу відображається без помилок на будь-яких гаджетах, було проведено тестування адаптивності за допомогою вбудованих інструментів Chrome DevTools у режимі емуляції мобільних пристроїв. Перевірку рендерингу основних сторінок платформи здійснювали для екранів абсолютно різної ширини. Сюди увійшли як компактні мобільні телефони (із розширенням в межах 360–480 пікселів), так і планшети (від 768 до 1024 пікселів) разом зі звичайними настільними комп'ютерами. Завдяки цим тестам вдалося підтвердити, що гнучка CSS-сітка каталогу товарів працює абсолютно коректно й підлаштовується під розміри дисплея. Крім того, велике десктопне навігаційне меню при згортанні сторінки без проблем трансформується у звичну мобільну шторку (бургер-меню), а всі спливаючі модальні вікна та форми замовлень автоматично масштабуються під ширину екрана, не вилазячи за його межі. Всі кнопки і навігація переходять у вигляд мобільного меню. Елементи зроблені спеціально великими, щоб

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				121
Змн.	Арк.	№ докум.	Підпис	Дата		

було легко влучати пальцями на будь-якому розширенні екрану. При цьому картинки не стискаються до нечитаємих розмірів, а просто акуратно обрізаються або зменшуються без витрат необхідної якості.(рис. 3.59).



Рис. 3.59. Результати тестування PageSpeed Insights

Щоб відстежувати реальну активність відвідувачів на платформі, до проєкту було підключено сервіс аналітики Google Analytics 4. Під час проведення тестового навантаження системи моніторинг у режимі реального часу зафіксував одночасну присутність 11 активних користувачів. Якщо проаналізувати статистику переглянутих ними сторінок, то цілком очікувано найпопулярнішою виявилася сторінка каталогу товарів. На неї припало 11 переглядів, що становить 40.74% від усього обсягу трафіку, і це зайвий раз підтверджує її ключову роль у загальній навігації маркетплейсом. Серед усіх зафіксованих системою подій найбільшу частку склали стандартні page_view (відвідування сторінок) загалом 27 разів. Також аналітика нарахувала по 13

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				122
Змн.	Арк.	№ докум.	Підпис	Дата		

подій для first_visit (перший візит) та session_start (початок сесії), 12 взаємодій типу user_engagement та 10 скролінгів сторінки (scroll). Якщо дивитися на розподіл переглядів детальніше, то окрім чистого каталогу користувачі активно взаємодіяли з каталогом, де було увімкнено фільтри товарів (8 переглядів), часто заходили на головну сторінку (7 переглядів) та один раз відкривали модуль порівняння характеристик. Загалом такі зібрані метрики чітко вказують на те, що система навігації працює без збоїв, а тестові користувачі виявили найбільший інтерес саме до функціоналу пошуку, перегляду та фільтрації асортименту годинників (рис. 3.60).

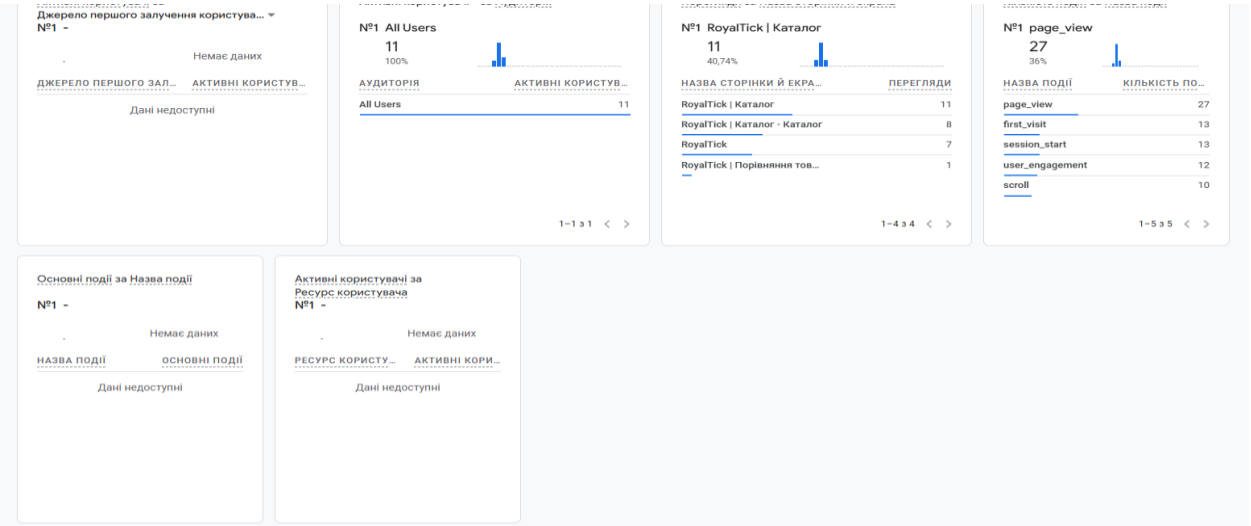


Рис. 3.60. Результат аналітики

Висновки до третього розділу

У цьому розділі було детально розібрано практичний процес розгортання та комплексного тестування створеної вебплатформи RoyalTick. Спочатку вдалося змоделювати детальну діаграму розгортання, яка наочно описує, як саме взаємодіють між собою клієнтська частина на базі фреймворку Next.js, серверні API-маршрути, хмарна база даних MongoDB Atlas та підключені зовнішні сервіси й системи.

Також у розділі покроково розписано всю послідовність конфігурації хостингу на платформі Vercel, включаючи правильне налаштування необхідних змінних середовища (env-файлів). Для більшої наочності основний

функціонал та модулі інтерфейсу користувача було представлено у вигляді графічних зрізів сюди увійшли сторінки каталогу товарів, аукціонний розділ, сторінка спільноти та кастомна панель модератора.

Окрему увагу в розділі приділено автоматизації перевірки коду. Для тестування серверної логіки та утиліт успішно прогнали 61 автотест на базі фреймворку Jest, а для перевірки повноцінних клієнтських сценаріїв та шляху користувача реалізували наскрізні end-to-end тести засобами Playwright. Фронтенд-частину платформи додатково проаналізували через сервіси PageSpeed Insights та SonarCloud, що дозволило отримати повний аудит реальної швидкодії, чистоти кодової бази, оптимізації та безпеки додатка. Крім того, за допомогою інтеграції інструментів Google Analytics 4 вдалося детально відстежити першу активність користувачів на сайті. Фінальним кроком стала перевірка адаптивності дизайну за допомогою вбудованих інструментів браузера, яка повністю підтвердила, що інтерфейс маркетплейсу відображається без помилок на будь-яких типах пристроїв.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				124
Змн.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВКИ

Під час виконання кваліфікаційної роботи бакалавра було проведено дослідження та розроблено вебплатформу RoyalTick, яка поєднує функції інтернет-магазину годинників із модулем аукціонів у реальному часі. На початковому етапі було проаналізовано існуючі рішення у сфері електронної комерції, що дозволило сформулювати чіткі вимоги до майбутньої системи зокрема, щодо підтримки багатомовності, авторизації через різні провайдери, наявності модуля спільноти та кастомної панелі модератора.

У роботі детально описано проєктування системи за принципом модульного моноліту на базі фреймворку Next.js, а також розібрано рольові моделі для покупця, продавця, модератора та адміністратора. За допомогою UML-діаграм використання, послідовностей та кооперації було побудовано об'єктно-орієнтовану модель проєкту та спроектовано структуру бази даних MongoDB. На етапі розробки вдалося успішно реалізувати захищену JWT-автентифікацію, логіку торгів із автоматичним завершенням, інтеграцію платіжного шлюзу WayForPay та збереження медіафайлів у хмарі Cloudinary.

Для забезпечення стабільної роботи платформи було визначено вимоги до серверного оточення та побудовано діаграму розгортання, після чого проєкт деплоїли на хостинг Vercel із підключенням хмарної бази даних MongoDB Atlas. На завершальному етапі провели всебічний аудит якості системи прогнали 61 автоматизований Jest-тест, реалізували наскрізні сценарії на базі Playwright, перевірили швидкість через PageSpeed Insights (де інтерфейс показав 89 балів ефективності та 100 балів за SEO) та оцінили чистоту коду через аналізатор SonarCloud.

Отримані результати повністю підтверджують правильність обраних архітектурних рішень. Створений програмний продукт є готовою базою для подальшого розвитку маркетплейсу, де основними точками росту визначено глибшу оптимізацію графічного контенту, розширення покриття автотестами та вдосконалення типізації TypeScript-компонентів.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		125

ДЖЕРЕЛА ІНФОРМАЦІЇ

1. Мінайленко В.О., Єфремов Ю.М. ВЕБПЛАТФОРМА ДЛЯ ТОРГІВЛІ ГОДИННИКАМИ ТА АКСЕСУАРАМИ З ПІДТРИМКОЮ АУКЦІОНУ ТА ОБМІНУ. Тези доповідей XVI Міжнародної науково-технічної конференції Інформаційно-комп'ютерні технології - 2026, 02-03 квітня 2026 року. Житомир: «Житомирська політехніка», 2026.
2. Мінайленко В.О., Єфремов Ю.М. ПРОЄКТУВАННЯ КЛІЄНТСЬКОЇ ЧАСТИНИ ВЕБПЛАТФОРМИ ДЛЯ ТОРГІВЛІ ГОДИННИКАМИ НА ОСНОВІ БІБЛІОТЕКИ REACT. Тези доповідей XVI Міжнародної науково-технічної конференції Інформаційно-комп'ютерні технології - 2026, 02-03 квітня 2026 року. Житомир: «Житомирська політехніка», 2026.
3. Фаулер М. Рефакторинг: поліпшення дизайну існуючого коду. 2-ге вид. Київ : Спадщина, 2019. 432 с.
4. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley, 1994. 395 p.
5. Pressman R. S., Maxim B. R. Software Engineering: A Practitioner's Approach. 9th ed. New York: McGraw-Hill, 2019. 848 p.
6. OMG. UML Profile for Software Architecture. URL: <https://www.omg.org/spec/UML/> (дата звернення: 01.01.2026).
7. Booch G., Rumbaugh J., Jacobson I. The Unified Modeling Language User Guide. 2nd ed. Boston: Addison-Wesley, 2005. 560 p.
8. Next.js Documentation. Getting Started with App Router. URL: <https://nextjs.org/docs> (дата звернення: 15.01.2026).
9. MongoDB Documentation. Introduction to MongoDB Atlas [Електронний ресурс] – Режим доступу до ресурсу: <https://www.mongodb.com/docs/atlas/>
10. Cockburn A. Writing Effective Use Cases. Boston: Addison-Wesley, 2000. 264 p.

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				126
Змн.	Арк.	№ докум.	Підпис	Дата		

- 11.StarUML Documentation [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.staruml.io>
- 12.TypeScript Documentation. TypeScript for JavaScript Programmers [Електронний ресурс] – Режим доступу до ресурсу: <https://www.typescriptlang.org/docs/>
- 13.SonarCloud Documentation. Exploring the Static Analysis [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.sonarcloud.io>
- 14.React Documentation. Quick Start [Електронний ресурс] – Режим доступу до ресурсу: <https://react.dev/learn>
- 15.Firebase Documentation. Authentication Overview [Електронний ресурс] – Режим доступу до ресурсу: <https://firebase.google.com/docs/auth>
- 16.Vercel Documentation. Deploying Next.js Applications [Електронний ресурс] – Режим доступу до ресурсу: <https://vercel.com/docs/frameworks/nextjs>
- 17.RFC 7519. JSON Web Token (JWT). Internet Engineering Task Force [Електронний ресурс] – Режим доступу до ресурсу: <https://datatracker.ietf.org/doc/html/rfc7519>
- 18.WayForPay. Документація API платіжного шлюзу [Електронний ресурс] – Режим доступу до ресурсу: <https://wiki.wayforpay.com>
- 19.Cloudinary Documentation. Image Management and Optimization [Електронний ресурс] – Режим доступу до ресурсу: <https://cloudinary.com/documentation>
- 20.Docker Documentation. Overview of Docker Compose and Multi-stage Builds [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.docker.com/compose/>
- 21.GitLab Documentation. CI/CD Pipelines [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.gitlab.com/ee/ci/>
- 22.Playwright Documentation. Getting Started with E2E Testing [Електронний ресурс] – Режим доступу до ресурсу: <https://playwright.dev/docs/intro>

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
		Єфремов Ю.М.				127
Змн.	Арк.	№ докум.	Підпис	Дата		

- 23.Jest Documentation. Getting Started with Unit Testing [Електронний ресурс] – Режим доступу до ресурсу: <https://jestjs.io/docs/getting-started>
- 24.Effector Documentation. State Management for JavaScript Applications [Електронний ресурс] – Режим доступу до ресурсу: <https://effector.dev/docs/introduction/motivation>
- 25.Sass Documentation. SCSS Syntax and Modules [Електронний ресурс] – Режим доступу до ресурсу: <https://sass-lang.com/documentation>
- 26.Provos N., Mazières D. A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference, 1999. P. 32–41.
- 27.Панасюк О. В. Використання паттерну Repository та Unit of Work в архітектурі сучасних вебзастосунків. Комп'ютерні технології та інновації. DOI: <https://doi.org/10.15407/cti2025.02.018>
- 28.Редмонд Е., Вілсон Дж. Сім баз даних за сім тижнів. Порівняльний аналіз реляційних та NoSQL СУБД. Даллас : Pragmatic Bookshelf, 2021. 312 р.
- 29.Google PageSpeed Insights Documentation. Performance Scoring [Електронний ресурс] – Режим доступу до ресурсу: <https://developers.google.com/speed/docs/insights/v5/about>
- 30.Framer Motion Documentation. Animation Library for React [Електронний ресурс] – Режим доступу до ресурсу: <https://www.framer.com/motion/>

ДОДАТКИ

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		129

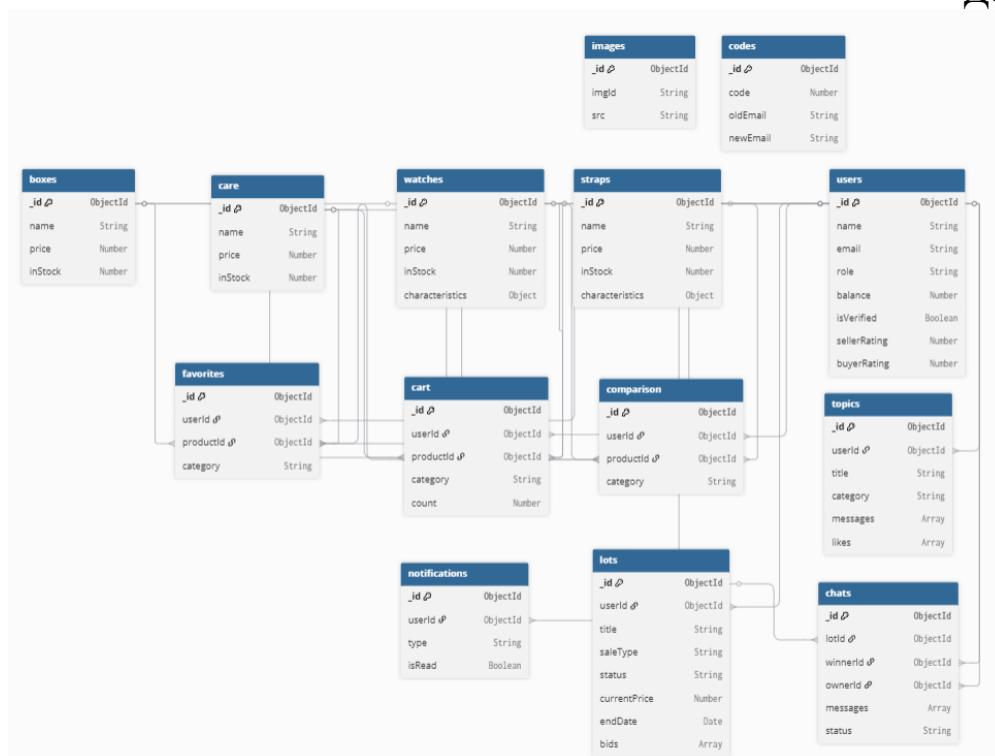


Рис. Скорочена діаграма структури бд

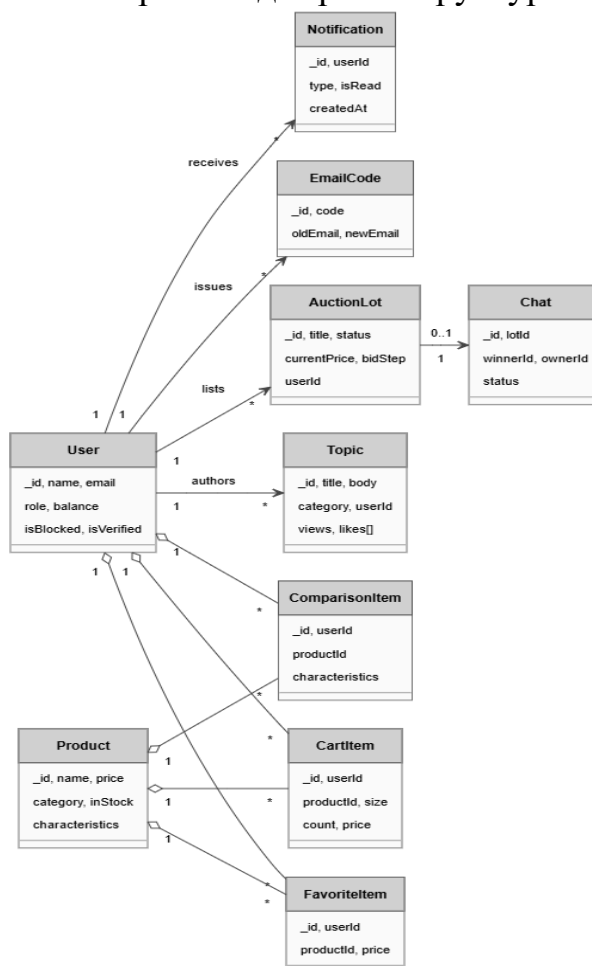


Рис. Скорочена діаграма класів

Лістинг авторизація з JWT:

```

import bcrypt from 'bcryptjs'
import { NextResponse } from 'next/server'
import clientPromise from '@lib/mongodb'
import { logger } from '@lib/logger'
import {
  findUserByEmail,
  generateTokens,
  getDbAndReqBody,
} from '@lib/utils/api-routes'
import { corsHeaders } from '@constants/corsHeaders'

export async function POST(req: Request) {
  try {
    const { db, reqBody } = await getDbAndReqBody(clientPromise, req)

    logger.info({ email: reqBody.email }, 'User login attempt')

    const user = await findUserByEmail(db, reqBody.email)

    if (!user) {
      logger.warn({ email: reqBody.email }, 'Login failed: user does not exist')
      return NextResponse.json({
        warningMessage: 'Користувача не існує',
      }, corsHeaders)
    }

    if (!bcrypt.compareSync(reqBody.password, user.password)) {
      logger.warn({ email: reqBody.email }, 'Login failed: invalid password')
      return NextResponse.json({
        warningMessage: 'Невірний логін або пароль!',
      }, corsHeaders)
    }

    const tokens = generateTokens(user.name, reqBody.email)

    logger.info({ email: reqBody.email }, 'User logged in successfully')
    return NextResponse.json({ ...tokens, role: user?.role, username: user.name },
corsHeaders)

  } catch (error) {
    logger.error(error, 'Unexpected error during login')
    return NextResponse.json({ message: 'Internal Server Error' }, { status: 500
})
  }
}

export async function OPTIONS(){

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		131

```

    return new NextResponse(null, {...corsHeaders, status: 200})
}

```

Лістинг логіка маршруту [id]/bid/route.ts:

```

import { NextResponse } from 'next/server'
import { corsHeaders } from '@/constants/corsHeaders'
import clientPromise from '@/lib/mongodb'
import { getDbAndReqBody, isValidAccessToken, parseJwt, findUserByEmail } from
'@/lib/utils/api-routes'
import { ObjectId } from 'mongodb'
import { createNotification } from '@/lib/utils/createNotification'

export async function POST(
  req: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params

    const token = req.headers.get('authorization')?.split(' ')[1]
    const validatedTokenResult = await isValidAccessToken(token)

    if (validatedTokenResult.status !== 200) {
      return NextResponse.json(validatedTokenResult, corsHeaders)
    }

    const { db, reqBody } = await getDbAndReqBody(clientPromise, req)
    const { bidAmount } = reqBody

    const lot = await db.collection('lots').findOne({ _id: new ObjectId(id)
  })

    if (!lot) {
      return NextResponse.json({ message: 'Лот не знайдено', status: 404 },
corsHeaders)
    }

    if (lot.status !== 'active') {
      return NextResponse.json({ message: 'Аукціон завершено', status: 400
}, corsHeaders)
    }

    if (new Date() > new Date(lot.endDate)) {
      return NextResponse.json({ message: 'Час аукціону вийшов', status:
400 }, corsHeaders)
    }

    const user = await findUserByEmail(db, parseJwt(token as string).email)

    if (user?.isBlocked) {

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		132

```

        return NextResponse.json({
            message: 'Ваш акаунт заблоковано. Ви не можете робити ставки.',
            status: 403,
        }, corsHeaders)
    }

    if (String(lot.userId) === String(user?._id)) {
        return NextResponse.json({ message: 'Не можна ставити ставку на
власний лот', status: 403 }, corsHeaders)
    }

    const minBid = lot.currentPrice + lot.bidStep
    if (bidAmount < minBid) {
        return NextResponse.json(
            { message: `Мінімальна ставка: ${minBid} €`, status: 400 },
            corsHeaders
        )
    }

    const newBid = {
        userId: user?._id,
        userName: user?.name,
        amount: bidAmount,
        createdAt: new Date(),
    }

    await db.collection('lots').updateOne(
        { _id: new ObjectId(id) },
        {
            $set: { currentPrice: bidAmount },
            $push: { bids: newBid } as any,
        }
    )

    await createNotification({
        db,
        userId: lot.userId,
        type: 'bid_on_lot',
        actorName: user?.name,
        lotTitle: lot.title,
        bidAmount,
        href: `/auction/${id}`,
    })

    const previousTopBid = lot.bids[lot.bids.length - 1]
    if (
        previousTopBid &&
        String(previousTopBid.userId) !== String(user?._id) &&
        String(previousTopBid.userId) !== String(lot.userId)
    ) {

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				133
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        await createNotification({
            db,
            userId: previousTopBid.userId,
            type: 'bid_outbid',
            actorName: user?.name,
            lotTitle: lot.title,
            bidAmount,
            href: `/auction/${id}`,
        })
    }

    const updatedLot = await db.collection('lots').findOne({ _id: new
    ObjectId(id) })

    return NextResponse.json({ status: 200, lot: updatedLot }, corsHeaders)
  } catch (error) {
    return NextResponse.json({ message: (error as Error).message, status: 500
    }, corsHeaders)
  }
}

```

`export const dynamic = 'force-dynamic'`

Логіка маршруту фіналізації аукціону `finalize/route.ts`:

```

import { NextResponse } from 'next/server'
import { corsHeaders } from '@/constants/corsHeaders'
import clientPromise from '@/lib/mongodb'
import { getDbAndReqBody } from '@/lib/utils/api-routes'
import { ObjectId } from 'mongodb'

export async function POST() {
  try {
    const { db } = await getDbAndReqBody(clientPromise, null)

    const expiredLots = await db.collection('lots').find({
      status: 'active',
      endDate: { $lt: new Date() },
      'bids.0': { $exists: true },
    }).toArray()

    let finalized = 0

    for (const lot of expiredLots) {
      const winnerBid = lot.bids[lot.bids.length - 1]

      const winner = await db.collection('users').findOne({
        _id: new ObjectId(String(winnerBid.userId)),
      })

      if (winner) {

```

		Мінайленко В. Ю.			ІІЗ.КР.Б – 121 – 26 – ІІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		134

```

const welcomeMessage = {
  _id: new ObjectId(),
  senderId: lot.userId,
  senderName: lot.userName,
  text: 'Вітаю! Ви перемогли в аукціоні. Давайте домовимось про
доставку.',
  createdAt: new Date(),
  isRead: false,
}

await db.collection('chats').updateOne(
  { lotId: lot._id },
  {
    $setOnInsert: {
      lotId: lot._id,
      lotTitle: lot.title,
      lotPhoto: lot.mainPhotoUrl,
      winnerId: winner._id,
      winnerName: winner.name,
      ownerId: lot.userId,
      ownerName: lot.userName,
      messages: [welcomeMessage],
      createdAt: new Date(),
      status: 'active',
      unreadForOwner: true,
      deletedFor: [],
      moderatorRequested: false,
      moderatorRequestedBy: null,
      moderatorId: null,
      moderatorName: null,
    },
  },
  { upsert: true }
)

finalized++
}

await db.collection('lots').updateOne(
  { _id: lot._id },
  { $set: { status: 'reserved' } }
)
}

return NextResponse.json({ status: 200, finalized }, corsHeaders)
} catch (error) {
  return NextResponse.json(
    { message: (error as Error).message, status: 500 },
    corsHeaders
  )
}

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		135

```

    }
}

```

```
export const dynamic = 'force-dynamic'
```

Лістинг інтеграція з WayForPay

payment/route.ts:

```

import { NextResponse } from 'next/server'
import crypto from 'crypto'
import clientPromise from '@lib/mongodb'
import { getAuthRouteData } from '@lib/utils/api-routes'

export async function POST(req: Request) {
  try {
    const { validatedTokenResult, reqBody } = await getAuthRouteData(
      clientPromise,
      req
    )

    if (validatedTokenResult.status !== 200) {
      return NextResponse.json(validatedTokenResult)
    }

    const merchantAccount = process.env.WAYFORPAY_MERCHANT_ACCOUNT
    const merchantSecretKey = process.env.WAYFORPAY_MERCHANT_SECRET_KEY

    if (!merchantAccount || !merchantSecretKey) {
      return NextResponse.json({ message: 'API keys are missing' }, {
status: 500 })
    }

    const baseUrl = process.env.NEXT_PUBLIC_BASE_URL ||
'http://localhost:3000'
    const orderReference = `order_${Date.now()}`
    const orderDate = Math.floor(Date.now() / 1000)
    const amount = reqBody.amount
    const currency = 'UAH'
    const productName = ['Замовлення RoyalTick']
    const productPrice = [amount]
    const productCount = [1]

    const amountString = amount.toString()

    const signatureString = [
      merchantAccount,
      'www.market.ua',
      orderReference,
      orderDate,
      amountString,

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				136
Змн.	Арк.	№ докум.	Підпис	Дата		


```

        currency,
        productName[0],
        productCount[0],
        productPrice[0].toString()
    ].join(';');

    const merchantSignature = crypto
        .createHmac('md5', merchantSecretKey)
        .update(signatureString)
        .digest('hex');

    const paymentData = {
        merchantAccount,
        merchantDomainName: 'www.market.ua',
        orderReference,
        orderDate,
        amount,
        currency,
        orderTimeout: 49000,
        productName,
        productPrice,
        productCount,
        clientFirstName: reqBody.orderDetails?.name_label || '',
        clientLastName: reqBody.orderDetails?.surname_label || '',
        clientEmail: reqBody.orderDetails?.email_label || '',
        clientPhone: reqBody.orderDetails?.phone_label || '',
        returnUrl: `${baseUrl}/api/success-callback`,
        serviceUrl: `${baseUrl}/api/success-callback`,
        merchantSignature,
        language: 'UA',
    }

    return NextResponse.json({ result: paymentData }, { status: 200 })
} catch (error) {
    console.error('Payment route error:', error)
    return NextResponse.json({ message: (error as Error).message }, { status:
500 })
}
}

```

Лістинг роута видалення теми topics/[id]/route.ts:

```

import { NextResponse } from 'next/server'
import { corsHeaders } from '@/constants/corsHeaders'
import clientPromise from '@/lib/mongodb'
import { getDbAndReqBody, isValidAccessToken, parseJwt, findUserByEmail } from
'@/lib/utils/api-routes'
import { ObjectId } from 'mongodb'

export async function GET(
    req: Request,

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		137

```

    { params }: { params: Promise<{ id: string }> }
  ) {
    try {
      const { id } = await params
      const { db } = await getDbAndReqBody(clientPromise, null)

      const topic = await db.collection('topics').findOne({ _id: new
ObjectId(id) })

      if (!topic) {
        return NextResponse.json({ message: 'Тему не знайдено', status: 404
}, corsHeaders)
      }

      return NextResponse.json({ status: 200, topic }, corsHeaders)
    } catch (error) {
      return NextResponse.json({ message: (error as Error).message, status: 500
}, corsHeaders)
    }
  }

export async function DELETE(
  req: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const token = req.headers.get('authorization')?.split(' ')[1]
    const validatedTokenResult = await isValidAccessToken(token)
    if (validatedTokenResult.status !== 200) {
      return NextResponse.json(validatedTokenResult, corsHeaders)
    }

    const { db, reqBody } = await getDbAndReqBody(clientPromise, req)
    const moderator = await findUserByEmail(db, parseJwt(token as
string).email)

    if (!moderator || ![ 'moderator', 'admin' ].includes(moderator.role)) {
      return NextResponse.json({ message: 'Доступ заборонено', status: 403
}, corsHeaders)
    }

    const topic = await db.collection('topics').findOne({ _id: new
ObjectId(id) })
    if (!topic) {
      return NextResponse.json({ message: 'Тему не знайдено', status: 404
}, corsHeaders)
    }

    await db.collection('topics').deleteOne({ _id: new ObjectId(id) })
  }
}

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				138
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    await db.collection('notifications').insertOne({
      userId: topic.userId,
      type: 'topic_deleted',
      actorName: moderator.name,
      topicTitle: topic.title,
      reason: reqBody?.reason || null,
      punishment: reqBody?.punishment || null,
      isRead: false,
      createdAt: new Date(),
      href: '/community',
    })

    return NextResponse.json({ status: 200 }, corsHeaders)
  } catch (error) {
    return NextResponse.json({ message: (error as Error).message, status: 500
  }, corsHeaders)
  }
}

```

export const dynamic = 'force-dynamic'

Лістинг useBreadcrumbs.ts:

```

'use client'
import { useCallback, useEffect, useState } from 'react'
import { usePageTitle } from './usePageTitle'
import { useCrumbText } from './useCrumbText'
import { useLang } from './useLang'
import { usePathname } from 'next/navigation'

export const useBreadcrumbs = (page: string) => {
  const [dynamicTitle, setDynamicTitle] = useState('')
  const { lang, translations } = useLang()
  const pathname = usePathname()
  const { crumbText } = useCrumbText(page)
  const [isMounted, setIsMounted] = useState(false)

  const [breadcrumbs, setBreadcrumbs] = useState<HTMLElement | null>(null)

  useEffect(() => {
    setIsMounted(true)
    setBreadcrumbs(document.querySelector('.breadcrumbs') as HTMLElement)
  }, [pathname])

  const getDefaultTextGenerator = useCallback(() => crumbText, [crumbText])

  const getTextGenerator = useCallback((param: string) => {
    const breadcrumbsTranslations = translations[lang].breadcrumbs as
    Record<string, string>

```

```

const translation = breadcrumbsTranslations[param]
if (translation) return translation

if (param === 'catalog') return breadcrumbsTranslations.catalog

return crumbText
}, [lang, translations, crumbText])

usePageTitle(page, dynamicTitle)

useEffect(() => {
  if (typeof window === 'undefined' || !isMounted) return

  const pathParts = pathname.split('/').filter(Boolean)
  const lastPart = pathParts[pathParts.length - 1]

  const breadcrumbsTranslations = translations[lang].breadcrumbs as
Record<string, string>
  const text = breadcrumbsTranslations[lastPart] || crumbText

  setDynamicTitle(text)

  const lastCrumb = document.querySelector('.last-crumb') as HTMLElement
  if (lastCrumb) {
    lastCrumb.textContent = text
  }
}, [crumbText, lang, pathname, translations, page, isMounted])

return { getDefaultTextGenerator, getTextGenerator, breadcrumbs }
}

```

Лістинг useLotBid.ts:

```

'use client'
import { useState } from 'react'
import toast from 'react-hot-toast'
import { isUserAuth, handleOpenAuthModal, addOverflowHiddenToBody } from
'@/lib/utils/common'
import { useLang } from '@/hooks/useLang'
import { ILot } from '@/types/lots'
import { openVerificationModal } from '@/context/modals'
import { $user } from '@/context/user/state'
import { useUnit } from 'effector-react'

export const useLotBid = (
  lotId: string,
  lot: ILot | null,
  setLot: (lot: ILot) => void
) => {
  const { lang, translations } = useLang()
  const t = (translations[lang] as any).auction

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		140

```

const currentUser = useUnit($user)
const [bidAmount, setBidAmount] = useState(0)
const [bidSpinner, setBidSpinner] = useState(false)

const initBidAmount = (currentPrice: number, bidStep: number) => {
  setBidAmount(currentPrice + bidStep)
}

const handleBid = async () => {
  if (!isUserAuth()) { handleopenAuthModal(); return }
  if (!lot) return

  if (!currentUser?.isVerified) {
    addOverflowHiddenToBody()
    openVerificationModal()
    return
  }

  const auth = JSON.parse(localStorage.getItem('auth') as string)
  setBidSpinner(true)

  try {
    const res = await fetch(`/api/auction/lots/${lotId}/bid`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${auth.accessToken}`,
      },
      body: JSON.stringify({ bidAmount }),
    })

    const data = await res.json()
    if (data.status === 200) {
      setLot(data.lot)
      setBidAmount(data.lot.currentPrice + data.lot.bidStep)
      toast.success(t.bid_success)
    } else {
      toast.error(data.message)
    }
  } catch {
    toast.error(t.error_generic)
  } finally {
    setBidSpinner(false)
  }
}

return { bidAmount, setBidAmount, bidSpinner, handleBid, initBidAmount }
}

```

Лістинг TopicPage.tsx:

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				141
Змн.	Арк.	№ докум.	Підпис	Дата		

```

'use client'
import { useEffect, useRef, useState } from 'react'
import { useParams, useRouter } from 'next/navigation'
import { useUnit } from 'effector-react'
import { $user } from '@context/user/state'
import { useLang } from '@hooks/useLang'
import { useBreadcrumbs } from '@hooks/useBreadcrumbs'
import { isUserAuth, handleOpenAuthModal } from '@lib/utils/common'
import { ITopic } from '@types/community'
import Link from 'next/link'
import styles from '@styles/community/index.module.scss'
import { FontAwesomeIcon } from '@fortawesome/react-fontawesome'
import { faSpinner, faTrash } from '@fortawesome/free-solid-svg-icons'
import Breadcrumbs from '@components/modules/Breadcrumbs/Breadcrumbs'
import { useModeratorDelete } from '@hooks/useModeratorDelete'
import ModeratorDeleteModal from
'@/components/modules/CommunityPage/ModeratorDeleteModal'

const TopicPage = () => {
  const { getDefaultTextGenerator, getTextGenerator } = useBreadcrumbs('topic')
  const params = useParams()
  const router = useRouter()
  const user = useUnit($user) as any
  const { lang, translations } = useLang()
  const t = (translations[lang] as any).community
  const messagesEndRef = useRef<HTMLDivElement>(null)
  const isModerator = ['moderator', 'admin'].includes(user?.role)

  const [topic, setTopic] = useState<ITopic | null>(null)
  const [spinner, setSpinner] = useState(true)
  const [text, setText] = useState('')
  const [sendSpinner, setSendSpinner] = useState(false)
  const [likeSpinner, setLikeSpinner] = useState(false)
  const [isLiked, setIsLiked] = useState(false)
  const [likesCount, setLikesCount] = useState(0)
  const [replyTo, setReplyTo] = useState<{ id: string; userName: string } |
null>(null)
  const textareaRef = useRef<HTMLTextAreaElement>(null)

  const { isModalOpen, isDeleting, target, openDeleteModal, closeDeleteModal,
handleDelete } =
    useModeratorDelete((data) => {
      if (data?.topic) {
        setTopic(data.topic)
      }
    })

  useEffect(() => {
    if (!params.id) return

```

```

const fetchTopic = async () => {
  try {
    const res = await fetch(`/api/community/topics/${params.id}`)
    const data = await res.json()
    if (data.status === 200) {
      setTopic(data.topic)
      setLikesCount(data.topic.likes?.length ?? 0)
      if (user?._id) {
        setIsLiked(
          data.topic.likes?.some((l: any) => String(l) ===
String(user._id))
        )

        fetch(`/api/community/topics/${params.id}/view`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ userId: user._id }),
        })
          .then(r => r.json())
          .then(d => {
            if (d.views !== undefined) {
              setTopic((prev: any) =>
                prev ? { ...prev, views: d.views } : prev
            )
          })
          .catch(console.error)
      }
    } else {
      router.push('/community')
    }
  } catch (error) {
    console.error(error)
  } finally {
    setSpinner(false)
  }
}

fetchTopic()
}, [params.id, user?._id])

useEffect(() => {
  if (topic?.title) {
    const lastCrumb = document.querySelector('.last-crumb') as
HTMLElement
    if (lastCrumb) lastCrumb.textContent = topic.title
  }
}, [topic?.title])

useEffect(() => {

```

```

        messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' })
    }, [topic?.messages?.length])

    const handleSend = async () => {
        if (!text.trim()) return
        if (!isUserAuth()) { handleOpenAuthModal(); return }
        const auth = JSON.parse(localStorage.getItem('auth') as string)
        setSendSpinner(true)
        try {
            const res = await fetch(`/api/community/topics/${params.id}/message`,
{
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                Authorization: `Bearer ${auth.accessToken}`,
            },
            body: JSON.stringify({
                text,
                replyToId: replyTo?.id || null,
                replyToUserName: replyTo?.userName || null,
            }),
        })
            const data = await res.json()
            if (data.status === 200) {
                setTopic(data.topic)
                setText('')
                setReplyTo(null)
            }
        } catch (error) {
            console.error(error)
        } finally {
            setSendSpinner(false)
        }
    }

    const handleReply = (msgId: string, userName: string) => {
        setReplyTo({ id: msgId, userName })
        textareaRef.current?.focus()
    }

    const handleLike = async () => {
        if (!isUserAuth()) { handleOpenAuthModal(); return }
        const auth = JSON.parse(localStorage.getItem('auth') as string)
        setLikeSpinner(true)
        try {
            const res = await fetch(`/api/community/topics/${params.id}/like`, {
                method: 'POST',
                headers: { Authorization: `Bearer ${auth.accessToken}` },
            })
            const data = await res.json()

```

		Мінайленко В. Ю.			ІІЗ.КР.Б – 121 – 26 – ІІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		144


```

        if (data.status === 200) {
            setIsLiked(data.isLiked)
            setLikesCount(data.likesCount)
        }
    } catch (error) {
        console.error(error)
    } finally {
        setLikeSpinner(false)
    }
}

const formatDate = (dateStr: string) =>
    new Date(dateStr).toLocaleString(
        lang === 'ua' ? 'uk-UA' : 'en-US',
        { day: '2-digit', month: 'long', year: 'numeric', hour: '2-digit',
minute: '2-digit' }
    )

    if (spinner) {
        return (
            <main>
                <section style={{ display: 'flex', justifyContent: 'center',
padding: '80px 0' }}>
                    <FontAwesomeIcon icon={faSpinner} spin size='2x'
color='#b8973f' />
                </section>
            </main>
        )
    }

    if (!topic) return null

    return (
        <main>
            <Breadcrumbs
                getDefaultTextGenerator={getDefaultTextGenerator}
                getTextGenerator={getTextGenerator}
            />
            <section className={styles.topic}>
                <div className='container'>

                    <div className={styles.topic__back}>
                        <Link href='/community'
className={styles.topic__back_link}>
                            {t.back}
                        </Link>
                        <span
className={styles.topic__category}>{topic.category}</span>
                    </div>

```

```

<div className={styles.topic__header}>
  <h1 className={styles.topic__title}>{topic.title}</h1>
  <div className={styles.topic__author}>
    <div className={styles.topic__author_avatar}>
      <img src={topic.userImage || '/img/no-image.jpg'}
alt={topic.userName} />
    </div>
    <Link
      href={String(topic.userId) === String(user?._id)
? '/profile' : `/user/${topic.userId}`}
      className={styles.topic__author_name}
    >
      {topic.userName}
    </Link>
    <span
className={styles.topic__author_date}>{formatDate(topic.createdAt)}</span>
    </div>
  </div>

  <div className={styles.topic__body}>
    <p>{topic.body}</p>
  </div>

  {(topic as any).photoUrls?.length > 0 && (
    <div className={styles.topic__photos}>
      {(topic as any).photoUrls.map((url: string, i:
number) => (
        <div key={i} className={styles.topic__photo}>
          <img src={url} alt={`photo-${i}`} />
        </div>
      )))}
    </div>
  )}

  {topic.tags?.length > 0 && (
    <div className={styles.topic__tags}>
      <span
className={styles.topic__tags_label}>{t.tags_label}</span>
      {topic.tags.map((tag, i) => (
        <span key={i}
className={styles.topic__tag}>{tag}</span>
      )))}
    </div>
  )}

  <div className={styles.topic__stats}>
    <span>👁 {topic.views}</span>
    <span>💬 {topic.messages?.length ?? 0}</span>
    <button

```

```

        className={`btn-reset ${styles.topic__like_btn}
${isLiked ? styles.topic__like_btn_active : ''}`}
        onClick={handleLike}
        disabled={likeSpinner}
      >
        {likeSpinner ? '...' : `👍 ${likesCount}`}
      </button>
    </div>

    <div className={styles.topic__messages}>
      <h2 className={styles.topic__messages_title}>
        {t.messages_title} {topic.messages?.length ?? 0}
      </h2>

      {topic.messages?.map((msg: any) => {
        const isMine = String(msg.userId) ===
String(user?._id)
        const isAuthor = String(msg.userId) ===
String(topic.userId)

        return (
          <div key={String(msg._id)}
className={styles.topic__message}>
            <div
className={styles.topic__message_avatar}>
              <img src={msg.userImage || '/img/no-
image.jpg'} alt={msg.userName} />
            </div>
            <div className={styles.topic__message_body}>
              <div
className={styles.topic__message_header}>
                <Link
                  href={isMine ? '/profile' :
`/user/${msg.userId}`}
                  className={styles.topic__message_
author}>
                  >
                    {msg.userName}
                </Link>
                {isAuthor && (
                  <span
className={styles.topic__message_badge_author}>
                    {t.badge_author || 'Автор'}
                  </span>
                )}
                {msg.replyToUserName && (
                  <span
className={styles.topic__message_reply_badge}>
                    🗨️ {msg.replyToUserName}
                  </span>
                )}
              </div>
            </div>
          </div>
        )
      })}
    </div>
  )
}

```

		Мінайленко В. Ю.			ІІЗ.КР.Б – 121 – 26 – ІІЗ	Арк.
		Єфремов Ю.М.				147
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    ))
    <span
      className={styles.topic__message_date}>
        {formatDate(msg.createdAt)}
      </span>
      {isModerator && (
        <button
          className={`btn-reset
            ${styles.topic__message_delete_btn}`}
          onClick={() =>
            openDeleteModal({
              type: 'message',
              topicId:
                String(params.id),
              msgId: String(msg._id),
              topicTitle: topic.title,
            })
            title={t.delete_message ||
              'Видалити коментар'}
          >
            <FontAwesomeIcon
              icon={faTrash} />
            </button>
          </div>
        <p
          className={styles.topic__message_text}>{msg.text}</p>
          {isUserAuth() && (
            <button
              className={`btn-reset
                ${styles.topic__message_reply_btn}`}
              onClick={() =>
                handleReply(String(msg._id), msg.userName)}
            >
               {t.reply_to || 'Відповісти'}
            </button>
          </div>
        </div>
      )
    )}
  <div ref={messagesEndRef} />
</div>

<div className={styles.topic__reply}>
  {isUserAuth() ? (
    <>
      {replyTo && (
        <div className={styles.topic__reply_to}>
          <span>

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		148

```

    {t.replying_to || 'Відповідь'}:{'
    <strong>{replyTo.userName}</strong>
    </span>
    <button
      className='btn-reset'
      onClick={() => setReplyTo(null)}
      style={{ marginLeft: 8, opacity: 0.5,
cursor: 'pointer', fontSize: 12 }}
    >
      ×
    </button>
  </div>
)}
<textarea
  ref={textareaRef}
  className={styles.topic__reply_input}
  placeholder={t.reply_placeholder}
  value={text}
  onChange={e => setText(e.target.value)}
/>
<button
  className={`btn-reset
  ${styles.topic__reply_btn}`}
  onClick={handleSend}
  disabled={sendSpinner || !text.trim()}
>
  {sendSpinner
    ? <FontAwesomeIcon icon={faSpinner} spin
    : t.reply_btn
  }
</button>
</>
) : (
  <p className={styles.topic__reply_hint}>
    <button
      className='btn-reset'
      onClick={handleopenAuthModal}
      style={{ color: '#a78bfa', cursor: 'pointer',
textDecoration: 'underline' }}
    >
      {t.login_hint_link}
    </button>
    {' '}{t.login_hint_text}
  </p>
)}
</div>

</div>

```

```

        </section>

        <ModeratorDeleteModal
            isOpen={isModalOpen}
            onClose={closeDeleteModal}
            onConfirm={handleDelete}
            isLoading={isDeleting}
            type={target?.type || 'message'}
            title={target?.topicTitle}
        />
    </main>
)
}

```

export default TopicPage

Лістинг ModeratorStatusPanel.tsx:

```

import { useState } from 'react'
import { useLang } from '@hooks/useLang'
import styles from '@styles/auction/index.module.scss'
import { FontAwesomeIcon } from '@fortawesome/react-fontawesome'
import { faSpinner } from '@fortawesome/free-solid-svg-icons'
import toast from 'react-hot-toast'

const STATUSES = [
    { value: 'active', label: '● Активний', color: '#52b788' },
    { value: 'reserved', label: '● Резерв', color: '#fbbf24' },
    { value: 'completed', label: '● Завершений', color: '#6b7280' },
]

interface IProps {
    lotId: string
    currentStatus: string
    onStatusChange: (newStatus: string) => void
}

const ModeratorStatusPanel = ({ lotId, currentStatus, onStatusChange }: IProps) => {
    const { lang, translations } = useLang()
    const t = (translations[lang] as any).auction
    const [spinner, setSpinner] = useState(false)
    const [selectedStatus, setSelectedStatus] = useState(currentStatus)

    const handleChange = async () => {
        if (selectedStatus === currentStatus) return
        const auth = JSON.parse(localStorage.getItem('auth') as string)
        setSpinner(true)
        try {
            const res = await fetch(`/api/auction/lots/${lotId}/status`, {
                method: 'POST',
            })

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		150

```

        headers: {
            'Content-Type': 'application/json',
            Authorization: `Bearer ${auth.accessToken}`,
        },
        body: JSON.stringify({ status: selectedStatus }),
    })
    const data = await res.json()
    if (data.status === 200) {
        onChange(selectedStatus)
        toast.success(t.mod_status_changed || 'Статус змінено')
    } else {
        toast.error(data.message)
    }
} catch (error) {
    console.error(error)
} finally {
    setSpinner(false)
}
}

return (
    <div className={styles.lot_page__mod_panel}>
        <span className={styles.lot_page__mod_panel__label}>
            🛡️ {t.mod_change_status || 'Змінити статус'}
        </span>
        <div className={styles.lot_page__mod_panel__row}>
            <select
                className={styles.lot_page__mod_panel__select}
                value={selectedStatus}
                onChange={e => setSelectedStatus(e.target.value)}
            >
                {STATUSES.map(s => (
                    <option key={s.value} value={s.value}>{s.label}</option>
                ))}
            </select>
            <button
                className={`btn-reset ${styles.lot_page__mod_panel__btn}`}
                onClick={handleChange}
                disabled={spinner || selectedStatus === currentStatus}
            >
                {spinner
                    ? <FontAwesomeIcon icon={faSpinner} spin />
                    : t.mod_apply || 'Застосувати'}
            </button>
        </div>
    </div>
)
}

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		151

```
export default ModeratorStatusPanel
```

Лістинг Menu.tsx:

```
'use client'
import Logo from '@components/elements/Logo/Logo'
import { AllowedLangs } from '@constants/lang'
import { setLang } from '@context/lang/index'
import { closeMenu } from '@context/modals/index'
import { $isMainMenuOpen } from '@context/modals/state'
import { useLang } from '@hooks/useLang'
import { removeOverflowHiddenFromBody } from '@lib/utils/common'
import { useUnit } from 'effector-react'
import { AnimatePresence, motion } from 'framer-motion'
import { useState } from 'react'
import { usePathname } from 'next/navigation'
import MenuItem from '../MenuItem'
import Accordion from '../Accordion/Accordion'
import { useMediaQuery } from '@hooks/useMediaQuery'
import BuyersListItems from '../BuyersListItems'
import ContactsListItems from '../ContactsListItems'
import AuctionListItems from '../AuctionListItems'
import Link from 'next/link'

const Menu = () => {
  const [activelistId, setActiveListId] = useState(0)
  const menuIsOpen = useUnit($isMainMenuOpen)
  const { lang, translations } = useLang()
  const pathname = usePathname()

  const isMedia800 = useMediaQuery(800)
  const isMedia640 = useMediaQuery(640)

  const handleSwitchLang = (lang: string) => {
    setLang(lang as AllowedLangs)
    localStorage.setItem('lang', JSON.stringify(lang))
  }

  const handleSwitchLangToUa = () => handleSwitchLang('ua')
  const handleSwitchLangToEn = () => handleSwitchLang('en')

  const handleShowCatalogList = () => setActiveListId(1)
  const handleShowBuyersList = () => setActiveListId(2)
  const handleShowContactsList = () => setActiveListId(3)
  const handleShowAuctionList = () => setActiveListId(4)

  const handleCloseMenu = () => {
    removeOverflowHiddenFromBody()
    closeMenu()
  }
}
```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Євдокимов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		152


```

        setActiveListId(0)
    }

    const handleRedirectToCatalog = (path: string) => {
        if (pathname.includes('/catalog')) {
            window.history.pushState({ path }, '', path)
            window.location.reload()
        }
        handleCloseMenu()
    }

    const watchesLinks = [
        { id: 1, text: translations[lang].comparison.classic, href:
        '/catalog/watches?offset=0&type=classic' },
        { id: 2, text: translations[lang].comparison.sport, href:
        '/catalog/watches?offset=0&type=sport' },
        { id: 3, text: translations[lang].comparison.premium, href:
        '/catalog/watches?offset=0&type=premium' },
        { id: 4, text: translations[lang].comparison.line, href:
        '/catalog/watches?offset=0&type=line' },
    ]

    const strapsLinks = [
        { id: 1, text: translations[lang].comparison.leather_strap, href:
        '/catalog/straps?offset=0&type=leather_strap' },
        { id: 2, text: translations[lang].comparison.metal_bracelet, href:
        '/catalog/straps?offset=0&type=metal_bracelet' },
        { id: 3, text: translations[lang].comparison.nato_strap, href:
        '/catalog/straps?offset=0&type=nato_strap' },
        { id: 4, text: translations[lang].comparison.rubber_strap, href:
        '/catalog/straps?offset=0&type=rubber_strap' },
        { id: 5, text: translations[lang].comparison.mesh_bracelet, href:
        '/catalog/straps?offset=0&type=mesh_bracelet' },
    ]

    const boxesLinks = [
        { id: 1, text: translations[lang].comparison.bboxes, href:
        '/catalog/bboxes?offset=0&type=bboxes' },
    ]

    const careLinks = [
        { id: 1, text: translations[lang].comparison.basic, href:
        '/catalog/care?offset=0&type=basic' },
        { id: 2, text: translations[lang].comparison.professional, href:
        '/catalog/care?offset=0&type=professional' },
    ]

    return (
        <nav className={`nav-menu ${menuIsOpen ? 'open' : 'close'}`}>
            <div className='container nav-menu__container'>

```

```

<div className={`nav-menu__logo ${menuIsOpen ? 'open' : ''}`}>
  <Logo />
</div>
<img
  className={`nav-menu__bg ${menuIsOpen ? 'open' : ''}`}
  src={` /img/menu-bg${isMedia640 ? '-small' : ''}.svg`}
  alt='menu background'
/>
<button
  className={`btn-reset nav-menu__close ${menuIsOpen ? 'open' : ''}`}
  onClick={handleCloseMenu}
/>
<div className={`nav-menu__lang ${menuIsOpen ? 'open' : ''}`}>
  <button
    className={`btn-reset nav-menu__lang__btn ${lang === 'ua' ? 'lang-
active' : ''}`}
    onClick={handleSwitchLangToUa}
  >UA</button>
  <button
    className={`btn-reset nav-menu__lang__btn ${lang === 'en' ? 'lang-
active' : ''}`}
    onClick={handleSwitchLangToEn}
  >EN</button>
</div>

<ul className={`list-reset nav-menu__list ${menuIsOpen ? 'open' : ''}`}>

  {!isMedia800 && (
    <li className='nav-menu__list__item'>
      <button
        className='btn-reset nav-menu__list__item__btn'
        onMouseEnter={handleShowCatalogList}
      >
        {translations[lang].main_menu.catalog}
      </button>
      <AnimatePresence>
        {activelistId === 1 && (
          <motion.ul
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className='list-reset nav-menu__accordion'
          >
            <li className='nav-menu__accordion__item'>
              <Accordion
                title={translations[lang].main_menu.watches}
                titleClass='btn-reset nav-menu__accordion__item__title'
              >
                <ul className='list-reset nav-
menu__accordion__item__list'>

```

```

        {watchesLinks.map((item) => (
            <MenuItem key={item.id} item={item}
handleRedirectToCatalog={handleRedirectToCatalog} />
        ))}
    </ul>
</Accordion>
</li>
<li className='nav-menu__accordion_item'>
    <Accordion
        title={translations[lang].main_menu.straps}
        titleClass='btn-reset nav-menu__accordion_item_title'
    >
        <ul className='list-reset nav-
menu__accordion_item_list'>
            {strapsLinks.map((item) => (
                <MenuItem key={item.id} item={item}
handleRedirectToCatalog={handleRedirectToCatalog} />
            ))}
        </ul>
    </Accordion>
</li>
<li className='nav-menu__accordion_item'>
    <Accordion
        title={translations[lang].main_menu.bboxes}
        titleClass='btn-reset nav-menu__accordion_item_title'
    >
        <ul className='list-reset nav-
menu__accordion_item_list'>
            {bboxesLinks.map((item) => (
                <MenuItem key={item.id} item={item}
handleRedirectToCatalog={handleRedirectToCatalog} />
            ))}
        </ul>
    </Accordion>
</li>
<li className='nav-menu__accordion_item'>
    <Accordion
        title={translations[lang].main_menu.care}
        titleClass='btn-reset nav-menu__accordion_item_title'
    >
        <ul className='list-reset nav-
menu__accordion_item_list'>
            {careLinks.map((item) => (
                <MenuItem key={item.id} item={item}
handleRedirectToCatalog={handleRedirectToCatalog} />
            ))}
        </ul>
    </Accordion>
</li>
</motion.ul>

```

```

    })
    </AnimatePresence>
  </li>
})

<li className='nav-menu__list__item'>
  {!isMedia640 ? (
    <>
      <button
        className='btn-reset nav-menu__list__item_btn'
        onMouseEnter={handleShowBuyersList}
      >
        {translations[lang].main_menu.buyers}
      </button>
      <AnimatePresence>
        {activelistId === 2 && (
          <motion.ul
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className='list-reset nav-menu__accordion'
          >
            <BuyersListItems />
          </motion.ul>
        )}
      </AnimatePresence>
    </>
  ) : (
    <Accordion
      title={translations[lang].main_menu.buyers}
      titleClass='btn-reset nav-menu__list__item_btn'
    >
      <ul className='list-reset nav-menu__accordion__item__list'>
        <BuyersListItems />
      </ul>
    </Accordion>
  )}
</li>

<li className='nav-menu__list__item'>
  {!isMedia640 ? (
    <>
      <button
        className='btn-reset nav-menu__list__item_btn'
        onMouseEnter={handleShowContactsList}
      >
        {translations[lang].main_menu.contacts}
      </button>
      <AnimatePresence>
        {activelistId === 3 && (

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		156

```

        <motion.ul
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          exit={{ opacity: 0 }}
          className='list-reset nav-menu__accordion'
        >
          <ContactsListItems />
        </motion.ul>
      )}
    </AnimatePresence>
  </>
) : (
  <Accordion
    title={translations[lang].main_menu.contacts}
    titleClass='btn-reset nav-menu__list__item__btn'
  >
    <ul className='list-reset nav-menu__accordion__item__list'>
      <ContactsListItems />
    </ul>
  </Accordion>
)}
</li>

<li className='nav-menu__list__item'>
  {!isMedia640 ? (
    <>
      <button
        className='btn-reset nav-menu__list__item__btn'
        onMouseEnter={handleShowAuctionList}
      >
        {translations[lang].main_menu.auction}
      </button>
      <AnimatePresence>
        {activelistId === 4 && (
          <motion.ul
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className='list-reset nav-menu__accordion'
          >
            <AuctionListItems handleCloseMenu={handleCloseMenu} />
          </motion.ul>
        )}
      </AnimatePresence>
    </>
  ) : (
    <Accordion
      title={translations[lang].main_menu.auction}
      titleClass='btn-reset nav-menu__list__item__btn'
    >

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІПЗ	Арк.
		Єфремов Ю.М.				157
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        <ul className='list-reset nav-menu__accordion__item__list'>
          <AuctionListItems handleCloseMenu={handleCloseMenu} />
        </ul>
      </Accordion>
    )}
  </li>

  </ul>
</div>
</nav>
)
}

```

export default Menu

Лістинг LotPage.tsx:

```

'use client'
import Breadcrumbs from '@components/modules/Breadcrumbs/Breadcrumbs'
import { useBreadcrumbs } from '@hooks/useBreadcrumbs'
import { useLang } from '@hooks/useLang'
import { formatPrice, isUserAuth } from '@lib/utils/common'
import { useParams, useRouter } from 'next/navigation'
import { useEffect, useState } from 'react'
import toast from 'react-hot-toast'
import styles from '@styles/auction/index.module.scss'
import { handleopenAuthModal } from '@lib/utils/common'
import { ILot } from '@types/lots'
import { $user } from '@context/user/state'
import { useUnit } from 'effector-react'
import Link from 'next/link'
import { useCountdown } from '@hooks/useCountdown'
import { useLotBid } from '@hooks/useLotBid'
import { useLotComments } from '@hooks/useLotComments'
import ModeratorStatusPanel from
'@components/modules/LotPage/ModeratorStatusPanel'
import BuyNowModal from '@components/modules/LotPage/BuyNowModal'
import LotComments from '@components/modules/LotPage/LotComments'
import LotSellerLots from '@components/modules/LotPage/LotSellerLots'
import LotGallery from '@components/modules/LotPage/LotGallery'

const LotPage = () => {
  const { lang, translations } = useLang()
  const t = (translations[lang] as any).auction
  const { getDefaultTextGenerator, getTextGenerator } =
useBreadcrumbs('auction')
  const params = useParams()
  const router = useRouter()
  const user = useUnit($user) as any
  const isModerator = user?.role === 'moderator' || user?.role === 'admin'

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		158

```

const [lot, setLot] = useState<ILot | null>(null)
const [spinner, setSpinner] = useState(true)
const [activeImg, setActiveImg] = useState('')
const [activeTab, setActiveTab] = useState<'description' |
'photos'>('description')
const [showBuyNowModal, setShowBuyNowModal] = useState(false)
const [buyNowConfirmed, setBuyNowConfirmed] = useState(false)
const [buyNowSpinner, setBuyNowSpinner] = useState(false)
const [sellerLots, setSellerLots] = useState<any[]>([])

useEffect(() => {
  if (lot?.title) {
    const lastCrumb = document.querySelector('.last-crumb') as
HTMLInputElement
    if (lastCrumb) lastCrumb.textContent = lot.title
  }
}, [lot?.title])

const timeLeft = useCountdown(lot?.endDate || '')

const { bidAmount, setBidAmount, bidSpinner, handleBid, initBidAmount } =
useLotBid(
  String(params.id),
  lot,
  setLot as (lot: ILot) => void
)

const {
  comments, commentText, setCommentText,
  commentSpinner, fetchComments, handleComment,
} = useLotComments(String(params.id))

useEffect(() => {
  const fetchLot = async () => {
    try {
      const res = await fetch(`/api/auction/lots/${params.id}`)
      const data = await res.json()
      if (data.status === 200) {
        setLot(data.lot)
        setActiveImg(data.lot.mainPhotoUrl)
        initBidAmount(data.lot.currentPrice, data.lot.bidStep)

        const isExpired = new Date() > new Date(data.lot.endDate)
        if (isExpired && data.lot.status === 'active') {
          await fetch('/api/auction/lots/finalize', { method:
'POST' })

          const refreshed = await
fetch(`/api/auction/lots/${params.id}`)
          const refreshedData = await refreshed.json()

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		159

```

        if (refreshedData.status === 200)
setLot(refreshedData.lot)
    }
    } else {
        router.push('/auction')
    }
} catch (error) {
    console.error(error)
} finally {
    setSpinner(false)
}
}
if (params.id) fetchLot()
}, [params.id])

useEffect(() => {
    if (!lot?.userId) return
    const fetchSellerLots = async () => {
        try {
            const res = await
fetch(`/api/auction/lots?userId=${lot.userId}&limit=10`)
            const data = await res.json()
            if (data.status === 200) {
                setSellerLots(data.lots.filter((l: any) => l._id !==
lot._id))
            }
        } catch (error) {
            console.error(error)
        }
    }
    fetchSellerLots()
}, [lot?.userId])

useEffect(() => {
    if (params.id) fetchComments()
}, [params.id])

const isOwner = lot && user?._id && String(lot.userId) === String(user._id)
const isExpired = lot && new Date() > new Date(lot.endDate)
const canBid = !isOwner && !isExpired && isUserAuth()
const isHighestBidder = !(lot?.bids?.length && lot.bids.length > 0 &&
String(lot.bids[lot.bids.length - 1].userId) === String(user?._id))

const handleBuyNow = async () => {
    if (!lot) return
    setBuyNowSpinner(true)
    const auth = JSON.parse(localStorage.getItem('auth') as string)
    try {
        const res = await fetch('/api/chats', {
            method: 'POST',

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІІЗ	Арк.
		Єдземов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		160


```

        headers: {
            'Content-Type': 'application/json',
            Authorization: `Bearer ${auth.accessToken}`,
        },
        body: JSON.stringify({ lotId: lot._id }),
    })
    const data = await res.json()
    if (data.status === 201 || data.status === 200) {
        setShowBuyNowModal(false)
        router.push(`/chats/${data.chat._id}`)
    }
} catch (error) {
    console.error(error)
    toast.error(t.error_generic)
} finally {
    setBuyNowSpinner(false)
}
}

const handleStatusChange = (newStatus: string) => {
    setLot((prev: any) => prev ? { ...prev, status: newStatus } : prev)
}

const allImages = lot ? [lot.mainPhotoUrl,
...lot.additionalPhotoUrls].filter(Boolean) : []

const formatDate = (dateStr: string) =>
    new Date(dateStr).toLocaleString(lang === 'ua' ? 'uk-UA' : 'en-US', {
        day: '2-digit', month: '2-digit', year: 'numeric',
        hour: '2-digit', minute: '2-digit',
    })

if (spinner) {
    return (
        <main>
            <section className={styles.lot_page}>
                <div className='container'>
                    <div className={styles.lot_page__skeleton} />
                </div>
            </section>
        </main>
    )
}

if (!lot) return null

return (
    <main>
        <Breadcrumbs
            getDefaultTextGenerator={getDefaultTextGenerator}

```

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Євремів Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		161

```

        getTextGenerator={getTextGenerator}
    />
    <section className={styles.lot_page}>
        <div className='container'>

            <h1 className={styles.lot_page__title}>{lot.title}</h1>

            <div className={styles.lot_page__meta}>
                <div className={styles.lot_page__meta__item}>
                    <span
className={styles.lot_page__meta__label}>{t.bids_count}</span>
                    <strong>{lot.bids.length}</strong>
                </div>
                <div className={styles.lot_page__meta__item}>
                    <span
className={styles.lot_page__meta__label}>{t.max_bid}</span>
                    <strong>{formatPrice(lot.currentPrice)} ₴</strong>
                </div>
                <div className={styles.lot_page__meta__item}>
                    <span
className={styles.lot_page__meta__label}>{t.added}</span>
                    <strong>{formatDate(lot.createdAt)}</strong>
                </div>
                <div className={styles.lot_page__meta__item}>
                    <span
className={styles.lot_page__meta__label}>{t.seller}</span>
                    <Link href={` /user/${lot.userId}`} style={{ color:
'inherit', textDecoration: 'none', fontWeight: 700 }}>
                        {lot.userName}
                    </Link>
                </div>
            </div>

            <div className={styles.lot_page__body}>

                <LotGallery
                    images={allImages}
                    activeImg={activeImg}
                    lotTitle={lot.title}
                    onThumbClick={setActiveImg}
                />

                <div className={styles.lot_page__sidebar}>

                    <div className={styles.lot_page__timing}>
                        <div>
                            <p
className={styles.lot_page__timing__label}>{t.end_date}</p>
                            <p
className={styles.lot_page__timing__value}>{formatDate(lot.endDate)}</p>

```

```

        </div>
        <div>
            <p
className={styles.lot_page__timing__label}>{t.time_left}</p>
            <p
className={styles.lot_page__timing__countdown}>
                {isExpired ? t.expired : timeLeft}
            </p>
        </div>
    </div>

    {lot.status === 'reserved' && (
        <div className={styles.lot_page__reserved}>
             {t.reserved}
        </div>
    )}

    {isModerator && (
        <ModeratorStatusPanel
            lotId={String(lot._id)}
            currentStatus={lot.status}
            onChange={handleStatusChange}
        />
    )}

    <div className={styles.lot_page__price_block}>
        <span className={styles.lot_page__price_label}>
            {t.current_price} ({lot.bids.length}
{t.bids_word})

        </span>
        <span className={styles.lot_page__price}>
            {formatPrice(lot.currentPrice)} €
        </span>
    </div>

    {isHighestBidder && !isOwner && (
        <div className={styles.lot_page__highest_bid}>
            ☒ {t.your_bid_highest}
        </div>
    )}

    {!isExpired && lot.status !== 'reserved' && (
        <div className={styles.lot_page__bid}>
            <p
className={styles.lot_page__bid__label}>{t.make_bid}</p>
            <div
className={styles.lot_page__bid__input_row}>
                <button
                    className={`btn-reset
                    ${styles.lot_page__bid__step_btn}`}

```

		Мінайленко В. Ю.			ІІЗ.КР.Б – 121 – 26 – ІІЗ	Арк.
		Єфремов Ю.М.				163
Змн.	Арк.	№ докум.	Підпис	Дата		

```

onClick={() => setBidAmount((prev) =>
Math.max(lot.currentPrice + lot.bidStep, prev - lot.bidStep))}
disabled={!canBid}
></button>
<div
className={styles.lot_page__bid__input_wrap}>
<input
type='number'
value={bidAmount}
onChange={(e) =>
setBidAmount(Number(e.target.value))}
disabled={!canBid}
className={styles.lot_page__bid__
input}
/>
<span>€</span>
</div>
<button
className={`btn-reset
${styles.lot_page__bid__step_btn}`}
onClick={() => setBidAmount((prev) =>
prev + lot.bidStep)}
disabled={!canBid}
></button>
</div>
<p
className={styles.lot_page__bid__step_hint}>
{formatPrice(lot.bidStep)} €
</p>
{isOwner ? (
<div
className={styles.lot_page__bid__owner_msg}>
{t.bid_own_lot_error}
</div>
) : (
<button
className={`btn-reset
${styles.lot_page__bid__submit}`}
onClick={handleBid}
disabled={bidSpinner || !canBid}
>
{bidSpinner ? '...' : t.make_bid_btn}
</button>
)}
</div>
)}
{lot.buyNowPrice && !isExpired && !isOwner &&
lot.status !== 'reserved' && (

```

		Мінайленко В. Ю.			ІІЗ.КР.Б – 121 – 26 – ІІЗ	Арк.
		Євремів Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		164

```

<button
  className={`btn-reset
  ${styles.lot_page__buy_now}`}
  onClick={() => {
    if (!isUserAuth()) {
      handleopenAuthModal(); return }
    setBuyNowConfirmed(false)
    setShowBuyNowModal(true)
  }}
  >
    {t.buy_now} – {formatPrice(lot.buyNowPrice)}
  </button>
)}
</div>
</div>

<div className={styles.lot_page__tabs}>
  <button
    className={`btn-reset ${styles.lot_page__tab}
    ${activeTab === 'description' ? styles.lot_page__tab_active : ''}`}
    onClick={() => setActiveTab('description')}
    >{t.tab_description}</button>
  <button
    className={`btn-reset ${styles.lot_page__tab}
    ${activeTab === 'photos' ? styles.lot_page__tab_active : ''}`}
    onClick={() => setActiveTab('photos')}
    >{t.tab_photos} {allImages.length}</button>
</div>

{activeTab === 'description' && (
  <div className={styles.lot_page__description}>
    <table className={styles.lot_page__desc_table}>
      <tbody>
        <tr><td>{t.condition}</td><td>{t[`condition_${
        lot.condition}`]}</td></tr>
        <tr><td>{t.location}</td><td>{lot.location}</
        td></tr>
        <tr>
          <td>{t.delivery_method}</td>
          <td>{lot.deliveryMethods.map((d) =>
            t[`delivery_${d}`]).join(', ')}</td>
        </tr>
        <tr>
          <td>{t.delivery_payer}</td>
          <td>{lot.deliveryPayer === 'buyer' ?
            t.payer_buyer : t.payer_seller}</td>
        </tr>
        <tr><td>{t.returns}</td><td>{lot.returnsAllow
        ed ? t.yes : t.no}</td></tr>
      </tbody>
    </table>
  </div>
)}

```

		Мінайленко В. Ю.			ІІЗ.КР.Б – 121 – 26 – ІІЗ	Арк.
		Євремів Ю.М.				165
Змн.	Арк.	№ докум.	Підпис	Дата		

```

                                {lot.guarantees &&
<tr><td>{t.guarantees}</td><td>{lot.guarantees}</td></tr>}
                                {lot.buyerComment &&
<tr><td>{t.buyer_comment}</td><td>{lot.buyerComment}</td></tr>}
                                </tbody>
                                </table>
                                {lot.description && (
                                <div className={styles.lot_page__desc_text}>
                                <strong>{t.description}</strong>
                                <p>{lot.description}</p>
                                </div>
                                )}
                                </div>
                                )}

                                {activeTab === 'photos' && (
                                <div className={styles.lot_page__photos_grid}>
                                {allImages.map((img, i) => (
                                <div key={i}
                                className={styles.lot_page__photos_grid__item}>
                                <img src={img} alt={`photo-${i}`} />
                                </div>
                                ))}
                                </div>
                                )}

                                <LotSellerLots lots={sellerLots} sellerName={lot.userName} />

                                <LotComments
                                comments={comments}
                                commentText={commentText}
                                commentSpinner={commentSpinner}
                                userId={String(user?._id)}
                                onTextChange={setCommentText}
                                onSubmit={handleComment}
                                />

                                </div>
                                </section>

                                <BuyNowModal
                                show={showBuyNowModal}
                                buyNowPrice={lot.buyNowPrice!}
                                confirmed={buyNowConfirmed}
                                spinner={buyNowSpinner}
                                onClose={() => setShowBuyNowModal(false)}
                                onConfirmChange={setBuyNowConfirmed}
                                onConfirm={handleBuyNow}
                                />
                                </main>

```

)
}

export default LotPage

		Мінайленко В. Ю.			ІПЗ.КР.Б – 121 – 26 – ІЗ	Арк.
		Єфремов Ю.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		167